Contents

SPECIFICATION — Specification Conformance Checker (`speccheck`; traceability graph, JUnit results, model-judged test strength; Python 3.12 + uv) **2**

0. Intent and purpose . . . 7
1. Actors and goals . . . 8
2. Requirements (intent, high level) . . . 9
3. Behavior and state model . . . 18
   - 3.1 Lifecycle . . . 18
   - 3.2 Executable flow . . . 20
   - 3.3 Durable artifacts . . . 21
4. Interfaces / contracts . . . 22
   - C-01 Spec ID grammar, declaration, citation, retirement . . . 22
   - C-02 `SpecIndex` . . . 24
   - C-03 `Citation`, `TestCase`, attribution . . . 25
   - C-04 JUnit XML subset . . . 28
   - C-05 Status algorithm and `IdStatus` . . . 29
   - C-06 Judge interface . . . 30
   - C-07 JSON report (`speccheck.json`, `schema_version` "1.5") . . . 32
   - C-08 Markdown report layout (`SPEC_CONFORMANCE_REPORT.md`) . . . 35
   - C-09 LLM provider configuration . . . 36
   - C-10 Judge instruction text (normative; F-004) . . . 36
   - C-11 Judge progress indicator (R-30) . . . 37
   - C-12 Spec-internal edges (v1.13; R-36) . . . 38
   - C-13 `impact`: changed set, walk, `impact.json`, `IMPACT_REPORT.md` (v1.13; R-37) . . . 39
   - C-14 Declared vs. incidental citations (v1.14; R-39) . . . 41
   - C-15 `declared` in the judge request; the instruction text is advisory (v1.14; R-39) . . . 41
   - C-16 `declared` and `declared_ratio` in the JSON report (v1.14; R-39) . . . 41
   - C-17 Jev triage provider (v1.15; K-16, I-015) . . . 42
   - C-18 `explain` stdout contract (v1.17; R-40) . . . 43
   - C-19 Help contract (v1.18; R-41) . . . 44
   - C-20 The lean adapter (v1.19; R-100) . . . 44
   - C-21 The proof manifest (v1.19; R-100) . . . 44
   - C-22 Foreign reference and namespace (v1.23; R-101, R-102) . . . 45
   - C-23 The citation map (v1.23; R-103) . . . 45
   - C-24 The JavaScript/TypeScript adapter (v1.22; R-104; moved from C-03 in v1.24) . . . 46
   - C-25 The provenance manifest (v1.25; R-106) . . . 47
   - C-26 `verify` (v1.25; R-107) . . . 48
   - C-27 The CONTROLS marker and the hazard roll-up (v1.25; R-108) . . . 48
   - C-28 The risk file (v1.25; R-109) . . . 49
5. Interface specification . . . 49
   - 5.1 CLI (`speccheck`), the only surface . . . 49
   - 5.2 GUI . . . 53
   - 5.3 Diagnostics and verbosity contract . . . 53
   - 5.4 Exit codes . . . 54
6. Invariants (must hold in every valid implementation) . . . 54
7. Constraints (precise and measurable) . . . 57
8. Edge cases and failure semantics . . . 59
9. Acceptance criteria, tests, and evals . . . 70
   - 9.1 Extraction (C-01, C-02) . . . 70
   - 9.2 Citation and attribution (C-03) . . . 71
   - 9.3 Results mapping (C-04) . . . 73
   - 9.4 Status algorithm and metrics (C-05, C-07) . . . 74
   - 9.5 Judge contract (C-06) . . . 74

9.6 Reports and determinism (C-07, C-08) . . . 77
9.7 CLI, exit codes, diagnostics, boundary (§5) . . . 78
9.8 Golden fixture (end to end) . . . 82
9.9 Self-application (recorded, not gating) . . . 83
9.10 Performance (recorded) . . . 84
9.11 LLM judge evaluation (opt-in, `--judge llm`, not run in CI) . . . 84
9.12 Edges and impact (C-12, C-13; v1.13) . . . 85
9.13 `explain` (C-18; v1.17) . . . 87
9.14 CLI help (C-19; v1.18) . . . 88
9.15 Proof evidence (C-20, C-21, K-17, E-62, E-63, E-65; v1.19) . . . 89
9.16 JavaScript/TypeScript adapter (C-03, C-04, E-74, E-75, E-76; v1.22) . . . 89
9.17 Namespaces and the citation map (C-22, C-23, I-019, K-19, E-67..E-73; v1.23) . . . 90
9.18 Provenance manifest and hazards (C-25..C-28, I-020, I-021, K-20, E-78..E-85; v1.25) . . . 92
10. Dependencies and environment . . . 93
11. Traceability matrix (id → where realized) . . . 95
12. Open questions and decisions to confirm . . . 101
Revision history . . . 124

SPECIFICATION — Specification Conformance Checker (`speccheck`; traceability graph, JUnit results, model-judged test strength; Python 3.12 + uv)

- **Status:** v1.25 — the shipped version: the code (1.25.0), `__version__` and the README's claim all say v1.25 (`test_prose_artifacts_claim_the_shipped_version` binds them). Built 2026-11-05 in three waves (`docs/plans/IMPLEMENTATION_PLAN.md`; `SPEC_BUILD_REPORT.md` §0p).
- **v1.25 (built):** a provenance manifest a quality system can file as a record, and hazard-to-control traceability (`docs/proposals/PROPOSAL_v1.25_provenance_manifest_and_risk_controls.md`, 2026-11-05; D-61..D-65 confirmed on the recommended branches, same day; `docs/plans/ISO13485_READINESS_PL` §3.2, §3.3). **(a)** `check --manifest` writes a third file, `speccheck.manifest.json`, binding both reports to the SHA-256 of every file the run read and to the tool version, because a code edit made after the test run left both v1.24 reports byte-identical (R-106, C-25, I-020). **(b)** `speccheck verify` re-hashes what a manifest names and exits `1` naming each changed or missing file (R-107, C-26). **(c)** A `*(controls: HAZ-003)*` marker on an R/C/I/K/E row names the hazards from the user's risk file that the id is a risk control for, and the reports roll them up as `VERIFIED`, `NOT_VERIFIED` or `UNCONTROLLED` (R-108, C-27). **(d)** `--risk-file` lists the hazards allocated to the software, and an `UNCONTROLLED` one fails `--strict` (R-109, C-28, R-15 amended). Every addition is conditional on its flag or marker: a run using none is byte-identical to v1.24 (I-021). No id status, verdict or metric moves.
- **v1.23.1 (built before v1.24):** Built 2026-10-27 in four waves (`docs/plans/IMPLEMENTATION_PLAN.md`; `SPEC_BUILD_REPORT.md` §0n): v1.23's namespaced ids and citation map, with the seventeen review findings below folded in, plus two defects the build itself found and fixed in this document (C-19's metavar list lacked `NS`; jest-junit writes a top-level test as `" > title"`, now joined by C-04, measured by T-117).
- **v1.24 (built before v1.25):** three suggestions from a regulated-domain debrief (`docs/proposals/PROPOSAL_v1.24_unproven_cut_rows_isolated_ids.md`, 2026-10-27; D-58..D-60 confirmed on the recommended branches, same day). **(a)** A non-strict run that exits `0` while `--strict` would reject it prints `UNPROVEN`, not `CONFORMING` (R-105, §5.1, §5.4); the exit code is unchanged. **(b)** C-01 (a)'s backtick span is the CommonMark code span, so a run of backticks no longer flips the splitter's state for the rest of a row (D-59). **(c)** E-66 also Notes a declared row with more cells than its table's header. **(d)** One Note

names the declared ids no C-12 edge touches (E-77, D-60). **(e)** Not in the proposal: the JS/TS adapter moves verbatim out of C-03 into a new C-24, because C-03 had crossed K-14's cap in v1.23.1 and its statement was being truncated. No status, verdict, metric or exit code moves.

- **Previous status:** v1.22 — a JavaScript/TypeScript test adapter (R-104, D-56, E-74..E-76, T-113..T-116): `it`/`test` calls under `--tests` are delimited by tokens and named by their `describe` chain, and C-04 joins them to vitest's and jest-junit's JUnit output (path classnames, `[`/`(` in titles, `it.each` placeholders). Built on v1.21 — four extractor/CLI gaps from a regulated-domain trial (C-12 range and row-cell edges, E-66, E-09's resolved-path hint; D-50). Full text of both: the revision history at the end of this document.
- **v1.23.1 (built):** v1.23 with the seventeen findings of the first full review since v1.10 folded in (`docs/reviews/SPEC_REVIEW_REPORT.md`, F-701..F-717, 2026-10-27). The ones that change behaviour: C-06's pinned request body is restored (F-701); the `--citation-map` and `--proof-results` files are excluded from every scan (F-702); a mapped citation is DECLARED by a stated third rule and is outside `declared_ratio` (F-703); a mapped dangling or stale entry carries `mapped: true` (F-704); C-14 covers JavaScript/TypeScript comment lines (F-705); the mock judge recognises `expect(` (F-706); the JS/TS tokenizer has a pinned regex-versus-division rule (F-708); K-18 counts an unreadable file as a usage fault (F-709); C-18's reasons name the right C-05 steps (F-711); the FOREIGN grammar has no length cap (F-712). The rest are corrections to lists, tables and wording. Full text: the revision history.
- **v1.23 (folded into v1.23.1, built with it):** namespaced ids and a citation map (`docs/proposals/PROPOSAL_v1.23_namespaces_and_citation_map.md`, 2026-10-27; D-51..D-55 confirmed on the recommended branches, D-57 from the fold's review `docs/reviews/SPEC_REVIEW_v1.23_fold.md`). **(a)** A token written `<ns>:ID` is someone else's id, not a citation or an edge (R-101, C-22, E-67, E-69): a test docstring naming `platform:R-99` no longer makes a local id PASSING, and `--namespace NS` (R-102, E-68) turns `NS:ID` into a local citation for the project that owns `NS`. **(b)** `--citation-map FILE` (R-103, C-23, E-70..E-73, K-19) lets an owner of the specs but not of the tests state which existing test proves which id, labelled `mapped` in the JSON, the Markdown report, `explain` and the summary line, with C-05 untouched. Both flags are accepted by `check`, `impact` and `explain`. A run using neither flag and containing no `ns:`-prefixed token is byte-identical to v1.22 (I-019).
- **Previous status:** v1.20 — three places where the spec's own formal model (`proof_from_spec/`, via `spec-model`) could not render a total function over the input space it enumerates, each fixed without waiting for a third-party review to find it first (`docs/proposals/PROPOSAL_v1.20_spec_determinac`; 2026-09-24; D-46..D-48 confirmed on the recommended branches, same day). **(a)** `judge_available` (C-07, amended) is now boolean and total: `false` whenever at least one edge was judge-eligible and no judge call succeeded — whether every call failed (E-14) or none was issued at all, the `--judge-budget`/`--jev-pre-triage` case E-35 already names (K-12) — and `true` otherwise; it is never `null` under `mock`/`llm`. E-32 (amended) records `strict_judge_failure: "unavailable"` for this case too, replacing a `null` reason on an exit-1 run R-28's own reason pair exists to prevent (T-101; D-46). **(b)** `explain --spec` (§5.1) is now stated as required in its synopsis, not just its flag table — the two disagreed on one enumerated invocation, `explain ID` with no `--spec`, and the model's `outcome` returned `none` for it; a missing `--spec` on `explain` is now E-64, exit 2, message `explain: --spec is required` (T-102; D-47). **(c)** §5.4 gains K-18: when one invocation carries both a usage fault (exit 2) and an input-contract violation (exit 3), flag/value/`--root`-containment validation completes before any spec/results/output read, so the usage fault is reported — closing the one case where two faithful implementations could disagree on the same invocation's exit code (T-103; D-48). No status, verdict, citation, or existing metric moves; the one behavioural change is Part (a)'s exit reason on an already-red `--strict --judge`

`llm` run whose eligible edges were all budget-skipped (`null` → `"unavailable"`), and the goldens are unaffected since `--self-check`/§9.8 run `--judge mock` with no budget.

- **Previous status:** v1.19.1 — one gap `spec-model`'s formal model of v1.19 found in v1.19 itself, fixed before any implementation existed (F-504, `docs/reviews/SPEC_MODEL_FINDINGS.md`; D-49 amended, 2026-09-24). C-07's top-level `proof` key was gated on `--proof` alone; a run given only `--proof-results` therefore read the manifest (C-21 states the read is unconditional) and wrote no trace of it anywhere — the build result silently dropped. The gate is now `--proof` **or** `--proof-results`; the per-id `proof` field is unaffected (still gated on `--proof` alone — there is nothing to attribute without a scan). No id, flag, exit code, or byte of a run under either flag alone or both together changes; only the *no-`--proof-results`-alone* case gains a `proof` key it previously lacked. Building the feature also refined K-17's file match: a real cross-repository run against `hello_world_deepseek`'s own Lean proof showed every citation reading `stale` instead of `checked`, because a citation's `file` is `--root`-relative (R-20) while `speccheck/tools/proof_evidence.py`'s manifest records `file` relative to its own `--proof` scan root — the join now tries exact match, then a path-suffix match, before concluding the two name different declarations. v1.19 — the formal-verification half of the conformance graph gets a machine tie into the checker (R-100, C-20, C-21, I-018, K-17, E-62, E-63, E-65; T-99, T-100; `docs/proposals/PROPOSAL_v1.19_proof_parameter.md`, 2026-09-23; D-43..D-45 confirmed on the recommended branches, 2026-09-24). A repeatable, `--root`-contained `--proof PATHS` flag (C-20) scans Lean source files/directories, splits each on a declaration head (`theorem`/`lemma`/`def`/`abbrev`/`instance`), reads the spec ids from the doc comment directly above (bold spans, `(a)`-style sub-case markers stripped), and attributes a *proof citation* per (declaration, id) pair to a new IdRecord field, `proof` — shaped like `tests` (name, file, line) and never sent to the judge, since a kernel-checked declaration is a fact, not an assertion (R-100). `--proof-results FILE` (C-21) reads — never runs — a JSON manifest (`build`, `theorems[]`, optional `deferred_to_pytest[]`) and joins it to the citations by declaration name (+ file/line, K-17), giving each cited id a proof state — `checked`, `failed` (with the build's error line), `unknown` (cited, absent from the manifest; E-63), or `stale` (a manifest entry naming a different file/line) — a parallel field, never a status input: C-05's algorithm, every status, and `impact`'s blast radius are unchanged (R-06, E-63). `speccheck.json` gains a per-id `proof` field and a top-level `proof` object only when `--proof` is given, `schema_version` unchanged (D-44, D-49); an absent-flag run is byte-identical to v1.18 (I-018, T-100). Under `--strict`, a `failed` proof state is a gate failure only when `--proof-results` was given and is readable (D-45). Both flags are `check`-only in this version — `explain`/`impact` reject them (E-54's pattern), and `explain`'s C-18 mechanism is untouched. v1.18 — the CLI documents its own parameters and environment (R-41, C-19, I-017; T-95..T-98; `docs/proposals/PROPOSAL_v1.18_cli_help_contract.md`, 2026-09-20; D-37..D-42 confirmed on the recommended branches). Every argument definition of the three parsers carries a help entry naming purpose, accepted values as literal tokens, the default and every precondition (C-19), and every `--help` screen carries an `environment:` block naming the eight `SPECCHECK_*` variables with their read condition, requiredness and default plus `COLUMNS` (D-41), and an exit-code/summary-line epilog (D-40). The help's value tokens are the validator's own usage-error tokens (T-95), every definition must carry help and a C-19 metavar (T-96), the screens are byte-pinned at `COLUMNS=80` (T-97, D-39), and the environment block is bound to the code's own `SPECCHECK_*` literals (T-98). No accepted value, default or exit code changes; C-06's wrong variable name (`SPECCHECK_LLMMODEL` → `SPECCHECK_JUDGE_MODEL`) is fixed here because v1.16 and v1.17 had already landed (D-42). v1.17 — `speccheck explain <ID>` (R-40, C-18, I-016, E-60, E-61; `docs/proposals/PROPOSAL_v1.17_explain_id.md`, 2026-09-20; D-33..D-36 confirmed) adds a third subcommand that renders one id's evidence trail — its statement, its status and the C-05 step that set it, its source and test citations with each citing case's outcome and verdict, and the C-12 blast radius at `--depth` — as one stdout trace.

It computes nothing new: it is a renderer over the `IdRecord`/edge/`Verdict` facts `check` already produces and the `walk`/`reverify` facts `impact` already runs (I-016), writes no report file (D-33), and carries `check`'s judge contract unchanged (E-61).

- **Previous status:** v1.16 — a judged edge now receives `related`, the titles of the other obligations its own statement names and that name it (a C-12 `depends_on` neighbourhood, both directions), so the judge can tell an assertion about this clause apart from an assertion about a neighbour it refers to (`docs/proposals/PROPOSAL_v1.14_obligation_aware_judge.md`, 2026-09-20; D-28 and its sub-decision D-28b confirmed). R-38 requires the request to carry `related` — the neighbourhood, own references first, at most eight, each a whitespace-collapsed 160-character title, a retired neighbour tagged `(retired)` (E-57), `[]` when the id has no neighbour — and C-10 defines that an assertion corresponding only to a related obligation is not evidence this statement holds, advisory, no new coercion rule (D-28 extends D-26's stance); the field is request-side only, so no `speccheck.json` field, status, Markdown row, or `schema_version` changes, and it also rides the C-17 Jev triage `state` (D-28b), roughly doubling that input. v1.15 — a truncated judge budget can now be spent on the edges Jev is least sure about (`docs/proposals/PROPOSAL_v1.16_jev_pre_triage.md`, 2026-09-20; D-29..D-32 confirmed). A new `--jev-pre-triage` flag (K-16) sends one triage task per judge-eligible edge — the C-06 request object for that edge, rendered as C-17's `state` text — to the C-17 Jev provider before any real-judge request is issued, and orders the otherwise unchanged real-judge issue loop (K-05, K-06, C-06) by ascending Jev top-choice probability, ties by ascending id; Jev's answer is read for that order and discarded (I-015). `--judge-budget` grows an `N%` form (K-12): exactly $\lceil N/100 \times E \rceil$ of the E eligible edges, least-confident first, are issued, and the rest take today's `judge: budget` disposition; `N%` requires `--jev-pre-triage` under `--judge llm` (E-58), the `SECONDS` form is unchanged, and `0%` is a pure triage dry run. A per-edge Jev failure orders that edge first and is counted in one Note (E-59). No `speccheck.json` field, no status, no Markdown row, and no `schema_version` changes (D-29..D-32). v1.14 — a citation's own words now tell the checker, and the judge, whether a test *declares* an id or merely *reuses* it (`docs/proposals/PROPOSAL_v1.15_declared_vs_incidental_citations.md`, 2026-09-20; D-26 and D-27 confirmed). A citation of an in-scope R/C/I/K/E id inside an attributed test case is **DECLARED** when its line sits inside that case's own docstring or is a whole-line comment, and **INCIDENTAL** otherwise (C-03, C-14; R-39); `Citation.declared` and a new `JudgeRequest.declared` (C-06, C-15) carry the fact into the judge, and `C-10`'s instruction text asks the model not to credit `ASSERTS` on an INCIDENTAL citation without checking the assertion is actually about that clause — advisory only, no new coercion rule (D-26's `yes` branch). `speccheck.json` gains `tests[].declared` and `metrics.declared_ratio` under every `--judge` mode, including `none` (C-07, C-16, `schema_version "1.5"`). Detection is whole-line-comment only, reusing the doc-comment-line recognition the Swift adapter already has rather than adding token-level parsing (D-27's `yes` branch; E-56, I-014). Statuses, the mock judge, and the Markdown report are unchanged. v1.13 made the edges this document already carries into data and added a second subcommand that walks them (D-24, D-25); v1.12 made `--src`/`--tests` comma-separated lists of files and/or directories (D-23, I-012, E-52); v1.11 applied F-401..F-407; v1.10 settled D-22 (R-35); v1.9 added clause-grounded verdicts (R-34). Earlier versions: see the revision history.
- **Language / stack:** Python 3.12 | standard library for the deterministic kernel (`re`, `ast`, `xml.etree`, `json`, `argparse`, `pathlib`) | CLI only; optional model-backed judge behind an `[llm]` extra
- **Sources:** `one_sentence_prompt.md` (the brief); `../skills/spec-writing/SKILL.md` (the ID taxonomy and `SPEC.md` shape the checker consumes); `../skills/spec-build/SKILL.md` §Phase 3 (the manual conformance audit this tool automates); `../skills/spec-review/SKILL.md` §3.17 (the intent → requirement → contract → invariant → test → evidence chain); `../outline.md` Chapters 15–18 (where this system is the worked example);

`docs/reviews/SPEC_REVIEW_REPORT.md` (one file, rewritten per review: v0.1 → F-001..F-017, v1.3 → F-201..F-210, v1.7 → F-301..F-307, v1.10 → F-401..F-407, v1.23 → F-701..F-717; all cited below); `FINAL_SPEC_REVIEW_REPORT.md` (review of v0.4; F-101..F-110 below point at it); `SPEC_v0.5_REVIEW_REPORT_by_QWEN.md` (independent review of v0.5 by a second model; its F-001..F-011 are cited below as Q-001..Q-011 to avoid collision); `docs/proposals/PROPOSAL_v1.7_heading_bodies.md` (the v1.7 change, its evidence, and D-20); `docs/proposals/PROPOSAL_v1.9_clause_grounding.md` (the v1.9 change, its two-model evidence, D-21, and the D-22 question); `docs/proposals/PROPOSAL_v1.13_impact.md` (the v1.13 change, its measurement of the edges in this document — 186 `depends_on`, 240 `verifies`, 110 `affects` in v1.12 — and D-24, D-25); `docs/proposals/PROPOSAL_v1.15_declared_vs_incidenta` (the v1.14 change; `JUDGE_CROSSCHECK_REPORT.md` §2b's cross-model evidence — 153/613 genuine judge conflicts on this document's own `check --judge llm` run, concentrated in ids reused as generic fixture data — and D-26, D-27); `docs/proposals/PROPOSAL_v1.16_jev_pre_triage.md` (the v1.15 change; its §1 calibration measurement — Jev's top-choice probability bucketed against a real `check --judge llm --strict` run's committed verdicts on this repository's own `SPEC.md`, monotonic from 53/111 = 47.75% below 0.60 to 279/295 = 94.58% at ≥ 0.95 over 582 edges, and 51.5% of all judge-eligible edges already at ≥ 0.95 — plus D-29..D-32); `docs/proposals/PROPOSAL_v1.14_obligation_aware_judge.md` (the v1.16 change; its sec. 1 evidence — the cross-checker's finding that a test can cite an id as placeholder data while asserting a fact about a neighbour it names, `JUDGE_CROSSCHECK_REPORT.md` sec. 2b — plus D-28 and its D-28b sub-decision); `docs/proposals/PROPOSAL_v1.19_proof_parameter.md`, `docs/proposals/PROPOSAL_v1.20_spec_determinacy` and `docs/proposals/PROPOSAL_v1.23_namespaces_and_citation_map.md` (the v1.19, v1.20 and v1.23 changes); `docs/reviews/SPEC_MODEL_FINDINGS.md` (F-501..F-504) and `docs/reviews/SPEC_REVIEW_v1.23_fold.md` (F-601..F-612)

- **Scope of this document:** The deterministic conformance kernel (spec-ID extraction, citation graph, test-result mapping, status computation, reporting) and the contract around the optional model-backed *judge*. It does not specify the quality of the specification under check (`spec-review` owns that), does not specify how tests are run (results are consumed, not produced), and does not specify any semantic analysis of source code. Since v1.13 it also specifies the typed edges between declared ids that the kernel records (C-12) and the `impact` walk over them (C-13); it does not specify what an edge *means* — an edge records that one statement names another id, nothing more. Since v1.15 it also specifies the optional Jev triage pass that orders — and, under `--judge-budget N%`, truncates — the queue of edges the judge is asked about (K-16, C-17, I-015); that pass is never the source of a status, and it does not change what the judge is asked or how its answer is validated. Since v1.16 it also specifies the `related` field a judged edge's request carries — the C-12 `depends_on` neighbourhood of the statement — so the judge can distinguish an assertion about the clause from one about a neighbour it names; `related` is a request-side, non-report field, carries no status or metric or `speccheck.json` key, and rides the triage `state` as well as the real-judge request (R-38, C-06, C-10, D-28, D-28b).
- **Normative language:** MUST/MUST NOT/SHALL/SHALL NOT = normative; SHOULD = strong recommendation; MAY = optional.
- **Principle:** *The model may only ever make the news worse.* Every status is computed deterministically from evidence the operator can `grep`; the judge is permitted to downgrade a status with cited evidence, never to upgrade one, and every claim in the report points at a file and line.

---

0. Intent and purpose

`speccheck` answers one question about a project: **for every ID the specification declares, what evidence exists that the implementation realizes it?** It reads a `SPEC.md` written in the house ID taxonomy (R-nn requirements, C-nn contracts, I-nnn invariants, K-nn constraints, E-nn edge cases, T-nn tests), finds where each ID is cited in the source and test trees, joins those citations to the outcomes in a JUnit XML results file, and writes a conformance report in which every ID has exactly one status and every status has traceable evidence.

This is Phase 3 of `spec-build` ("re-read the spec and audit every artifact") made mechanical. Humans doing that audit are slow and get tired; agents doing it are fast and confabulate. The design splits the work accordingly:

- The **deterministic kernel** does everything that can be done by pattern, graph, and arithmetic: which IDs exist, which are cited where, which tests ran and how they ended, what that makes each ID's status, and what the coverage ratios are. Two runs on identical inputs produce byte-identical reports.
- The **judge** — a model-backed component, off by default — answers exactly one question per (test case, ID) edge: *does this test assert the observable behavior this ID describes, or does it merely execute code near it?* — at the granularity of a clause: a statement with several clauses (a pinned interface and the prose around it, R-33) is asserted when a test asserts any one of them (C-10). Its answer MUST cite lines inside the test and MUST quote the clause of the statement it judged against; an answer without valid evidence, or whose clause cannot be found in the statement, is discarded as `UNKNOWN` (v1.9, R-34). The judge can turn a `PASSING` into a `WEAKLY_PASSING`; it can never turn anything into `PASSING`. It is never consulted about a *recorded* test — a T id marked `*(recorded)*` whose proof is a recorded run rather than an assertion (v1.10, R-35).
- The **triage pass** — a second, optional model-backed component, off by default, enabled by `--jev-pre-triage` (v1.15) — answers the same question about *every* judge-eligible edge before the judge is asked about any of them, and its answers are used for exactly one thing: the order in which edges are issued to the judge (K-16). A budgeted run (`--judge-budget`, K-12) then spends itself on the edges the triage is least sure about instead of on whatever declaration order produced first. Its verdict, probabilities, and rationale are never validated against C-06, never written to `speccheck.json` or the Markdown report, and never the source of a `Verdict` (I-015).

Why this boundary (context the implementer should not re-derive): a checker that lets a model decide conformance has merely moved the vibe-coding problem one level up — the report becomes something to trust rather than something to verify. Keeping the model on the downgrade-only side of the line means a green report is as trustworthy as `grep` plus the test runner, and a yellow one carries a reason you can click on. The triage pass sits on the same side of that line: it may reorder the judge's queue and, under K-12's `N%` form, shorten it, but it can never write a verdict, a clause, or a status.

- The **`explain` surface** (v1.17) — a third subcommand, `speccheck explain <ID>` — renders one id's evidence trail as a single readable trace on stdout: its statement, its status and the C-05 step that set it, its source citations, each citing test case with its outcome and its verdict, and the C-12 blast radius a change to it might touch. It computes nothing new — it is a renderer over the `IdRecord`/edge/`Verdict` facts `check` already produces and the `walk`/`reverify` facts `impact` already runs (R-40, C-18, I-016) — and it writes no report file (D-33).

- The **help surface** (v1.18) — the CLI documents its own interface: every flag's purpose, its accepted values as literal tokens, its default and every precondition under which it is ignored or rejected, plus the environment the kernel reads and the exit-code contract, all reachable from `--help` (R-41, C-19). The value tokens are the *validator's own* tokens (T-95) and the environment block is bound to the code's own `SPECCHECK_*` literals (T-98), so the help cannot rot the way a prose copy without a mechanical tie does; `--help` itself reads nothing and exits `0` (I-017).

Non-goals (explicit, to constrain the solution space):

- No semantic analysis of source code — no type inference, control-flow, or "does this function implement R-07" reasoning, by the kernel *or* the judge. Source citations are literal token matches.

- No test execution. The checker consumes a results file; it never runs `pytest`, `go test`, or anything else.
- No spec-quality review (ambiguity, contradictions, missing sections). That is `spec-review`.
- No remediation. The checker reports; it never edits the spec, the code, or the tests.
- No multi-repository or multi-spec runs; one spec, one source tree set, one results file per invocation. `impact --against` reads a second spec file only as a *prior version of the same spec*, to diff its declarations against `--spec`; the prior version is never checked, scanned for, or reported on (C-13).
- No change semantics. An edge (C-12) records that one statement names another id — not why, and not whether the dependency is real. `impact` (R-37) lists what a change *might* touch, at the depth the operator asked for; it never marks an id stale, failed, or non-conformant. The next `check` reports what actually happened.
- No IDE integration, daemon mode, watch mode, or web UI.
- Language adapters exist for Python (v0.1), Swift (v1.6, R-31) and JavaScript/TypeScript (v1.22, R-104); every other language gets file-level attribution (O-2). No adapter parses its language with a real parser except Python (`ast`); the Swift adapter is line-based (D-17) and the JavaScript/TypeScript adapter is token-based (D-56).
- The checker never reads its own outputs as inputs: the spec, the results file, the two report files of `check` and the two of `impact`, and (v1.25) the manifest are excluded from every scan (C-03, F-002).
- No risk management (v1.25). A hazard id in a CONTROLS marker or a risk file (C-27, C-28) is a label owned by the user's ISO 14971 risk management file; the checker reports whether the requirements labelled as its software controls are `PASSING`, never whether a risk is acceptable, and never estimates or evaluates one.
- No signatures or attribution (v1.25). The manifest (C-25) is a deterministic, hashable record of what a run read and wrote; who ran it, who approved it, and its electronic signature belong to the user's quality system (D-62).

A known limitation, stated rather than hidden (F-013): citation is literal. A test that mentions `R-03` as *data* — asserting on a report that contains it, or on an error message — cites R-03 exactly as a test that proves it does. The `speccheck:ignore` markers in C-01 are the opt-out; they are the author's responsibility, and `speccheck` never infers intent from context. Since v1.23 two lexical, author-declared exceptions exist: a token written `ns:ID` is read as someone else's id (C-22), and `--citation-map` lets an owner of the specs but not of the tests state a citation outside the code (C-23) — labelled `mapped` everywhere it is read, never inferred.

Relationship to the skills: `spec-writing` produces the input; `spec-review` grades it; `spec-build` builds from it and, in Phase 3, does by hand what this tool does by machine. The report this tool writes is intended to be pasted into `SPEC_BUILD_REPORT.md` as the per-ID evidence table.

---

1. Actors and goals

Actor	Goals
Operator (human at a terminal, or a CI job)	Run one command against a project and get a report plus an exit code that can gate a merge. Never has to trust a number without a path to its evidence.
Extractor (`speccheck/extract.py`)	Turn `SPEC.md` into the set of declared IDs (with title, statement text, family, retired flag), the §12 decision rows, and the typed edges between ids (C-12; v1.13), and turn source/test files into citations, deterministically.

Actor	Goals
Attributor (`speccheck/attribute.py`)	Map each citation in a test file to the test case that contains it (Python adapter), falling back to file-level attribution for anything it cannot parse.
Results Mapper (`speccheck/results.py`)	Read a JUnit XML file and join each `<testcase>` to an attributed test case, yielding outcomes per test case.
Grapher (`speccheck/graph.py`)	Build the ID → source-file / ID → test-case edge sets and compute each ID's deterministic status and the coverage metrics.
Impact walker (`speccheck/impact.py`)	From the spec-internal edges the Extractor records (C-12: `depends_on`, `verifies`, `affects`), and a changed set given on the command line or computed by diffing two versions of the spec, list every id reached at its shortest depth, the T ids that verify the set, and — through the Extractor's citations — what to re-cite and re-run (C-13). Deterministic; consults no judge and no results file.
Judge (`speccheck/judge.py`; providers `judge_mock.py`, `judge_llm.py`)	For each (test case, ID) edge of a `PASSING` ID, return a verdict with evidence lines; the mock provider is deterministic, the LLM provider is opt-in. Single-principal: the judge sees one edge at a time and holds no state across calls.
Reporter (`speccheck/report.py`)	Render the Markdown and JSON reports from the graph and judge results — for `impact`, the two C-13 reports from the walk, and for `explain` (v1.17) the one-id C-18 trace on stdout — in a fixed order, with no content that varies between identical runs.
Triage provider (`speccheck/jev.py`)	With `--jev-pre-triage` under `--judge llm` (v1.15), ask the C-17 Jev endpoint the C-06 verdict question once per judge-eligible edge and return one confidence per edge; the kernel uses those confidences only to order the judge's issue queue (K-16) and discards the answers (I-015). Stateless, advisory, and single-principal like the judge; its answer is never a `Verdict` and never reaches a report.
CLI (`speccheck/cli.py`, entry point `speccheck`)	The only surface; three subcommands, `check`, `impact` (v1.13) and `explain` (v1.17), each documenting its own flags and the environment it reads (v1.18, R-41). Parse arguments, wire the pipeline, apply the exit-code and verbosity contracts.

2. Requirements (intent, high level)

ID	Statement	Source
R-01	The checker MUST read a Markdown specification and extract every *declared* spec ID together with its family and statement text, per the grammar in C-01.	prompt; spec-writing §ID taxonomy
R-02	The checker MUST recognize a declared ID as *retired* when its declaration is struck through (`~~R-07~~`), and MUST exclude retired IDs from coverage denominators.	spec-writing §ID taxonomy ("retire old ones with a strike-through")
R-03	The checker MUST scan every text file under each directory — and every file named directly — in the `--src` list, except the files C-03 excludes, and record a *source citation* for each (file, ID) pair where the ID token occurs, per C-03. The `--src` list is the comma-separated, per-occurrence, deduplicated effective set defined by C-03's PATHS rule (D-23).	prompt ("analyzes an implementation")
R-04	The checker MUST scan every text file under each directory — and every file named directly — in the `--tests` list, except the files C-03 excludes, and record a *test citation* for each (test case, ID) pair, attributing the citation to the enclosing test case when the file's language adapter can delimit test cases, and to the file otherwise. The `--tests` list is the comma-separated, per-occurrence, deduplicated effective set defined by C-03's PATHS rule (D-23).	prompt ("and its tests")
R-05	When `--results` is given, the checker MUST read a JUnit XML file and join each `<testcase>` to an attributed test case, recording the outcome `passed`, `failed`, `error`, or `skipped` per C-04.	spec-build Phase 1 exit gate
R-06	The checker MUST assign every declared, non-retired ID exactly one deterministic status from the set in C-05, computed by the algorithm in C-05 and by nothing else.	spec-build Phase 3.1
R-07	The checker MUST list every *dangling citation*: an ID token cited in a source or test file that is not declared in the specification.	spec-build Phase 3.1 ("no ID is dangling")
R-08	The checker MUST list every *stale citation*: a citation of a retired ID.	R-02
R-09	The checker MUST compute the coverage metrics in C-07 §metrics, applying the zero-denominator rules stated there; it MUST NOT raise or emit `0.0` for an undefined ratio.	spec-writing §precision rules
R-10	When `--judge` is `mock` or `llm`, the checker MUST obtain, for each (test case, ID) edge whose ID is `PASSING` and whose test outcome is `passed` (F-104), one judge verdict from the set `ASSERTS`, `EXECUTES_ONLY`, `UNRELATED`, `UNKNOWN`, with evidence per C-06.	prompt ("AI-assisted")
R-11	A judge verdict MUST only ever lower an ID's status (`PASSING` → `WEAKLY_PASSING`); the judge MUST NOT change any status other than `PASSING` and MUST NOT create, remove, or re-attribute citations.	§0 principle

ID	Statement	Source
R-12	The checker MUST write a Markdown report (`SPEC_CONFORMANCE_REPORT.md`) laid out per C-08 in which every declared ID appears exactly once.	prompt ("traceable conformance report")
R-13	The checker MUST write a JSON report (`speccheck.json`) conforming to C-07 that contains everything the Markdown report shows.	CI consumption
R-14	The checker MUST exit with the codes in §5.4, and the exit code MUST be a pure function of the JSON report content plus the `--strict` flag.	CI gating
R-15	With `--strict`, any in-scope ID whose status is not `PASSING`, any dangling citation, any stale citation, and (v1.25) any hazard whose status is `UNCONTROLLED` (C-27) MUST cause exit `1`. Status (C-05) and hazard status (C-27) are the only inputs to this rule; individual judge verdicts are not consulted again (F-003).	spec-build "done when"; the hazard clause: `docs/proposals/PROPOSAL_v1.25_provenance_man` §1 B, D-64
R-16	With `--judge none` or `--judge mock`, two runs over byte-identical inputs MUST produce byte-identical `SPEC_CONFORMANCE_REPORT.md` and `speccheck.json`.	spec-writing §precision rules (determinism)
R-17	The checker MUST implement the diagnostics contract in §5.3: silent by default; `--verbose`/`--verbose INFO` emit metadata only; `--verbose DEBUG` additionally emits judge prompts and responses; all diagnostics go to stderr and never alter stdout or the report files.	spec-writing §5 cross-cutting contracts
R-18	With `--judge none` or `--judge mock`, the process MUST perform no network I/O, and `speccheck --self-check` MUST verify this boundary.	§0 boundary
R-19	The checker MUST treat all inputs as read-only; it MUST NOT create, modify, or delete any file outside `--out`, except as I-001 permits for `--self-check` (F-104).	safety
R-20	All paths in reports MUST be relative to `--root` (default: current directory) and use `/` as separator regardless of host OS.	R-16 (portability of golden reports)
R-21	On completion (exit 0 or 1) the checker MUST print exactly one summary line to stdout in the format of §5.1 and nothing else.	CI log hygiene
R-22	The mock judge MUST be deterministic and MUST NOT require configuration: it returns `ASSERTS` when the test case's span contains an assertion token per C-06, else `EXECUTES_ONLY`.	R-16; testability of the judge path
R-23	The LLM judge MUST read its endpoint, model name, and API key from environment variables per C-09; a missing variable MUST be a usage error (exit `2`), and the key MUST never appear in any output, log, or report at any verbosity.	secret handling

ID	Statement	Source
R-24	Every deterministic status (C-05 steps 1–4), every count, and every metric in the report MUST be reproducible by an operator using only the report's own evidence table, `grep`, and the results file; `WEAKLY_PASSING` and the judge-derived metrics MUST be reproducible from the same inputs plus the verdicts *as recorded* in the report (F-017).	prompt ("traceable")
R-25	The checker MUST treat family `T` per C-05: a T id's status is determined by test citations and their outcomes only; source citations of a T id are recorded as evidence but never change its status, and a T id with no test citation is `UNCITED`, never `UNTESTED` (F-001).	spec-build ("every T-nn cites its spec ID")
R-26	The LLM judge MUST use exactly the request body, response path, and instruction text pinned in C-06 and C-10, and the report MUST record the SHA-256 of the instruction text used (F-004).	review F-004
R-27	The checker MUST honor the ignore markers in C-01: a line containing `speccheck:ignore` yields no citations, and a file whose first three lines contain `speccheck:ignore-file` is not scanned; ignored files are counted in a Note (F-013).	review F-013
R-28	With `--strict` and `--judge llm`, the checker MUST exit `1` when the judge was unavailable (`judge_available == false`) or when `unknown_rate` exceeds `--max-unknown` (K-11), and the summary line MUST name the reason (F-012).	review F-012
R-29	The summary line MUST be a single line of ASCII text written to stdout as UTF-8 regardless of locale, in the exact format of §5.1 (F-010).	review F-010
R-30	With `--judge llm`, the checker MUST display a progress indicator for the judge stage on stderr, in the format of C-11, whenever `--progress` resolves to on (default `auto`: on iff stderr is a TTY and verbosity is not `DEBUG`; §5.1). The indicator MUST be redrawn in place per K-13, MUST be erased before anything else is written to stderr or stdout after the judge stage begins, and MUST NOT alter stdout, either report file, or the exit code. With `--judge none` or `--judge mock` no indicator is ever drawn.	requester (2026-09-13: "speccheck should include progress bar when running with LLM judge, since that takes quite a bit of time")

ID	Statement	Source
R-31	For every `.swift` file in the `--tests` list (named directly as a PATHS element or found under a directory element; D-23) the checker MUST delimit test cases per the C-03 Swift adapter — Swift Testing `@Test` functions and XCTest `test*` methods — with a span that begins at the first line of the doc comment / attribute block above the declaration and ends at the function's closing brace, and MUST join their citations to SwiftPM's xUnit `<testcase>` outcomes by function identifier (C-04), so that a citation written where `spec-build` puts it (the test's doc comment) counts exactly as a Python citation does under R-04 and R-05.	requester (2026-09-17: "what would it take to make it work for Swift projects as well as Python ones?" — "go ahead"); the MonteCarloPi Swift build (`SPEC_BUILD_REPORT.md` of that project), where every Swift citation was file-level and the run reported `62 unverified`
R-32	The declaration parser MUST declare an ID whose bold form is the *beginning* of a table row's first cell and is followed by whitespace-separated decoration (`| **K-07** **[port]** |`), ignoring the decoration; the statement remains the second cell (C-01 (a), E-44).	the MonteCarloPi port spec marks port-specific ids `**[port]**` inside the id cell; six declared ids were reported as 43 dangling citations
R-33	For an ID declared by a heading (C-01 (b)), the checker MUST use the heading text followed by the section body beneath it — prose, bullets, and fenced code blocks, up to the next heading of the same or a higher level, capped per K-14 — as the ID's statement (`SpecId.text`, `JudgeRequest.statement`, JSON `statement`), so that the judge and the JSON report see the contract's pinned shape and not its title. The heading text alone is kept as `SpecId.title` / JSON `title`, and that is what the Markdown report renders (C-08). Table-row declarations are unchanged.	`docs/proposals/PROPOSAL_v1.7_heading_bodies.` (2026-09-17): two judges over the same 130 MonteCarloPi edges — `gpt-4o-mini` 65/70, 0 weak; `gemini-3.8-flash` 56/70, 9 weak, 15 UNRELATED, 3 UNKNOWN — and every disagreement was a `###`-declared contract whose statement was a title (`Data structures`, `` `EstimationWorker` (an `actor`) ``) while the pinned API sat unsent beneath it
R-34	For every `ASSERTS` or `EXECUTES_ONLY` verdict the judge MUST name the clause of the statement it judged against, as a verbatim excerpt, and the kernel MUST discard as `UNKNOWN` any such verdict whose excerpt cannot be located in the statement (K-15, E-48) — the statement-side mirror of the evidence-line rule (C-06, I-005), so that a recorded verdict is grounded in both the test and the statement.	`docs/proposals/PROPOSAL_v1.9_clause_groundin` with bodies in the statement, `gpt-4o-mini` graded long contracts by their gist — 13 of 18 stable downgrades on the mdv tree were tests asserting a body clause nearly verbatim — while a model that located the clause (`gemini-3.8-flash`) did not; nothing in the report showed *which* clause either had judged

ID	Statement	Source
R-35	A T id declared with the literal marker `*(recorded)*` immediately after its ID form (C-01) is RECORDED: the checker MUST compute its status by C-05 steps 1–4 exactly as for any other T id — it still needs a citing test case with a passed outcome to be `PASSING` — and MUST NOT send any of its edges to the judge, so that C-05 step 5 never applies to it; the marker MUST be carried into both reports (C-07 `recorded`, C-08). On a non-T id the marker is ignored with a Note (E-50).	D-22: the recorded tests of this document (T-48, T-49, T-51) are cited by presence checks so that they are not `UNCITED`; an honest judge reads such a check as `EXECUTES_ONLY` — `gemini-3.8-flash` did so to T-48 on 2026-09-18 — and `--strict --judge llm` went red for a reason the spec intended but had not written down
R-36	The checker MUST extract, from every declared id's statement, the ids it names, and record them as typed edges — `depends_on` between two R/C/I/K/E ids, `verifies` between a T id and an R/C/I/K/E id, `depends_on` between two T ids — together with every §12 decision row (family D, declaration-only) and the ids its *Affects* cell names as `affects` edges, in `speccheck.json` (C-12, C-07 `decisions`, `edges`). An edge whose target is undeclared is a Note, not an error; an edge whose target is retired is recorded with `retired: true` (E-55). D ids are never citation targets, never appear in `ids`, and count in no metric.	`docs/proposals/PROPOSAL_v1.13_impact.md` §1: in v1.12 of this document 134 of 203 live ids name another id (434 tokens; 186 `depends_on`, 240 `verifies`, 110 `affects` edges), and nothing read them
R-37	The checker MUST provide an `impact` subcommand that, given a changed set — the ids of `--changed`, or the ids whose declarations differ between `--spec` and a prior version named by `--against` — reports every id reached from that set by `affects` edges and by *reverse* `depends_on` edges (dependents, never what the changed id itself depends on), each at its shortest depth with the edge that first reached it, up to `--depth`; the T ids that verify any id of the set; and, when `--src`/`--tests` are given, every source citation and every test case citing any of them; as `impact.json` and `IMPACT_REPORT.md` under `--out` (C-13), deterministically (I-002) and under I-001's write discipline. `impact` never consults a results file or the judge and has no gate.	`docs/proposals/PROPOSAL_v1.13_impact.md` §2 Part B; D-24

ID	Statement	Source
R-38	For every judged edge the checker MUST send the judge, with the statement, the `related` titles of the obligations the statement names and of those that name it — a C-12 `depends_on` neighbourhood, both directions, in-scope R/C/I/K/E ids only, the statement's own references first, at most eight total in C-07 id order, each a whitespace-collapsed title tail-truncated to 160 characters with …, a retired neighbour keeping its title with `(retired)` (D-28, E-57), and `[]` when the id has no neighbour — so that the judge can distinguish an assertion about this obligation from an assertion about a neighbour it refers to. The C-10 instruction text MUST define that an assertion corresponding only to a related obligation is not evidence for this one (C-10), and the `related` field MUST ride the C-17 triage `state` as well as the real-judge request (D-28's sub-decision, recorded as `D-28b`). `related` is request-side context only: it is never a status, a metric, or a `speccheck.json` field, and it changes no C-06 coercion rule — the D-26 advisory stance extends to it.	`docs/proposals/PROPOSAL_v1.14_obligation_awa` §1–2 and §5 (reach); §7 addendum (the triage `state` sub-decision)
R-39	The checker MUST compute, for every citation of an in-scope R/C/I/K/E id inside an attributed (non-file-level) test case, whether it is DECLARED (a citation line inside the case's own docstring, or a whole-line comment) or INCIDENTAL (C-03, C-14); the fact MUST be available under every `--judge` mode, including `none`, and MUST NOT depend on the results file, the judge, or any network access (I-014).	`docs/proposals/PROPOSAL_v1.15_declared_vs_in` §1–2
R-40	The checker MUST accept a third subcommand, `speccheck explain <ID>`, that runs the same extract / attribute / map-results / graph / (triage /) judge stages as `check` over the same inputs, then renders the one id named by `<ID>` as a single human-readable trace on stdout (C-18); `explain` produces no status, verdict, citation, metric or note that a `check` run over the same inputs does not already compute, and its stdout is a pure function of those inputs and of C-18 (I-016).	`docs/proposals/PROPOSAL_v1.17_explain_id.md` §1

ID	Statement	Source
R-41	The checker MUST document its own interface: for every flag the parser defines on `speccheck`, `speccheck check`, `speccheck impact` and `speccheck explain`, the `--help` output MUST name the flag's purpose, every accepted value as a literal token where the set is finite, the default where the flag has one, and every precondition under which the flag is ignored or rejected — `--judge-budget N%`'s companion (`--jev-pre-triage` with `--judge llm`, E-58), every judge flag's `--judge llm` precondition, `--root` containment (E-09), and `impact`'s absent directory default for `--src`/`--tests` (C-13) — and the help MUST also name every environment variable the checker reads (C-09, C-17) with the condition under which it is read, whether it is required and its default, and MUST carry the §5.4 exit codes and the §5.1 summary line (C-19).	`docs/proposals/PROPOSAL_v1.18_cli_help_contr` §1–2
R-100	With `--proof PATHS`, the checker MUST scan the named Lean files or directories (repeatable, comma-separated, each element resolved inside `--root`, E-09) with the lean adapter (C-20) and attribute a proof citation per (declaration, spec id) pair to the id's `proof` field; without the flag it MUST scan no Lean file, run no prover, and write no `proof` key (I-018). Both proof flags are defined on `check` only; `impact`/`explain` reject them (E-54's pattern).	`docs/proposals/PROPOSAL_v1.19_proof_paramete` §1–2
R-101	The checker MUST treat a TOKEN immediately preceded by `<ns>:` — where `<ns>` matches `[A-Za-z][A-Za-z0-9_-]*` (no length cap in recognition, F-712), is not itself shaped like an ID (`[RCIKET]-[0-9]{1,3}`, so two adjacent ids joined by a colon stay two local tokens), and is not the `--namespace` value — as a *foreign reference*: it MUST NOT yield a citation, a C-12 edge, a dangling citation or an undeclared-target Note (C-22, E-67, E-69).	`docs/proposals/PROPOSAL_v1.23_namespaces_and` §1 A: a test docstring naming `platform:R-99` made the local `R-99` PASSING (1/1), and an undeclared foreign id is a `--strict` dangling-citation failure
R-102	`check`, `impact` and `explain` MUST accept `--namespace NS` (at most once; a second occurrence is exit 2, `--namespace: given more than once`; the value matching `[A-Za-z][A-Za-z0-9_-]{0,31}` and not shaped like an ID); a token written `NS:ID` MUST then be a citation of local `ID`, identically to the bare token, and a bare token MUST remain a local citation (E-68, D-52).	proposal §2 Part A; a downstream project's tests cite the platform spec's ids
R-103	`check`, `impact` and `explain` MUST accept `--citation-map FILE`; each resolving entry MUST yield one *mapped* test citation of its id on the named test case, marked in `speccheck.json`, the Markdown report, the `explain` trace and the summary line, and MUST NOT change C-05, the judge's eligibility or any exit code on its own (C-23, E-70..E-73, I-019).	proposal §1 B: a ~1,100-id retrofit needed an out-of-tool injector and a throwaway copy because the engineers own the repositories

ID	Statement	Source
R-104	For every `.ts`, `.tsx`, `.mts`, `.cts`, `.js`, `.jsx`, `.mjs` and `.cjs` file in the `--tests` list the checker MUST delimit test cases per the JavaScript/TypeScript adapter (C-24) — `it` and `test` calls (with the `only`, `skip`, `todo`, `concurrent`, `each`, `runIf`, `skipIf` forms) named by the `describe` titles enclosing them — with a span that begins at the first line of the comment block above the call and ends at the line holding the call's closing parenthesis, and MUST join their citations to a JUnit `<testcase>` by C-04's path-classname and `describe > test` name rules, so that a citation written above or inside a vitest or jest test counts exactly as a Python citation does under R-04 and R-05.	a regulated-domain user's report (2026-10-14: "TypeScript tests can never join a result"; their review interface could reach `UNVERIFIED` and no further); `vitest run --reporter=junit` measured 2026-10-27 (C-04)
R-105	On exit `0` without `--strict`, the summary line's `<STATUS>` (§5.1) MUST be `UNPROVEN` instead of `CONFORMING` whenever R-15's conditions do not hold — some in-scope id is not `PASSING`, or `dangling` or `stale` is non-empty. `CONFORMING` MUST be printed only when they all hold, and `NOT CONFORMING` on exit `1` as before. The exit code, every status and `speccheck.json` are unchanged (R-14, §5.4); C-08's §1 verdict line, being the summary line verbatim, follows.	`docs/proposals/PROPOSAL_v1.24_unproven_cut_r` §1 A: on the golden fixture without `--results`, a non-strict run printed `CONFORMING - 0/20 passing` and exited `0`
R-106	`check` MUST accept a boolean `--manifest` flag (v1.25). With it, after both reports are written, `check` MUST write `speccheck.manifest.json` under `--out` per C-25, recording the tool version, the effective options, the SHA-256 and byte length of every file the run read, and the SHA-256 and byte length of both reports, and MUST NOT change a byte of either report. Without it, no manifest is written and `SOURCE_DATE_EPOCH` is not read (I-021).	`docs/proposals/PROPOSAL_v1.25_provenance_man` §1 A: a line appended to `src/speccheck/report.py` after the test run left both v1.24 reports byte-identical (`d51c504c…`, `5a9f30c3…`); D-61, D-62
R-107	The checker MUST accept a fourth subcommand, `speccheck verify --manifest FILE [--root DIR] [--verbose [LEVEL]]` (v1.25), which MUST re-read, relative to `--root`, every file a C-25 manifest names in `inputs` and `reports`, recompute each SHA-256 and byte length, and report per C-26: exit `0` when every file is present with its recorded hash and length, `1` when any is missing or differs, `2/3` on a usage or manifest fault (E-82, E-83). It MUST NOT write any file, MUST NOT open a network connection, and MUST NOT re-run `check` (D-65).	`docs/proposals/PROPOSAL_v1.25_provenance_man` §1 A; D-65

ID	Statement	Source
R-108	A declaring row or heading of an R, C, I, K or E id MAY carry the CONTROLS marker (C-01, C-27; v1.25). The checker MUST record the hazard ids it names on that id (C-07 `controls`) and MUST compute, for every hazard named by a marker or listed in a risk file, one status — `VERIFIED`, `NOT_VERIFIED` or `UNCONTROLLED` — from the C-05 statuses of its non-retired controls (C-27). The marker MUST NOT change any id's status, any C-12 edge, the judge's eligibility, or any value in `metrics`.	`docs/proposals/PROPOSAL_v1.25_provenance_man` §1 B: a medical-device team built "a control row and a verifying test for every hazard family in our risk file" by hand (`letter_from_johnathan.md`, "What we had to adapt"); D-63
R-109	`check` MUST accept `--risk-file FILE` (C-28; v1.25). Every hazard the file lists that no non-retired declared id controls MUST have status `UNCONTROLLED` (E-81), and with `--strict` an `UNCONTROLLED` hazard MUST cause exit `1` (R-15). A CONTROLS marker naming a hazard the file does not list MUST yield a Note (E-79) and MUST NOT affect the exit code.	`docs/proposals/PROPOSAL_v1.25_provenance_man` §1 B; D-64

3. Behavior and state model

3.1 Lifecycle

A run is a single, stateless pipeline. There is no persistent state between runs, no cache, and no partial-output mode: either both report files are written or neither is. The Reporter guarantees this by rendering both documents in memory, then, in this order (F-106): (1) delete any leftover `<out>/.speccheck.json.*.tmp` and `<out>/.SPEC_CONFORMANCE_REPORT.md.*.tmp` from earlier runs; (2) write `.speccheck.json.<nonce>.tmp` and `.SPEC_CONFORMANCE_REPORT.md.<nonce>.tmp`, where `<nonce>` is 8 random hex digits chosen per run (F-103); (3) rename the JSON temporary to `speccheck.json`; (4) rename the Markdown temporary to `SPEC_CONFORMANCE_REPORT.md`. If any step fails — an interrupt (E-41) included — every temporary and any file this run already renamed is removed before exit `3` (F-015). The nonce never appears in either report, so I-002 is unaffected. With `--manifest` (v1.25, C-25) the manifest is a third document in the same sequence: rendered in memory after both reports (its `reports` hashes are of their rendered bytes), its leftover `<out>/.speccheck.manifest.json.*.tmp` deleted at step (1), its temporary written at step (2), and renamed to `speccheck.manifest.json` as step (5), after the Markdown rename; a failure at any step removes it too.

Stage	Owner	Input	Output	Fails with
parse-args	CLI	argv, env	`Config`	exit `2`
extract-spec	Extractor	`SPEC.md`	`SpecIndex` (C-02)	exit `3` (E-01, E-02, E-03)
scan-src	Extractor	`--src` roots minus C-03 exclusions	`Citation[]` (C-03)	never (E-10, E-11 are notes)
scan-tests + attribute	Extractor, Attributor	`--tests` roots	`TestCase[]`, `Citation[]`	never (E-12 is a note)

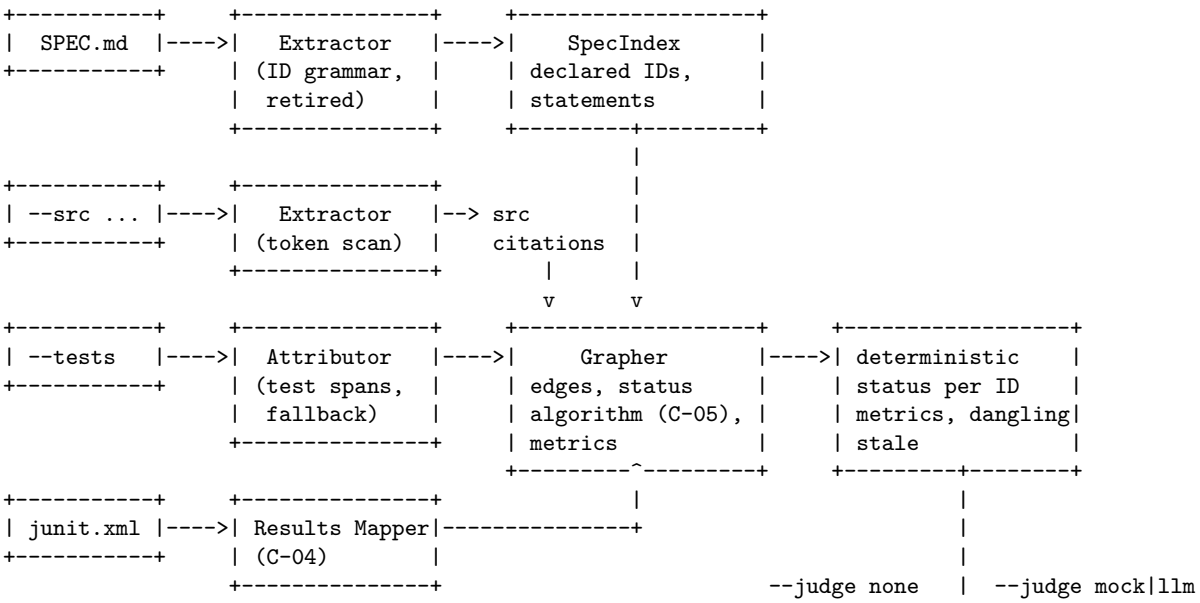
Stage	Owner	Input	Output	Fails with
apply-map (optional)	Extractor	`--citation-map` (C-23), the scanned `TestCase[]`	mapped `Citation[]` added to the scan's; `citation_map` counts; unresolved-entry Notes (E-71)	exit `3` (E-70)
map-results	Results Mapper	`--results`	`Outcome[]` (C-04)	exit `3` (E-05)
read-risk-file (optional)	CLI	`--risk-file` (C-28; v1.25)	`Hazard[]` (id, title)	exit `3` (E-84)
graph + status	Grapher	all of the above	`IdRecord[]` with deterministic status; dangling/stale lists; metrics	never
triage (optional)	Triage provider	judge-eligible edges (I-010)	one confidence per edge, used to order the judge's queue and then discarded (K-16, C-17, I-015)	never (E-59 is a Note)
judge (optional)	Judge	eligible edges, in K-16's ascending-confidence order when triage ran, else declaration order	`Verdict[]` (C-06); progress indicator on stderr while running (R-30, C-11)	never (E-14..E-17 yield `UNKNOWN`)
hazards (optional)	Grapher	`SpecId.controls` (C-27), `Hazard[]`, the final statuses	`HazardRecord[]` (C-27; v1.25), present only when a marker or `--risk-file` exists	never (E-78..E-81 are Notes)
report	Reporter	everything	`SPECCONFORMANCE_REPORT.md`, `speccheck.json`	exit `3` if `--out` unwritable
manifest (optional)	Reporter	the paths and bytes every earlier stage read, the two reports' bytes	`speccheck.manifest.json` (C-25; v1.25), only with `--manifest`, renamed into place with the two reports (§3.3)	exit `3` if `--out` unwritable
exit	CLI	JSON report, `--strict`	exit code (§5.4), summary line	—

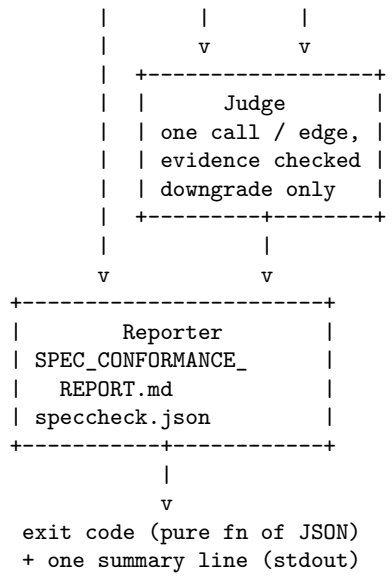
The `impact` pipeline (v1.13). `speccheck impact` is a second, shorter pipeline over the same Extractor and Reporter; it shares parse-args, extract-spec, and — only when `--src`/`--tests` are given — the two scan stages, and it has no results, graph, or judge stage. It writes `impact.json` and `IMPACT_REPORT.md` with the same in-memory render, nonce-named temporaries (`.impact.json.<nonce>.tmp`, `.IMPACT_REPORT.md.<nonce>.tmp`), leftover deletion, and rename order (JSON first) as above, and the same cleanup on failure (E-18, E-41).

The two subcommands never write each other's files.

Stage	Owner	Input	Output	Fails with
parse-args	CLI	argv	`ImpactConfig` (exactly one of `--changed`, `--against`; E-54)	exit `2`
extract-spec	Extractor	`--spec` (and `--against`, parsed by the same C-01 rules)	`SpecIndex` ×1 or ×2 with `decisions` and `edges` (C-02, C-12)	exit `3` (E-01, E-02, E-03 — for `--against`, the message names that file)
changed-set	Impact walker	`--changed` ids, or the diff of the two indexes (C-13)	`Changed[]` with a reason each; an undeclared `--changed` id → E-53	exit `2` (E-53)
walk	Impact walker	`edges`, changed set, `--depth`	`Impact[]` (id, depth, via), `Reverify[]`	never
scan (optional)	Extractor, Attributor	`--src`/`--tests` per C-03 (no directory default here); `--citation-map` applied as in `check` (v1.23)	`Citation[]`, `TestCase[]` restricted to the impact and re-verify sets	exit `3` (E-70)
report	Reporter	everything	`IMPACT_REPORT.md`, `impact.json`	exit `3` if `--out` unwritable
exit	CLI	—	`0`; one summary line (§5.1)	—

3.2 Executable flow





The triage pass (v1.15). With `--jev-pre-triage` under `--judge llm`, the judge stage above is preceded by one triage pass over the same eligible-edge set (K-16): one C-17 request per edge, at most `--judge-concurrency` in flight, whose per-edge top-choice probability orders the queue the Judge box then consumes (ascending, ties by C-07's id order; a failed edge first). The pass writes nothing — no report, no citation, no status — and its answer is discarded once the order is computed (I-015). With `--judge-budget N%` (K-12) the same order also decides which edges are issued at all.

The `explain` surface (v1.17). `explain <ID>` runs the same stages as `check` — extract-spec, scan-src, scan-tests, map-results, graph, and (unless `--judge none`) the triage and judge stages — and then renders the C-18 trace for the one named id instead of writing the two reports. It adds no stage, reads no report file as input (§0's boundary), and writes nothing at all (D-33, I-016).

3.3 Durable artifacts

Artifact	Written to	Version	Shape
`SPEC_CONFORMANCE_REPORT.md`	`--out` (default `.`)	mirrors JSON `schema_version`	C-08
`speccheck.json`	`--out`	`schema_version`: `"1.5"` (`"1.0"` through v1.6; `"1.1"` added `title` in v1.7; `"1.2"` `clause` per verdict in v1.9; `"1.3"` `recorded` per id in v1.10; `"1.4"` `decisions` and `edges` in v1.13; `"1.5"` `declared` per test edge and `declared_ratio` in v1.14)	C-07
`IMPACT_REPORT.md`	`--out`, by `impact` only	mirrors `impact.json` `schema_version`	C-13
`impact.json`	`--out`, by `impact` only	`schema_version`: `"1.0"` (v1.13)	C-13
`speccheck.manifest.json`	`--out`, by `check --manifest` only (v1.25)	`manifest_version`: `"1.0"`	C-25

A subcommand's two files are overwritten on every successful run of that subcommand, via the temp-and-rename sequence in §3.1; `check` never touches the `impact` files and vice versa. No other file is ever written, and no temporary survives a run (I-001).

`explain` (v1.17) writes **no** durable artifact: its trace goes to stdout only (C-18, D-33), so this table and every golden in it are unchanged by it.

`check --manifest` (v1.25) writes its manifest in the same temp-and-rename sequence as the two reports: the manifest's `reports` hashes are of the report bytes about to be renamed into place, and either all three files are written or none is. A run without the flag never creates, modifies or deletes a manifest, including a stale one left in `--out` by an earlier run (D-61). `verify` (v1.25) writes **no** durable artifact (C-26).

---

4. Interfaces / contracts

C-01 Spec ID grammar, declaration, citation, retirement

```
ID        := FAMILY "-" DIGITS
FAMILY    := "R" | "C" | "I" | "K" | "E" | "T"          # O-n, D-nn and F-nnn are NOT conformance
 ↪ IDs;
                                                       # D-nn is DECLARATION-ONLY since v1.13 -
 ↪ rule (c)
DIGITS    := [0-9]{1,3}
TOKEN     := ID bounded by non-alphanumerics on both sides   # regex:
 ↪ (?<![A-Za-z0-9])([RCIKET])-([0-9]{1,3})(?![A-Za-z0-9])

Declaration (in SPEC.md only) - an ID is DECLARED when, outside fenced code blocks,
either form occurs (F-009). A fenced code block (F-108) opens at a line whose first non-space
characters are three backticks (U+0060 x3) or three tildes (~~~), optionally followed by an info
string, and closes at the next line whose first non-space characters are the same three-character
marker - the same character as the opener - followed by nothing but whitespace; a closing line's
info string is not permitted, a backtick fence is never closed by tildes nor a tilde fence by
backticks, and an unclosed fence runs to end of file (Q-008).
Lines (F-301): SPEC.md is split into lines on "\n"; a trailing "\r" on any line is removed before
  every other rule is applied, so a CRLF file parses as its LF twin and yields byte-identical
  output (I-002). A BLANK line is one whose text is empty or consists only of spaces and tabs.
  Body lines (see (b)) are re-joined with "\n"; no other whitespace is added or removed - trailing
  spaces on a body line are kept.
  (a) table row:    | **R-07** | <statement> | ...
        row       := a line whose first non-space character is "|"
        cells     := the row split on every "|" that is not preceded by "\" and not inside a
                     backtick span; the leading and trailing empty cells are dropped. A BACKTICK
                     SPAN (v1.24, D-59) is the CommonMark code span: a run of N backticks (N >= 1)
                     together with everything up to and including the next run of exactly N
                     backticks on the same line; a run with no such closing run is literal text and
                     opens no span (`` ` ``` ` `` is one span holding a run of three)
        the trimmed content of the FIRST cell MUST BEGIN with one of the forms
          **ID**   ~~**ID**~~   **~~ID~~**
        and whatever follows the form MUST be empty or begin with whitespace (R-32, E-44):
          `**K-07** **[port]**` declares K-07; `**K-07**x` and `**K-07**, note` declare nothing.
        Text after the form is DECORATION: it is not part of the statement, and a bold ID inside
        it is not a declaration (`**R-01** **R-02**` declares R-01 only).
        RECORDED marker - see the rule after (b); in a row it is the first token of the DECORATION:
          `| **T-48** *(recorded)* | ... |`  and  `| ~~**T-48**~~ *(recorded)* | ... |`  (R-35,
 ↪ F-404)
        statement := the trimmed text of the SECOND cell ("" if there is none)
        title     := statement  (a table declaration has no separate title; R-33)
        A bold ID in any other cell is not a declaration (E-31). A separator row (cells made of
        "-" and ":" only) is never a declaration.
```

```
 (b) heading:      ### C-03 <title>
       HEADING LINE := a line, outside fenced code blocks, indented by at most three spaces, whose
                       first non-space characters are one to six "#" followed by whitespace or by end
                       of line; its LEVEL is the number of "#". ATX headings only (F-304): a line
                       indented four or more spaces is not a heading (and not a declaration), a line
                       underlined by === or --- (setext) is not a heading, "###C-03" (no space) is
↪  not
                       a heading, and a bare "###" is a HEADING LINE with an empty title.
       the declaring line MUST be a HEADING LINE and the ID token (optionally wrapped ~~ ~~) MUST
       be the first token after the "#" run; "### 9.1 Extraction (C-01, C-02)" declares nothing.
       title     := the rest of the heading text after the ID token - and after the RECORDED
↪  marker
                       when the token that follows the ID token is exactly `*(recorded)*` (rule
↪  below),
                       and after a CONTROLS marker that follows either (v1.25, rule below) -
                       trimmed, whitespace-collapsed, with a trailing run of "#" that is preceded by
                       whitespace removed ("### C-03 Title ###" -> "Title"; "### T-48 *(recorded)*
↪  Run"
                       -> "Run")
       statement := title, then - when the SECTION BODY is non-empty - a newline and the
                       SECTION BODY; when title is "" and the body is non-empty, the SECTION BODY
                       alone (F-305). (R-33; v1.7. Through v1.6 the statement was the title alone.)
       SECTION BODY := the lines after the HEADING LINE up to (not including) the next HEADING
                       LINE whose LEVEL is <= the declaring heading's LEVEL, or end of file;
                       fenced code blocks INCLUDED (the pinned shape is the contract; D-20);
                       leading and trailing blank lines dropped; whitespace inside preserved
                       line for line (a code block's indentation is meaning); the body's own
                       deeper headings (#### under a ###) are part of it; a thematic break
                       (---) is body text like any other line; capped at K-14 bytes (E-46).
                       A body that is empty after trimming - "### C-04 Data structures" followed
                       by blank lines and another ### - leaves the statement equal to the title
                       (E-46).
       The body is context, not grammar: a table row inside it is still parsed for its own
       declarations, a deeper heading inside it still declares its own ID (whose own body is
       delimited by the same rule), and a bold ID in the body is not a declaration of the
       heading's ID (E-31, E-47). IDs inside the body's fenced blocks remain non-declarations
       (E-04).
 RECORDED marker (R-35, v1.10; one rule for both forms, F-404): a declaration is RECORDED when the
 whitespace-separated token immediately after its ID form is exactly `*(recorded)*` - in a row,
↪  the
 first token of the DECORATION; in a heading, the first token after the ID token, which is then
↪  not
 part of the title. The marker is meaningful for family T only; on any other family it is ignored
 and noted (E-50). `*(Recorded)*`, `(recorded)`, and a marker that is not that first token are
 decoration like any other and mark nothing. A retired declaration may also be recorded.
 CONTROLS marker (R-108, v1.25; C-27 pins its grammar and the roll-up): a declaration CONTROLS the
 hazards a `*(controls: HAZ-003, HAZ-007)*` marker names when that marker begins at the token
 immediately after the ID form - or immediately after the RECORDED marker when one is present.
 In a row it is part of the DECORATION and so never part of the statement; in a heading it is not
 part of the title, exactly as RECORDED is not. The marker may contain spaces (after each comma),
 so it is matched as the longest prefix of the remaining text that C-27's CONTROLS grammar
 accepts, not as one whitespace-separated token. It is meaningful for families R, C, I, K and E;
 on a T id it is ignored and noted (E-78). Text that begins `*(controls:` in that position but
↪  does
 not match C-27 controls nothing and stays decoration (or title text), with a Note (E-80). A
↪  hazard
 id is not a TOKEN: it yields no citation, no C-12 edge and no Note of its own, whatever its
↪  shape.
 A SECOND declaration of the same ID is E-02 (exit 3); the message names the ID and both lines.
 Nothing "wins": the run produces no report (Q-006).
```

```
  (c) decision row (v1.13; R-36; family D, DECLARATION-ONLY): a TABLE is a maximal run of
        consecutive rows (per (a)) outside fenced code blocks with no BLANK line between them,
    ↪ whose
        second row is a separator row; its first row is the HEADER ROW. The DECISION TABLE is the
        first TABLE in the file whose HEADER ROW has a cell whose trimmed, case-folded content is
        exactly "affects"; that cell's index (cells per (a), leading and trailing empties dropped)
        is the AFFECTS COLUMN. Every row of the decision table after the separator whose trimmed
        first cell matches ^D-[0-9]{1,3}$ (no bold, no strike-through, no decoration) declares the
        decision D-nn (normalized to two digits: D-8 -> D-08; I-011). Its AFFECTS CELL is the cell
    ↪ at
        the AFFECTS COLUMN ("" when the row is shorter); every TOKEN in it (the TOKEN rule above -
        R/C/I/K/E/T only) names an `affects` target (C-12); everything else in the cell ("§10",
        "s5.2", prose) is ignored. A second table with an "affects" header, and a D-nn row anywhere
        else, declare nothing. A second row declaring the same D-nn is E-02 (exit 3). A decision
        is not a conformance ID: it is never in `ids`, never a citation target (the TOKEN regex
        does not match "D-"), never in any metric (D-25); it exists as the source of `affects`
        edges and as a row of C-07 `decisions`.

Retirement - a declaration whose ID token is wrapped in ~~ ~~ :
  | ~~**R-07**~~ | ... |   or   | **~~R-07~~** | ...   or   ### ~~C-03~~ ...
  A retired ID is DECLARED and RETIRED. Retirement is not un-done by a later plain declaration
    ↪ (E-03).

Citation (in --src / --tests files) - every TOKEN occurrence in a text file is a citation,
  regardless of comments, strings, or code. Fenced code blocks are NOT excluded in source files
  (only in SPEC.md). ID numbers are compared numerically: R-7 and R-07 and R-007 are the same ID.
  Since v1.14, a citation inside an attributed test case additionally carries a DECLARED/INCIDENTAL
  fact (C-03, C-14): this changes nothing about what counts as a citation, only what the judge and
  the report are told about it.

Foreign references (v1.23; R-101, C-22): a TOKEN immediately preceded by `<ns>:` is not a citation
  and not an edge target unless `<ns>` is the --namespace value; C-22 states the grammar and wins
  over the TOKEN rule above wherever the two disagree.

Ignore markers (F-013), matched literally and case-sensitively:
  speccheck:ignore        anywhere on a line  -> that line yields no citations (the line still
                                                 counts toward a test case's span)
  speccheck:ignore-file   anywhere in the first 3 lines of a file -> the file is not scanned at
                                                 all (no citations, no test cases); one Note per
                                                 run: "ignored N file(s) by speccheck:ignore-file"
  The markers have no effect inside SPEC.md.

Normalized form used everywhere in output: FAMILY "-" zero-padded to 2 digits for R/C/K/E/T,
  3 digits for I  (R-07, I-005). An ID declared as R-7 is reported as R-07.

Family T (F-001): a T id names an acceptance test. It is IN SCOPE and is expected to be cited by
  the test case that realizes it (in that case's docstring or a comment). Source citations of a
  T id are recorded but do not affect its status (C-05 step 2b). All six families count toward
  `in_scope` and `conformance`. A RECORDED T id (marker above; R-35) is a test whose proof is a
  recorded run (a self-application, an evaluation, a benchmark) rather than an assertion: it is
  still cited by a test case - typically a presence check that the recorded artefact exists and
  has the right shape - and that citation and its outcome give the id its status; its edges are
  never judged (C-05 step 5, I-010).
```

C-02 SpecIndex

```python
@dataclass(frozen=True)
class SpecId:
    family: str          # one of "R","C","I","K","E","T"
    number: int          # 0..999
```

```
    title: str           # table: the second cell, whitespace-collapsed, may be ""; heading: the
↪   heading
                         # remainder, whitespace-collapsed (C-01). What C-08 renders. Added in v1.7
                          ↪  (R-33).
    text: str            # the statement (C-01). table: == title. heading: title + "\n" + SECTION
↪   BODY
                         # when the body is non-empty (body lines per the C-01 Lines rule: "\r"
                          ↪  stripped,
                         # re-joined with "\n", other whitespace preserved; fenced blocks included;
                          ↪  capped
                         # per K-14); == title when the body is empty; == body when title is ""
                          ↪  (R-33, E-46)
    line: int            # 1-based line of the declaration in SPEC.md
    retired: bool
    recorded: bool       # C-01 RECORDED marker (R-35); only ever True for family T
    controls: tuple[str, ...]  # v1.25, C-01 CONTROLS marker (R-108, C-27): the hazard ids it
↪   names,
                         # unique, code-point sorted; () when absent, malformed (E-80), or on a T
                          ↪  id (E-78)

@dataclass(frozen=True)
class Decision:                      # v1.13, C-01 (c); never a SpecId, never in `ids`
    id: str                          # normalized "D-nn"
    line: int                        # 1-based line of the row in SPEC.md
    affects: tuple[str, ...]         # the DECLARED ids named in the AFFECTS CELL, normalized,
↪   unique,
                                     # sorted per the C-12 id order; undeclared tokens are Notes
                                      ↪  (C-12)

@dataclass(frozen=True)
class Edge:                          # v1.13, C-12
    src: str                         # normalized id: a D id for "affects", the T id for "verifies"
    kind: str                        # "affects" | "depends_on" | "verifies"
    dst: str                         # normalized, always a declared R/C/I/K/E/T id
    retired: bool                    # dst is RETIRED (E-55)

@dataclass(frozen=True)
class SpecIndex:
    path: str                       # relative, POSIX
    ids: tuple[SpecId, ...]         # sorted by (family order R,C,I,K,E,T ; number)
    decisions: tuple[Decision, ...] # sorted by number (v1.13)
    edges: tuple[Edge, ...]         # sorted per C-12 (v1.13)
    # invariant: no two SpecId share (family, number); no two Decision share a number;
    #            no two Edge share (src, kind, dst)
```

C-03 Citation, TestCase, attribution

```
@dataclass(frozen=True)
class TestCase:
    file: str            # relative, POSIX
    name: str            # function name, e.g. "test_zero_rate"; "" for file-level
    classname: str       # dotted module path used to match JUnit classname, e.g. "tests.test_core"
    start: int           # 1-based first line of the span; 1 for file-level
    end: int             # inclusive last line; last line of file for file-level

@dataclass(frozen=True)
class Citation:
    id: str              # normalized ID
    file: str
    line: int
```

```
    kind: str            # "src" | "test"
    testcase: TestCase | None     # for kind=="test": the enclosing case, or the file-level case
    declared: bool        # kind=="test" only (v1.14, C-14, R-39); DECLARED vs INCIDENTAL, below
    mapped: bool          # v1.23, R-103, C-23: True only for a citation made by a --citation-map
↪   entry

DECLARED vs INCIDENTAL (v1.14; R-39): every "test"-kind citation carries `declared`, computed by
  C-14's rule, which is stated there once (v1.23.1: this paragraph used to restate it, and the two
  copies had drifted, F-705). Each adapter below says which of its lines are doc-comment or
  comment-only lines; C-14 reads that, and nothing else, from the adapter (I-014).

Python adapter (files ending .py under a --tests root):
  * parse with `ast`; a test case is (F-007):
      - every top-level FunctionDef or AsyncFunctionDef whose name starts with "test_";
      - every FunctionDef/AsyncFunctionDef whose name starts with "test" (underscore NOT
        required: "testAddition" and "test_addition" both qualify; F-102) that is a direct member
        of a top-level ClassDef which is RECOGNIZED: its name starts with "Test", OR any of its
        bases is a Name/Attribute whose final identifier ends with "TestCase";
    span = node.lineno .. node.end_lineno (decorators included, per `ast`).
  * a "test*" function that is a member of an UNRECOGNIZED top-level class, or of a nested class,
    is not a test case; its lines belong to the file-level case, and the file gets one Note:
    "undelimited tests in <path>: <Class.name>, ..." (E-28).
  * classname = the file path relative to --root with "/" -> "." and ".py" dropped
    ("tests/test_core.py" -> "tests.test_core"); methods append "." + ClassName.
  * a citation on a line inside a span is attributed to that case; any other citation in the file
    is attributed to the file-level case (name "").
  * a .py file that fails to parse -> whole file is file-level (E-12; a note, not an error).
Swift adapter (files ending .swift under a --tests root; R-31, D-17, D-18, D-19). The file is
  delimited BY LINES, not by a parser, so the kernel stays standard-library (D-17). Every rule
  below reads a line's text with its `//` comment removed (a `//` inside a string literal is
  treated as a comment start too - D-17) and counts braces `{` / `}` only outside `"…"` and
  `"""…"""` string literals (the latter may span lines).
    TYPE LINE  := a line matching, after that stripping,
                    ^\s*(?:@\S+\s+|(?:public|package|internal|private|fileprivate|open|final|
                    ↪  indirect)\s+)*
                    (?:struct|class|actor|enum|extension)\s+([A-Za-z_][A-Za-z0-9_]*)
                  whose brace depth is greater at the end of the line than at its start. Its CHAIN
↪   is
                  the identifiers of the TYPE LINEs still open at that depth (outermost first) plus
↪   its
                  own identifier; an `extension X` line contributes `X`. Nesting depth is
↪   unbounded.
    FUNC LINE  := a line matching, after that stripping,
                    ^\s*(?:@[A-Za-z_][A-Za-z0-9_]*(?:\([^)]*\))?\s+|(?:public|package|internal|
                    ↪  private|fileprivate|open|static|class|final|override|mutating|nonmutating|
                    ↪  nonisolated|isolated|consuming|borrowing)\s+)*
                    func\s+([A-Za-z_][A-Za-z0-9_]*)\s*[<(]
                  whose identifier is the FUNC NAME. A FUNC LINE on which no `{` opens before SPAN
↪   END
                  can be computed (a protocol requirement, a declaration without a body) is not a
↪   case.
    ATTRIBUTE BLOCK := the maximal run of lines immediately above the FUNC LINE, each of which is
                  (i) a doc-comment line: first non-space characters `///`, or any line of a
                      `/** … */` block; or
                  (ii) an attribute line: first non-space character `@`, or a continuation of an
                      attribute whose parentheses had not balanced on the previous line of the run;
↪   or
                  (iii) a blank line that lies between two lines of kinds (i)/(ii);
                  together with the text of the FUNC LINE itself that precedes `func`.
```

```
    SPAN END   := the first line, at or after the FUNC LINE, on which the running brace depth
↪   returns
                  to the depth in force before the FUNC LINE's first `{`.
  A test case is:
    (S) Swift Testing - a FUNC LINE whose ATTRIBUTE BLOCK contains the token `@Test` bounded on the
        right by `(`, whitespace or end of line (`@Testable`, `@TestSuite` do not match).
        name := FUNC NAME (the parameter list is not part of the name; D-19)
        classname := MODULE, or MODULE "." + CHAIN of the innermost enclosing TYPE LINE joined with
↪       "."
    (X) XCTest - a FUNC LINE whose FUNC NAME starts with `test`, that is a DIRECT member (brace
↪   depth
        exactly one deeper) of a TYPE LINE declaring a `class` whose text contains the token
        `XCTestCase` after the identifier.
        name := FUNC NAME;  classname := MODULE "." + that class's CHAIN joined with "."
    A FUNC LINE that satisfies both is (S). A FUNC LINE whose FUNC NAME starts with `test` and is
    neither (S) nor (X) - a method of a non-`XCTestCase` type, a member of a class nested inside an
    `XCTestCase`, a free function - is NOT a test case: its citations are file-level and the file
    gets one Note "undelimited tests in <path>: <Chain.name>, …" (E-43; E-28's rule). Any other
    function is a helper and produces no Note.
  MODULE := the first path component of the file's path relative to the --tests root under which it
    was found, or the last path component of that root when the file lies directly under it
    (SwiftPM's `Tests/<Target>/…` layout; D-18). `Tests/ProbeTests/Unit/Foo.swift` under
     `--tests Tests` -> `ProbeTests`. (Under `--tests Tests/ProbeTests` the same file yields
↪    `Unit`,
    which will not join; pass the `Tests` directory.) A file named DIRECTLY as a --tests list
↪    element
    (D-23) is "found under" its own parent directory, so its MODULE is that parent's last path
    component (`--tests p/q/Foo.swift` -> `q`); the Python classname is unaffected, being the path
    relative to --root, never to a --tests root.
  span := first line of the ATTRIBUTE BLOCK .. SPAN END. A citation on a line inside a span is
      attributed to that case (the doc comment is inside the span - that is the point of R-31);
      every other citation is file-level (E-13).
  A .swift file whose brace depth goes negative on any line, or is non-zero at end of file, is
      one file-level case with Note `parse fallback: <path>` (E-42; E-12's rule).
  What the join sees (measured 2026-09-17, Swift 6.4 / SwiftPM; D-19): `swift test --xunit-output
      junit.xml` writes `junit-swift-testing.xml` for Swift Testing with
        classname = "<Module>.<Outer>.<Inner>"    (dotted CHAIN; a free function: "<Module>")
        name      = "<name>(<label>:<label>:)"     (the signature; ONE <testcase> per function -
                                                    parameterized tests are not expanded; a
                                                    `.disabled` test carries <skipped>)
      and, on toolchains that write it, `junit.xml` for XCTest with classname = "<Module>.<Class>"
      and name = "<testName>". C-04 strips the signature, so both forms join by identifier.
JavaScript/TypeScript adapter (files ending .ts .tsx .mts .cts .js .jsx .mjs .cjs; R-104): C-24
↪   (moved
  out of C-03 in v1.24 to bring C-03 back under K-14's cap).
Fallback adapter (any other text file): one file-level case per file; its classname is derived
  exactly as for Python (path relative to --root, "/" -> ".", final extension dropped; F-108).

PATHS -- the --src / --tests list (D-23): each --src and each --tests OCCURRENCE on the command
  line is a comma-separated list of paths, and the occurrences build the effective list the
  extractor walks. From the argv occurrences, the effective list is built by:
    1. split each occurrence on the literal "," character;
    2. trim leading and trailing whitespace (" " and "\t") from every resulting segment;
    3. drop every now-empty segment, with no Note and no error (so "--src", "--src a," and
       "--src a,,b" are all tolerable; T-78);
    4. concatenate the surviving segments in occurrence order, then in within-occurrence order;
    5. resolve each surviving path once, inside --root (E-09; a symlink element resolves to its
       target per Q-007), and deduplicate the resolved paths: a file reached by more than one
       element is scanned exactly once, its recorded --src/--tests root is the FIRST-SEEN covering
       element (which fixes a directly-swift-file's MODULE per D-18), and all files are then
```

```
    emitted in ascending resolved-path order (determinism, I-002/C-07), independent of the order
    in which they were named.
  A resolved path that is a DIRECTORY is descended exactly as a single --src/--tests root was in
  v1.11 (K-02 size, K-03 never-descend list, E-10 oversized, E-11 UTF-8, E-13 file-level case,
  E-29 binary, E-30 descent-time symlinks, E-33 ignore markers); a resolved path that is a regular
  FILE is scanned as that one file, with no descent. Every per-file filter above applies
  ↪ identically
  whether the file was reached by descent or named directly. A segment that resolves inside --root
  but is NEITHER a directory nor a regular FILE is a usage error: exit 2, message
  "--<flag>: no such file or directory: <segment>", replacing the former "non-directory ->
  usage error" check (E-52). The "src"/"tests" default (the directory if it exists, else none)
  applies ONLY when the flag is entirely absent; a present --src/--tests whose occurrences are all
  empty yields the empty list (no default), and two empty lists together is E-19.

Text vs binary (F-008): a file is BINARY if any of its first 8192 bytes is 0x00. Binary files are
  skipped silently (no Note, no citations, no test case). Everything else is text and is decoded
  as UTF-8 with errors="replace" (E-11). Symbolic links - to files or to directories - ENCOUNTERED
  DURING DESCENT are never followed (K-03); one Note per run if any were skipped (E-30). A path
  given on the command line (--spec, --src, --tests, --results, --out, --root) is resolved once,
  checked against --root (E-09), and then used in its resolved form: a symlinked --src root is
  descended (a directory) or scanned (a file), and is not counted as a skipped symlink (Q-007,
  ↪ D-23).

Excluded from every scan (F-002), whether or not they lie under a --src/--tests root, and
  whether or not they exist yet: the file named by --spec, the file named by --results,
  <out>/SPEC_CONFORMANCE_REPORT.md, <out>/speccheck.json, <out>/IMPACT_REPORT.md, <out>/impact.json
  (v1.13), the file named by --against (impact only), the file named by --citation-map and the
  file named by --proof-results (v1.23.1, F-702: an input is never read as citations), the file
  named by --risk-file and <out>/speccheck.manifest.json (v1.25), and every
  file directly under <out> whose
  name starts with ".speccheck.json.", ".SPEC_CONFORMANCE_REPORT.md.", ".impact.json.",
  ".IMPACT_REPORT.md." or ".speccheck.manifest.json." and ends with ".tmp" (the §3.1 temporaries,
  ↪ including leftovers from a
  killed run; F-103). Exclusion is by
  resolved path (symlinks resolved), is silent (no Note), and applies before K-02/K-03 filtering.
```

C-04 JUnit XML subset

```
Root: <testsuites> or <testsuite>. Every <testcase> element anywhere below the root is an outcome.
  classname := @classname (may be ""; an empty classname ends every dotted name on a component
               boundary, so it matches every TestCase with the same join_name, and the
               longest-suffix rule below then applies with suffix length 0 - F-105)
  name      := @name (required; missing -> E-05)
  outcome   := "failed"  if a <failure> child exists
             | "error"   if an <error> child exists
             | "skipped" if a <skipped> child exists
             | "passed"  otherwise
Join to a TestCase (F-006): a result joins the TestCase whose `join_name` equals the result's
  join_name AND whose dotted `classname` ends with the result's dotted classname on a component
  boundary ("tests.test_core" matches "tests.test_core" and "test_core"; it does not match
  "unit_test_core"). Among candidates, the one whose classname has the LONGEST common dotted
  suffix with the result wins; if two or more candidates tie, the result is UNATTRIBUTED and one
  Note names the result and every candidate (E-27). `join_name` is defined as:
    join_name := step 1: if name ends with "]" and contains "[":  name[:name.index("[")]
                         else:                                    name
                 i.e. everything from the FIRST "[" to the end is removed, whatever it contains
                 (F-005, F-101): "test_x[3-True]" -> "test_x"; "test_x[list[int]]" -> "test_x";
                 "test_y[a][b]" -> "test_y".
                 step 2 (R-31, D-19): if the step-1 result ends with ")" and contains "(":
                 everything from the FIRST "(" is removed as well - a Swift signature names the
```

```
                test, not a variant: "twoArgs(a:b:)" -> "twoArgs"; "freeFunction()" ->
                "freeFunction"; "testAddition" -> "testAddition"; "test_x[f(1)]" -> "test_x".
    param     := the text removed in step 1 without its outer brackets ("3-True", "list[int]",
                "a][b"), or null; a signature removed in step 2 is never a param.
  Path classnames (R-104): a result classname is read as dotted -- a leading "./" is dropped and
↪   every "/"
    becomes "." ("src/calc.test.ts" -> "src.calc.test.ts") -- before the suffix rule below, so it
    matches a JavaScript/TypeScript TestCase ("web.src.calc.test.ts") on a component boundary even
↪   when
    the test runner's root is a sub-directory of --root. A classname with no "/" is unchanged.
  Leading separator (v1.23.1, measured by T-117): a result name that begins with " > " --
↪   jest-junit's
    `titleTemplate: "{classname} > {title}"` for a test outside every `describe`, whose
↪   `{classname}`
    is empty -- has that prefix removed before step 1 (" > top level" -> "top level"). No Python,
    Swift or vitest result name begins with it.
  Case names go through the same two steps (R-104): a TestCase is joined under
↪   join_name(case.name), not
    case.name, so a title that contains "[" or "(" ("subtracts [neg]") reduces to the same string
    as its result; for every Python and Swift case the step changes nothing.
  Templated titles (E-76; JavaScript/TypeScript cases only): a case whose join_name contains an
    `each` placeholder -- %s %d %i %f %j %o %O %p %# %% or $identifier / $a.b -- matches every
↪   result whose
    join_name fully matches the name with each placeholder replaced by `.*?` ("%%" by a literal
↪   "%");
    those results join the one case as parametrized variants do (E-06, E-24).
  Two Swift Testing cases in one type with the same identifier (overloads by label: `f(a:)` and
  `f(b:)`) share `join_name` and `classname`: a result for either ties, is UNATTRIBUTED, and one
  Note names the result and both candidates (E-45; the E-27 rule).
  Every <testcase> that joins the same TestCase (parametrized variants, or duplicates) is kept in
  that case's `results` list; the case's single `outcome` = worst of them, worst order
  error > failed > skipped > passed (E-06, E-24).
  A <testcase> that joins no TestCase is an UNATTRIBUTED result (E-07): listed, never counted.
Malformed XML, or a root that is neither element: E-05 -> exit 3.
```

C-05 Status algorithm and `IdStatus`

```
IdStatus (deterministic):  RETIRED | UNCITED | UNTESTED | UNVERIFIED | FAILING | SKIPPED | PASSING
IdStatus (judge-only):     WEAKLY_PASSING

For each declared ID x:
  1. if x.retired                                      -> RETIRED   (excluded from denominators)
  2. T := { test cases with >=1 test citation of x }   # file-level cases included
     S := { source files with >=1 src citation of x }
     2a. if x.family != "T":
           if T == {} and S == {}                        -> UNCITED
           if T == {}                                    -> UNTESTED
     2b. if x.family == "T":                             # F-001: S is evidence only
           if T == {}                                    -> UNCITED
  3. if no --results given, or no case in T has an outcome  -> UNVERIFIED
  4. O := outcomes of cases in T that have one
     if any o in O is failed or error                 -> FAILING
     elif every o in O is skipped                     -> SKIPPED
     else                                             -> PASSING
  5. (judge enabled only) if status == PASSING and not x.recorded:       # R-35: recorded ids skip
↪   step 5
       V := verdicts for edges (t, x), t in T with outcome passed
       if V is non-empty and no v in V is ASSERTS and some v in V is EXECUTES_ONLY or UNRELATED
                                                        -> WEAKLY_PASSING
       else                                             -> PASSING   (UNKNOWN never changes status)
```

```
Cases in T without an outcome are reported as "unrun" for x and ignored in step 4.
The judge is never consulted for edges whose ID is not PASSING after step 4, nor for any edge of a
  RECORDED id (I-010, R-35); a recorded id's `tests[].verdict` is null (E-37) and its edges are not
  eligible edges for the progress indicator's <total> (C-11).
Step 5 is the ONLY place verdicts influence anything: an ID with verdicts {ASSERTS, EXECUTES_ONLY}
  is PASSING, and PASSING is what --strict tests (R-15, §5.4; F-003). There is no per-edge rule
  anywhere else in this document.
```

C-06 Judge interface

```python
@dataclass(frozen=True)
class JudgeRequest:
    id: str                 # "R-07"
    statement: str          # SpecId.text - for a heading-declared ID: title, newline, section body
 ↪  (R-33); may be multi-line
    testcase: TestCase
    declared: bool          # v1.14, C-15, R-39: DECLARED vs INCIDENTAL (C-14) for this (id,
 ↪  testcase) edge; read-only context
    source: str             # lines testcase.start..testcase.end, each prefixed "<lineno>\t"
 ↪  (F-109)

@dataclass(frozen=True)
class Evidence:
    file: str
    line: int               # MUST satisfy testcase.start <= line <= testcase.end

@dataclass(frozen=True)
class Verdict:
    verdict: str            # "ASSERTS" | "EXECUTES_ONLY" | "UNRELATED" | "UNKNOWN"
    clause: str             # verbatim excerpt of the statement judged against (K-15); "" for
 ↪  UNRELATED / UNKNOWN (v1.9, R-34)
    evidence: tuple[Evidence, ...]
    rationale: str          # <= 280 characters, single line

class Judge(Protocol):
    def judge(self, req: JudgeRequest) -> Verdict: ...
```

```
Validation applied by the kernel to EVERY provider's raw answer (judge.py, not the provider).
  The rules are applied IN THIS ORDER; the first rule that fires determines the verdict and the
  rationale, and no later rule is evaluated (F-402). Rule 8 is applied to whatever rationale
  results. A verdict coerced to UNKNOWN by any rule is recorded with `clause` "" (E-49).
  1. provider raised, timed out (K-05), or returned non-JSON   -> UNKNOWN, rationale = "judge:
 ↪  <class of failure>"
  2. verdict not in the four-value set                          -> UNKNOWN, rationale = "judge:
 ↪  malformed response" (E-15)
  3. clause absent, or present but not a JSON string            -> clause := ""   (then continue)
  4. verdict is UNRELATED or UNKNOWN                             -> clause := "" whatever was
 ↪  returned (E-49); done
  5. verdict is ASSERTS or EXECUTES_ONLY and clause is not LOCATED (K-15)
                                                                  -> UNKNOWN, rationale = "judge:
 ↪  unlocated clause" (E-48)
  6. verdict == ASSERTS and evidence is empty                    -> UNKNOWN, rationale = "judge:
 ↪  ungrounded" (E-16)
  7. any evidence.file != testcase.file, or line outside span    -> UNKNOWN, rationale = "judge:
 ↪  ungrounded" (E-16)
  8. rationale longer than 280 chars                             -> truncated to 277 + "..."
  An UNKNOWN produced by validation is recorded with `coerced: true` in the JSON report.

LLM provider wire format (judge_llm.py; F-004) - an OpenAI-compatible chat-completions endpoint:
```

```
  Request:  POST SPECCHECK_JUDGE_URL
            Headers: Content-Type: application/json ; Authorization: Bearer
↪   <SPECCHECK_JUDGE_API_KEY>
            Body (JSON, exactly these keys):
              { "model": <SPECCHECK_JUDGE_MODEL>,
                "temperature": 0,
                "max_tokens": 4000,
                "messages": [
                  { "role": "system", "content": <the C-10 instruction text, verbatim> },
                  { "role": "user",   "content": <the JSON text of the object
                                                 {id, statement, related, declared, file, start,
↪   end, source},
                                                 keys in that order, `indent=2`,
↪   `ensure_ascii=False`, no
                                                 trailing newline; id = the normalized id;
                                                 statement = SpecId.text verbatim - body, fenced
↪   blocks and
                                                 indentation included for a heading-declared ID
↪   (R-33, T-74);
                                                 related = the R-38 neighbourhood (v1.16, D-28);
                                                 declared = the C-14 edge value for this edge
↪   (v1.14, C-15);
                                                 file, start, end = the TestCase's; source = the
↪   span with every
                                                 line prefixed by its absolute 1-based line number
↪   and one TAB
                                                 (F-109)> } ] }
            `messages` is a JSON array of exactly these two objects, system first (F-701).
↪   `declared`
            and `related` are request-side context only: neither is reported, and no C-06 coercion
            rule acts on them.
  Response: HTTP 200 with a JSON body; the model's text is read from  choices[0].message.content .
            Any other status, a non-JSON body, or a missing path -> E-14 / E-15.
            The text is stripped of surrounding whitespace and of ONE enclosing ``` or ```json
↪   fence
            if present, then MUST parse as a single JSON object {verdict, clause,
↪   evidence:[{file,line}],
            rationale}; `clause` MAY be absent (it is then ""); anything else is non-JSON (->
↪   UNKNOWN, E-15).
  Exactly one HTTP request per edge; no retries (K-06). The SHA-256 of the C-10 text as sent is
  recorded in the report as `judge_prompt_sha256` (C-07).

Mock provider (judge_mock.py):
  verdict = ASSERTS if the span contains an assertion token, else EXECUTES_ONLY
  assertion token := a line matching ^\s*assert\b  or containing  .assert  or  pytest.raises(
                     or (Swift; R-31) containing  #expect(  or  #require(  or  XCTAssert  or
↪   XCTFail(  or  Issue.record(
                     or (JavaScript/TypeScript; R-104, F-706) containing  expect(  or  expect.
↪   (vitest/jest
                     `expect(x).toBe(…)`, `expect.assertions(n)`; node's
↪   `assert(…)`/`assert.equal(…)` already
                     match ^\s*assert\b)
  evidence = every such line (file, line); rationale = "mock: assertion token on N line(s)" /
↪   "mock: no assertion token"
  clause   = the whitespace-collapsed statement cut to its first 280 characters - a prefix, so it
↪   is always LOCATED
             under K-15 (equal to a statement shorter than 12 characters, including ""); the mock
↪   reads no statement
             semantics and stays deterministic (R-16, R-22)
```

C-07 JSON report (`speccheck.json`, `schema_version` "1.5")

```
{
  "schema_version": "1.5",
  "spec": "SPEC.md",
  "judge": "none | mock | llm",
  "judge_available": null,
  "judge_prompt_sha256": "<64 hex chars; present only when judge == llm>",
  "strict": false,
  "max_unknown": 0.2000,
  "strict_judge_failure": null,
  "proof": {"build": {"exit": 0, "theorems_total": 66, "theorems_checked": 66, "errors": []}},  //
   ↪ v1.19.1, D-44/D-49 (amended, F-504); key omitted entirely unless --proof or --proof-results
   ↪ was given; "build" null unless --proof-results was given
  "ids": [
    {
      "id": "R-07",
      "family": "R",
      "recorded": false,
      "controls": ["HAZ-003", "HAZ-007"],   // v1.25, C-27; key omitted entirely on an id with no
       ↪ CONTROLS marker
      "title": "…",
      "statement": "…",
      "line": 88,
      "status": "PASSING",
      "src": [ {"file": "src/pkg/core.py", "lines": [12, 40]} ],
      "proof": [ {"name": "rowInterrupt", "file": "proof/Theorems.lean", "line": 105, "state":
       ↪ "checked", "error": null} ],  // v1.19; [] when --proof was given but this id has no
       ↪ citation; key omitted entirely without --proof (D-49)
      "tests": [
        {
          "file": "tests/test_core.py", "name": "test_zero_rate", "classname": "tests.test_core",
          "lines": [17], "declared": true,
          "outcome": "passed",
          "results": [ {"name": "test_zero_rate[0]", "param": "0", "outcome": "passed"},
                       {"name": "test_zero_rate[1]", "param": "1", "outcome": "passed"} ],
          "verdict": {"verdict": "ASSERTS", "clause": "returns the arithmetic sum of `a` and `b`",
                      "evidence": [{"file": "tests/test_core.py", "line": 21}],
                      "rationale": "…", "coerced": false}   // null when the case was not judged
                       ↪ (Q-001); clause "" for UNRELATED / UNKNOWN (E-49)
        }
      ],
      "unrun": [ {"file": "…", "name": "…"} ]
    }
  ],
  "decisions": [ {"id": "D-08", "line": 1331, "affects": ["C-10", "R-26", "T-49", "T-76"]} ],
   ↪ // v1.13, C-01 (c); [] when the spec has no decision table
  "edges": [ {"src": "R-01", "kind": "depends_on", "dst": "C-01", "retired": false},
   ↪ // v1.13, C-12 order; [] when no statement names another id
            {"src": "T-01", "kind": "verifies",   "dst": "R-01", "retired": false},
            {"src": "D-08", "kind": "affects",    "dst": "C-10", "retired": false} ],
  "hazards": [ {"id": "HAZ-003", "title": "Wrong measurement shown", "controls": ["R-07"],
   ↪ "status": "VERIFIED"} ],   // v1.25, C-27; key omitted entirely unless a CONTROLS marker or
   ↪ --risk-file exists
  "dangling": [ {"id": "R-99", "file": "src/pkg/x.py", "line": 3} ],
  "stale":    [ {"id": "R-02", "file": "tests/test_old.py", "line": 9} ],
  "unattributed_results": [ {"classname": "tests.test_gone", "name": "test_x", "outcome": "passed"}
   ↪ ],
  "notes": [ "skipped 1 file over 2 MiB: data/big.json", "parse fallback: tests/test_bad.py" ],
  "metrics": {
```

```
    "declared": 61, "retired": 2, "in_scope": 59,
    "by_status": {"PASSING": 50, "WEAKLY_PASSING": 3, "FAILING": 1, "SKIPPED": 0, "UNVERIFIED": 2,
     ↪ "UNTESTED": 2, "UNCITED": 1},   // in-scope statuses only; RETIRED is the separate
     ↪ "retired" count
    "conformance_ratio": "50/59",   "conformance": 0.8475,
    "by_family": { "R": {"in_scope": 24, "passing": 22, "ratio": 0.9167}, "C": {"…": "…"}, "I":
     ↪ {"in_scope": 0, "passing": 0, "ratio": null}, "K": {"…": "…"}, "E": {"…": "…"}, "T": {"…":
     ↪ "…"} },
    "judge_strength_ratio": "50/53", "judge_strength": 0.9434,
    "unknown_rate": 0.0000,
    "declared_ratio": 0.9821
  },
  "exit_code": 1
}

Rules: keys emitted in exactly this order (`judge_prompt_sha256` omitted entirely unless judge ==
 ↪ llm;
  `max_unknown` always present, echoing K-11; `strict_judge_failure` is null, "unavailable", or
  "unknown_rate" and is non-null only when R-28 forced exit 1 - when BOTH reasons hold,
  "unavailable" is recorded, since a fully unavailable judge makes the unknown rate vacuous;
 ↪ Q-004);
  `ids` sorted by (family order R,C,I,K,E,T ; number); `decisions` and `edges` always present,
 ↪ after
  `ids`, sorted per C-12 (v1.13; a D id never appears in `ids`, `dangling`, `stale`, or any metric
 ↪ - D-25);
  `results` sorted by name;
  `src`/`tests`/`dangling`/`stale`/`unattributed_results` sorted by (file, line) then name;
  `notes` is a list of strings sorted ascending by Unicode code point (each Note's text is fixed by
  the E-case that produces it, so this order is total and reproducible; Q-002);
  `lines` ascending, unique. No timestamps, hostnames, absolute paths, durations, or version
  strings other than schema_version (R-16).
Title and statement (R-33): `title` is `SpecId.title` and `statement` is `SpecId.text` (C-01,
 ↪ C-02).
  For a table-declared ID the two are equal; for a heading-declared ID with a body, `statement` is
  multi-line - title, `\n`, the section body with its fenced blocks and indentation, truncated per
  K-14 when over the cap - and `title` is the heading text alone. `schema_version` was `"1.0"`
  through v1.6, `"1.1"` from v1.7 (`title` added), and `"1.2"` from v1.9 (`clause` added to every
 ↪ verdict
  object, after `verdict`; R-34), and `"1.3"` from v1.10 (`recorded` added to every id record,
 ↪ after
  `family`; R-35 - `true` only for a T id carrying the C-01 marker), `"1.4"` from v1.13
 ↪ (`decisions`
  and `edges` after `ids`; R-36), and `"1.5"` from v1.14 (`declared` added to every `tests[]`
 ↪ entry,
  after `lines`, and `declared_ratio` added to `metrics`; R-39, C-14); no other key changed.
  `schema_version` does NOT change in v1.19: the top-level `proof` object and each id's `proof`
  field (R-100, C-20, C-21) are conditional keys, present only when `--proof` was given (D-44,
  D-49) - unlike every prior addition above, which was unconditional and so bumped the version.
  The literal is stated in this contract (heading and example) and in §3.3 only; no §9 row repeats
  it - a test asserts "the C-07 value" (F-401).
Citation-map and foreign keys (v1.23; R-101, R-103, C-22, C-23): `foreign` (top level, after
  `unattributed_results`, before `notes`) is present only when at least one foreign reference was
  found in a scanned file; `citation_map` (top level, directly before `foreign`) only when
  `--citation-map` was given; `mapped` (in every `tests[]` entry, after `declared`) only when
  `--citation-map` was given, as is `mapped` on every `dangling` and `stale` entry (after `line`;
  true for an entry a map produced, F-704). `schema_version` stays `"1.5"` (I-019): like `proof`,
 ↪ these are
  conditional keys, and a run that needs none of them is byte-identical to v1.22.
Hazard keys (v1.25; R-108, R-109, C-27, C-28): `controls` (in an id record, after `recorded`) is
  present only on an id whose declaration carries a valid CONTROLS marker; `hazards` (top level,
```

```
  after `edges`) only when at least one id carries one or `--risk-file` was given, and then it
↪ lists
  every hazard either source names (C-27). `schema_version` stays `"1.5"` (I-021): like `proof` and
  `citation_map`, these are conditional keys, and a run that needs neither is byte-identical to
↪ v1.24.
  The manifest (C-25) is a separate file and adds no key here.
Proof field (v1.19, amended v1.19.1 per F-504; R-100, C-20, C-21, D-49): `proof` is an array of
  `{name, file, line, state, error}`, one entry per declaration that cites the id (shaped like
  `tests`); `state` is `checked`, `failed`, `unknown`, or `stale` (K-17); `error` is the build's
  error line for `failed`, else `null`. An id with `--proof` given but no citation has `proof: []`;
  the **per-id** key is omitted entirely when `--proof` was not given (there is nothing to
  attribute without a scan). The **top-level** `proof` key is present whenever `--proof` OR
  `--proof-results` was given (v1.19.1: was `--proof` alone, which silently dropped a manifest
  read via `--proof-results` on its own, `docs/reviews/SPEC_MODEL_FINDINGS.md` F-504); its
  `proof.build` is C-21's `build` object echoed verbatim when `--proof-results` was given, else
  `null`.
Verdict field (Q-001): every `tests[]` entry has the key `verdict`. It is a verdict object when
  the case was judged (judge enabled, ID `PASSING` after C-05 step 4, ID not RECORDED, outcome
  `passed` - the E-37 condition) and `null` otherwise - including every entry under `--judge none`.
  The key is never omitted.
Names (Q-011): `tests[].name` is the `TestCase.name` - `""` for a file-level case. `(file)` is a
  Markdown-only rendering (C-08) and never appears in the JSON. R-13's "contains everything"
  concerns data; a rendering label is not data.
Numbers (Q-009): every ratio is computed as Decimal(numerator) / Decimal(denominator) under a
  28-digit context and quantized to 4 places with ROUND_HALF_EVEN; it is emitted as a JSON number
  with exactly four decimal places (0.9000, not 0.9), so the JSON text is byte-determined without
  reference to binary floating point. K-11 compares these quantized values. `<pct>` in the summary
  line is the quantized `conformance` times 100, quantized to 1 place with ROUND_HALF_EVEN.
  `max_unknown` is not a ratio but is emitted the same way: the K-11 Decimal quantized to 4 places
  (0.2000), matching the summary-line suffix (F-206).
Metrics:
  conformance        = |PASSING| / in_scope                        (WEAKLY_PASSING is NOT passing)
  by_family[f].ratio = |PASSING in f| / |in_scope in f|
  judge_strength     = |PASSING \ RECORDED| / (|PASSING \ RECORDED| + |WEAKLY_PASSING|)   (judge
↪ enabled only; else absent;
                       RECORDED ids are PASSING without a judged edge and are outside this
↪ population - F-405;
                       `conformance` and `by_family` DO include them, since they are PASSING by
↪ citation and result)
  unknown_rate       = |edges with verdict UNKNOWN| / |judged edges| (judge enabled only; else
↪ absent)
  declared_ratio     = |{c in C : c.declared}| / |C|, where C is every "test"-kind, non-mapped
                       Citation of an in-scope R/C/I/K/E id attributed to a non-file-level test
                       case (R-39's population), counted per citation line   (v1.14, C-14, R-39;
↪ present
                       under every --judge mode, including none; a T id's own citations are outside
                       C, matching R-39's scope; a mapped citation (C-23) is outside C, since the
                       metric measures the in-code convention - F-703)
  proof_failed       = |{id : any proof[].state == "failed"}|                (v1.19, K-17; present
↪ only when --proof given)
  proof_unknown      = |{id : no "failed" proof[], any proof[].state in {"unknown","stale"}}|
↪ (v1.19, K-17; present only when --proof given)
Zero denominators: ratio = null (JSON) / "n/a" (Markdown); the "a/b" string is still emitted
↪ ("0/0");
  `judge_strength` is null when every PASSING id is RECORDED and nothing is WEAKLY_PASSING (F-405);
  `declared_ratio` is null when no in-scope R/C/I/K/E id has any citation (E-19's shape).
  in_scope == 0 is E-01 (exit 3), so `conformance` itself is always defined on exit 0/1.
Shape rules (F-016): `by_family` ALWAYS has exactly the six keys R, C, I, K, E, T in that order,
  each with in_scope/passing/ratio, even when in_scope is 0. `by_status` ALWAYS has exactly the
  seven in-scope statuses in the order shown, each a count (0 allowed); RETIRED is never a key.
```

`judge_available` is `null` under `--judge none`. With `--judge mock` or `--judge llm` it is `true` when at least one judge call succeeded, or when no edge was judge-eligible (I-010; vacuously available), and `false` when at least one edge was judge-eligible and no judge call succeeded - whether because every call failed (E-14) or because no call was issued at all (E-35: a budget of `0`, or the `SECONDS`/`N%` deadline reached before the first request). It is never `null` under `mock`/`llm` (v1.20, amended; F-502). The C-07 example's `// comments` are explanatory and never emitted.
Family T counts toward in_scope and conformance like every other family (F-001).

C-08 Markdown report layout (SPEC_CONFORMANCE_REPORT.md)

```
# Specification Conformance Report

**Spec:** `SPEC.md` · **Judge:** none|mock|llm (available|unavailable) · **Strict:** on|off
# the parenthetical is omitted (with its leading space) when `judge_available` is null, i.e. under
 ↪  --judge none (F-207)

## 1. Verdict
<one line: the §5.1 summary line, verbatim>

## 2. Metrics
| Metric | Value |            # conformance a/b (0.xxxx); by-status counts; per-family table;
                               # judge_strength and unknown_rate rows only when judge enabled

## 3. Per-ID evidence
| ID | Status | Statement | Source citations | Test citations (outcome · verdict) |
# v1.19: when --proof was given, the row gains a sixth cell, `Proof (state)`, rendering each C-07
#   proof[] entry as `name (file:line, state)`, semicolon-joined, or an em dash when proof[] is
#   empty; the column is absent entirely without the flag (I-018)
# one row per declared ID in C-07 order; RETIRED rows included, with the ID cell (only) struck
 ↪  through: `~~R-07~~`;
# a RECORDED id's ID cell is `T-48 (recorded)`, and a retired recorded id's is `~~T-48~~
 ↪  (recorded)` - strike the id, then the
# label (R-35, F-404); the verdict position shows the em dash, as for any unjudged case;
# the Statement cell renders the JSON `title` - the heading text or the table cell - never a
 ↪  section body, so
# this table is byte-identical before and after v1.7 (R-33, T-73); the body the judge saw is in
 ↪  `speccheck.json`
# citations rendered as `file:line` (a mapped citation, v1.23, as `file (map)` -- it has no line;
 ↪  C-23); verdict rendered as ASSERTS / EXECUTES_ONLY / UNRELATED / UNKNOWN(coerced),
# or as an em dash (-) when the JSON `verdict` is null (Q-001); a file-level case is rendered with
 ↪  the
# label `(file)` in place of its empty name (Q-011)

## 4. Dangling citations      # table or "None."
## 5. Stale citations         # table or "None."
## 6. Unattributed results    # table or "None."
## 7. Unrun test citations    # table (ID, file, name) or "None."
## 8. Judge details           # present only when judge enabled: one row per judged edge (verdict
 ↪  non-null): ID, case, Verdict,
                               # Clause (the JSON `clause`, tail-truncated to 80 characters, em
 ↪  dash when ""), Rationale (v1.9, R-34)
## 9. Notes                   # bullet list or "None."
## 10. Hazards                # v1.25, C-27: present only when the JSON `hazards` key is; one row
 ↪  per hazard in its order:
                               # | Hazard | Title | Controls | Status |, Title an em dash when
 ↪  null, each control rendered
                               # `R-12 (PASSING)` (its C-05 status), semicolon-joined, an em dash
 ↪  when there is none
```

C-09 LLM provider configuration

```
SPECCHECK_JUDGE_URL      required with --judge llm   e.g. https://host/v1/chat/completions
SPECCHECK_JUDGE_MODEL    required with --judge llm   model identifier passed through verbatim
SPECCHECK_JUDGE_API_KEY  required with --judge llm   sent as "Authorization: Bearer <key>"
SPECCHECK_JUDGE_TIMEOUT  optional, seconds, default 30, integer 1..300 (else exit 2)
The URL and model MAY appear at INFO; the key MUST NOT appear anywhere, including DEBUG output
and error messages (redact to "***").
```

C-10 Judge instruction text (normative; F-004)

The file `speccheck/judge_prompt.md` is shipped as package data and its content is exactly the text below (trailing newline, `\n` line endings). It is sent verbatim as the system message (C-06). Changing it is a spec change: bump this document's version and the file together (v1.7 added the any-clause rule for statements that carry a section body — R-33, D-20; v1.9 made the question clause-first and added the `clause` field — R-34, D-21; v1.14 added the `declared` field and the skepticism rule below — R-39, C-15, D-26; **v1.16 added the `related` field and its rule below, and sent it on the triage state as well as the real-judge request — R-38, D-28 incl. its D-28b sub-decision**; v1.23.1 extended the `declared` definition to a mapped citation and stated the `related` order as specification id order — F-703, F-717, D-57; the build updates `speccheck/judge_prompt.md` in step, which changes `judge_prompt_sha256` and voids the recorded T-49 runs per F-303).

```
You are a test-strength judge for a specification conformance checker.

You will receive one JSON object with these fields:
  id        - a specification ID, e.g. "R-07"
  statement - the normative text of that ID: its title on the first line and, for an ID declared
              by a heading, the section beneath it - prose, tables, and code blocks. A statement
              may have several clauses: a pinned interface, numbered rules, table rows.
  declared  - true when this test's own docstring or a comment names this ID - the convention
              this project's tests are expected to follow - or when a citation map the project's
              owner maintains names this test for this ID (the ID then need not appear in source);
              false when the ID appears only elsewhere in the test body.
  related   - the names of the other obligations this statement names and that name it - its
              C-12 `depends_on` neighbourhood, both directions; the statement's own references
 ↪  first,
              then the others, each group in specification id order. Context only: an assertion
 ↪  that corresponds
              to a related obligation and not to this statement is not evidence that this statement
 ↪  holds
              (see the related rule in the Rules below). The empty list when the id has no
 ↪  neighbour.
  file      - the path of one test file
  start     - the first line number of one test case in that file
  end       - the last line number of that test case (inclusive)
  source    - the text of lines start..end, one line per source line, each line prefixed by its
              absolute line number and a tab (e.g. "17<TAB>assert x == 2"); cite those numbers

Answer exactly one question: does any assertion in this test case check any one clause of the
statement? Pick the clause you judge against and quote it verbatim in "clause".

Reply with one JSON object and nothing else - no prose, no markdown fence:
  {"verdict": <VERDICT>, "clause": <text>, "evidence": [{"file": <file>, "line": <n>}, ...],
 ↪  "rationale": <text>}

VERDICT is exactly one of:
  "ASSERTS"       - the test contains at least one assertion (assert statement, assertion method,
                    expected-exception context, or equivalent) whose expected value or condition
                    corresponds to the clause. You MUST list every such assertion line in evidence.
  "EXECUTES_ONLY" - the test runs code the clause describes but no assertion checks it
```

```
                  (assertions absent, trivial, or about something else).
  "UNRELATED"     - the test does not exercise any clause of the statement.
  "UNKNOWN"       - you cannot decide from the source given. Prefer UNKNOWN over guessing.

Rules:
  - clause is a verbatim excerpt of the statement, at least 12 characters and at most 280; copy
    it, do not paraphrase. For UNRELATED and UNKNOWN, clause is "".
  - A statement with several clauses is asserted when any one of them is; it is not required to
    be asserted whole by one test.
  - Every evidence line number MUST be between start and end inclusive, in the given file.
  - Cite only lines that exist in source. Do not invent lines.
  - rationale is one sentence, at most 280 characters.
  - Judge only the given test case. Do not assume what other tests do.
  - A test that mentions the ID in a comment or string is not evidence of asserting it.
  - When declared is false, do not credit ASSERTS merely because a clause is locatable and some
    assertion exists nearby: check that the assertion's own subject is unambiguously this
    obligation, not a token reused as example or fixture data for a different one.
  - related lists the names of the other obligations this statement names and that name it (its
↪ C-12
    depends_on neighbourhood, both directions; own references first, at most eight, in
↪ specification id
    order; a retired neighbour keeps its title with (retired) appended; the empty list when none).
    An assertion that corresponds only to a related obligation is not evidence that this statement
↪ holds;
    judge this statement's own clauses, not a neighbour's. When no assertion corresponds to this
    statement's own clause, prefer EXECUTES_ONLY or UNRELATED over an ASSERTS that matches a
↪ related obligation only.
```

C-11 Judge progress indicator (R-30)

The indicator is one physical line on stderr, redrawn in place. It is written directly to the stderr stream, **not** through the `speccheck` logger, so the §5.3 logging level is unaffected and the line never carries a level prefix. While it is displayed — from the first draw to the erase — the indicator owns stderr: nothing else is written to that stream (F-201). Every write is one of the two byte sequences below, each issued as a single atomic write (F-203, F-205); no terminal escape sequence is ever used, so the indicator renders on any console that honours a carriage return:

```
draw:   "\r" + <line> + " " * (W - len(<line>))     # W = length of the longest <line> drawn so far
↪ in this run
erase:  "\r" + " " * W + "\r"                          # what remains on stderr after the judge
↪ stage is nothing

<line> ::= "judge: [" <bar> "] " <done> "/" <total> " edges  " <elapsed> " elapsed  ~" <left> "
↪ left"
<bar>     exactly 20 cells: k times "#" followed by (20 - k) times "-"
<done>    decimal integer, edges whose verdict is determined (received, coerced to UNKNOWN, or
↪ budget-skipped)
<total>   decimal integer, number of eligible edges (I-010)
<elapsed> M:SS  (minutes without upper bound, seconds zero-padded to two digits)
<left>    M:SS, or "?:??" while done == 0

Regex <line> MUST match (T-62), before padding:
^judge: \[[#-]{20}\] \d+/\d+ edges  \d+:\d{2} elapsed  ~(\d+:\d{2}|\?:\?\?) left$
```

The padding exists because `<left>` can shrink (`~12:00 left` → `~9:59 left`); padding every draw to the widest line so far means a plain `\r` never leaves residue, and the erase needs no erase-to-end-of-line escape (F-205).

With n the number of eligible edges, d the number determined so far, and t the wall-clock seconds since t_0, the filled cell count and the estimate of time remaining are

$$k = \left\lfloor \frac{20\,d}{n} \right\rfloor$$

$$\text{left} = \frac{t}{d}\,(n - d) \quad \text{for } d > 0$$

t_0 is the instant of the first draw — the start of the judge stage, immediately before the first request is issued — so the first draw shows `0:00 elapsed`. K-12's budget deadline (the `SECONDS` form) keeps its own origin, the moment the first request is issued, which is $\geq t_0$ (F-202); under the `N%` form there is no deadline — the edges K-12 does not issue are already determined when the stage starts, so the first draw MAY count them and `<done>` stays non-decreasing across draws, exactly as for a verdict received. `left` is undefined (rendered `?:??`) while $d = 0$; the line is never drawn when $n = 0$ (E-36: no request is sent). Both durations are floored to whole seconds before rendering, with $M = \lfloor s/60 \rfloor$ and $\text{SS} = s \bmod 60$. The line is pure ASCII (the cells are `#` and `-`, never Unicode block characters) so it renders under any locale (R-29 rationale). It carries no file name, statement text, prompt, response, or key (I-007). The final state ($d = n$, a full bar) is always drawn before the erase.

C-12 Spec-internal edges (v1.13; R-36)

```
Sources of edges -- all read from the SpecIndex, nothing else:
  (1) statement tokens: for every declared id X (retired included), every TOKEN (C-01) in X.text --
      title and section body, fenced blocks included, after the K-14 cap -- whose normalized id Y
      is != X:
        Y undeclared                              -> no edge; one Note per distinct (X, Y):
                                                     "edge to undeclared id: <X> -> <Y>"
        X and Y both in R/C/I/K/E                 -> Edge(X, "depends_on", Y)
        exactly one of X, Y in family T           -> Edge(<the T id>, "verifies", <the other>)
                                                     direction is normalized: an obligation naming
 ↪  its
                                                     T and the T naming the obligation are the SAME
                                                     edge, emitted once
        X and Y both in family T                  -> Edge(X, "depends_on", Y)
      Row cells (v1.21): for an id declared by a TABLE row, the tokens of every other non-ID cell
 ↪  of
      the row (the cells after the statement cell) are read as part of X's statement for this pass
      only. `SpecId.text`, the judged statement, `title` and the C-13 diff are unchanged; an
      edge-case row's outcome column is therefore visible to `impact` while the judge still sees
      the statement cell alone (D-50).
      Ranges (v1.21): where two tokens of the same family form a range -- `X-a..X-b`, `X-a...X-b`,
      `X-a to X-b`, `X-a through X-b`, `X-a` U+2013/U+2014/U+2026 `X-b`, with the second family
      letter optional only after `..`, where the end id is then named too -- with a < b <= a + 100,
      every DECLARED id of that family with a < n < b is named too, exactly as if its token
 ↪  appeared. An undeclared id strictly
      inside a range produces no Note; the endpoints are ordinary tokens and behave as above. A
      range in a family that differs, with b <= a, or wider than 100 names only its endpoints.
  (2) decision rows (C-01 (c)): for every Decision D and every TOKEN Y in its AFFECTS CELL:
        Y undeclared                              -> no edge; Note "edge to undeclared id: <D> ->
 ↪  <Y>"
        otherwise                                 -> Edge(D, "affects", Y)
      Decision.affects is the sorted, unique list of the DECLARED Y only.
  retired := Y is RETIRED (E-55). X being retired does not suppress X's edges: a retired statement
  still names what it named, and whether to walk from a retired id is the operator's call.
  A foreign reference (C-22) is not a token for this pass: it yields no edge and no Note (E-69);
  nor is it a range endpoint, so `platform:R-01..R-05` names R-05 alone (F-716).
  A self-reference (Y == X) and a repeated token produce nothing; each (src, kind, dst) is
  emitted at most once.
  ISOLATED (v1.24, E-77): a declared, non-retired id that is the src or dst of no edge above. One
  Note per run when any exist: "<N> declared id(s) touch no edge: <ID>, <ID>, <ID>", followed by
  " and <N-3> more" when N > 3, ids in C-07 order. `check` and `explain` compute it; `impact`
```

```
  does not emit it.

Order (total, so I-002 holds): edges sorted by (id_key(src), kind, id_key(dst)), where
  id_key(id) = (family order R, C, I, K, E, T, D ; number) and kind orders
  "affects" < "depends_on" < "verifies" (ASCII); decisions sorted by number.

What an edge is not (D-25; the scope note in the front matter): it is not a citation (C-05 never
  reads it), not a status or metric input, and it carries no meaning beyond "the statement of
  src contains the token dst". The judge (C-06) is not shown edges as such; since v1.16 its
  request carries the titles of a `depends_on` neighbourhood as `related` (R-38). The Markdown
  conformance report (C-08) does not render them; they live in the JSON and in `impact`.
```

C-13 `impact`: changed set, walk, `impact.json`, `IMPACT_REPORT.md` (v1.13; R-37)

```
Changed set:
  --changed IDS   IDS is a comma-separated list, split, trimmed, and deduplicated like a PATHS
                  list (C-03 steps 1-3). Each element MUST be a TOKEN of family R/C/I/K/E/T, or
                  "D-" DIGITS, and is normalized (I-011); an element that is not a declared id or a
                  declared decision -> exit 2, message "--changed: undeclared id: <element>"
                  (E-53); a list that is empty after trimming -> exit 2, "--changed: no ids".
                  Every element gets reason "changed".
  --against FILE  FILE is a prior version of the same spec: resolved inside --root (E-09), decoded
                  and parsed exactly as --spec (C-01, K-14); its E-01/E-02/E-03 exit 3 with the
                  message prefixed "--against: ". The changed set, in the C-12 id order, is:
                    id declared in both, whitespace-collapsed `text` differs     reason "statement
↪ differs"
                    id declared in --spec only                                   reason "added"
                    id declared in FILE only                                     reason "removed"
                                                                                 (walked by
↪ --spec's edges,
                                                                                 of which it is dst
↪ of none;
                                                                                 `line` is its line
↪ in FILE)
                    id declared in both, `retired` differs                       reason "retired
↪ flag differs"
                    D declared in both, `affects` differs                        reason "affects
↪ differs"
                    D declared in exactly one                                    reason "added" /
↪ "removed"
                  An id with several reasons carries all of them, in the order above, joined by ";
↪ ".
                  An empty changed set is not an error: every section reads "None." and exit is 0.
  Exactly one of --changed / --against MUST be given (E-54).

Walk (over the edges of --spec, C-12; breadth-first):
  depth 0 := the changed set. The successors of a node X at depth d are:
    X a D id            -> every Y with Edge(X, "affects", Y)
    X in R/C/I/K/E      -> every Y with Edge(Y, "depends_on", X)        (REVERSE: X's dependents;
                                                                         what X depends on is never
                                                                         a successor)
    X in T              -> every Y with Edge(Y, "depends_on", X); the obligations X verifies are
↪ NOT
                           successors (a test changing does not change what it tests)
  An id enters IMPACT once, at the first depth reached, with `via` := the edge that reached it --
  among several at that depth, the smallest by the C-12 order. Changed-set ids never enter IMPACT.
  No edge has a D id as dst, so a D id is never reached. --depth N (integer 0..999; default 1;
  0 = unbounded): a node that would enter at depth > N is neither entered nor expanded; the count
  of such nodes under N = 0 that are not in IMPACT is one Note when non-zero:
  "<k> further id(s) beyond --depth <N>; --depth 0 lists them".
```

```
  REVERIFY := every T id with a "verifies" edge whose dst is in (changed  IMPACT), plus every T id
  in (changed  IMPACT); each with the sorted list of the obligations in (changed  IMPACT) it
  verifies ([] for a T that is there only because it changed or was reached). A retired T is
  included, flagged. Nothing is walked from REVERIFY.
  IMPACT is sorted by (depth, id order); every other list by id order.

Citations (only with --src and/or --tests; the C-03 scan and PATHS rules apply unchanged, except
  that an absent flag means "not scanned" -- the `src`/`tests` directory default of `check` does
 ↪  not
  apply to impact):
  RECITE     := every src-kind citation whose id is in (changed  IMPACT  REVERIFY), grouped as
                (id, file, lines[]), sorted by (id order, file), lines ascending unique.
  TEST_CASES := every TestCase (C-03) holding a test-kind citation of an id in that set, as
                (file, name, classname, ids[]) with ids the sorted ids of the set it cites, sorted
                by (file, start); a file-level case has name "" (Q-011).
  Dangling and stale citations are not computed here; --results is not accepted.

impact.json -- keys in exactly this order (K-09 formatting):
{
  "schema_version": "1.0",
  "spec": "SPEC.md",                                        // relative to --root, as C-07 `spec`
  "against": null,                                          // or the --against path, relative to
 ↪  --root
  "depth": 1,                                               // the --depth value as given; 0 =
 ↪  unbounded
  "changed":    [ {"id": "K-15", "family": "K", "line": 951, "retired": false, "reason": "changed"}
 ↪  ],
  "impact":     [ {"id": "R-34", "depth": 1, "retired": false,
                   "via": {"src": "R-34", "kind": "depends_on", "dst": "K-15"}} ],
  "reverify":   [ {"id": "T-75", "retired": false, "verifies": ["K-15"]} ],
  "recite":     [ {"id": "K-15", "file": "src/speccheck/judge.py", "lines": [6, 76, 143]} ],
  "test_cases": [ {"file": "tests/test_judge.py", "name": "test_clause_bounds",
                   "classname": "tests.test_judge", "ids": ["K-15", "T-75"]} ],
  "notes":      [ "2 further id(s) beyond --depth 1; --depth 0 lists them" ]
}
  `recite` is [] without --src and `test_cases` is [] without --tests; `against` is null under
  --changed; `notes` is sorted as C-07 notes are and also carries the C-12 undeclared-target Notes
  and the scan Notes (E-10, E-11, E-30 ...) when a scan ran. `family` of a D id is "D". No
  timestamps, hostnames, absolute paths, durations, or version strings other than schema_version
  (R-16).

IMPACT_REPORT.md -- sections in this order, every one present, "None." when empty:
# Impact Report
<spec> · changed by `--changed` | against `<file>` · depth <N> | depth unbounded        # one line
## 1. Changed      | ID | Line | Reason |                      the ID cell (only) struck through
 ↪  when retired
## 2. Impact       | ID | Depth | Via |                       Via rendered `R-34 -depends_on->
 ↪  K-15`
## 3. Re-verify    | T id | Verifies |   then, when --tests was given, a second table | File | Case
 ↪  | IDs |
                                          (a file-level case renders `(file)`, as C-08 does)
## 4. Re-cite      | ID | File | Lines |                      "Not scanned." when --src is absent
## 5. Notes        bullet list, or "None."
  The byte-exact rendering is pinned by the golden fixture (T-80) as C-08's is by T-46. UTF-8, `\n`
  line endings (K-09).

Summary line (stdout, exactly one line, on exit 0 only; pure ASCII, single trailing "\n"):
  speccheck impact: <changed> changed, <impact> impacted (depth <N> | unbounded), <reverify> to
 ↪  re-verify, <recite> citations, <cases> test cases
  Regex it MUST match (T-80):
```

```
  ^speccheck impact: \d+ changed, \d+ impacted \((depth \d+|unbounded)\), \d+ to re-verify, \d+
↪  citations, \d+ test cases$
  <recite> is the number of (id, file) rows; <cases> the number of TEST_CASES rows; both 0 when
  the respective root was not given.
```

C-14 Declared vs. incidental citations (v1.14; R-39)

```
For a "test"-kind Citation of id X inside test case T's span (C-03), X is DECLARED when at least
one citation line of X within [T.start, T.end] is:
  - inside T's own docstring (Python: the line range of the `Expr` node ast.get_docstring reads,
    not the text it returns; a multi-line docstring's second and later lines count), or
  - a whole-line comment: the first non-whitespace character on that citation's own line is "#"
    for Python; for Swift, a line whose first non-space characters are "///", or that lies inside
    a "/** ... */" block -- the doc-comment line kind the C-03 ATTRIBUTE BLOCK rule already
    recognizes, reused here rather than re-detected; for JavaScript/TypeScript (v1.23.1, F-705),
    a comment-only line of the C-24 JS/TS adapter (comment text and no code token), every line
    of a "/* ... */" or "/** ... */" comment token included; or
  - a mapped citation (v1.23.1, F-703, D-57): a citation made by a resolving --citation-map
    entry (C-23). The map is the owner's statement of which test proves which id, so the judge
    is told `declared: true` (C-10 says so in its field definition); `mapped` is the separate
    label for where that statement lives, and `declared_ratio` leaves mapped citations out (C-07).
Otherwise X is INCIDENTAL -- including a citation on a code line that also carries a trailing
comment (`x = 1  # R-02`; a deliberate simplification: line ranges only, no column tracking, and
every real DECLARED citation found while drafting this rule was a whole-line comment or a
docstring line). A "src"-kind citation, and a "test"-kind citation attributed to a file-level case
(T.name == ""), are always INCIDENTAL (E-56) -- there is no per-test declaration of intent to
check a file-level citation, or a case that was never delimited, against.

`Citation.declared` (C-03) carries this fact for every citation; where a report or a judge request
needs one bool per (id, testcase) edge rather than per citation line (C-07's `tests[].declared`,
C-15's `JudgeRequest.declared`), the edge's value is true iff any of its citation lines is
↪  DECLARED.
It is computed identically under every --judge mode, including none, and depends on nothing but
the source and test trees already read for attribution -- no results file, no judge call, no
network access (I-014).
```

C-15 declared in the judge request; the instruction text is advisory (v1.14; R-39)

```
`JudgeRequest` (C-06) gains `declared: bool`, inserted after `testcase`, computed per C-14 for the
specific (id, testcase) edge about to be judged. The field is read-only context: no C-06 validation
rule inspects it, it changes no coercion rule (K-15, E-48, E-49 apply exactly as before), and I-004
and I-005 are untouched.

`C-10`'s instruction text gains the field's definition and one rule: when `declared` is false, the
model MUST NOT credit ASSERTS merely because a clause is locatable and some assertion exists
nearby -- it MUST check that the assertion's own subject is unambiguously this obligation, not a
token reused as example or fixture data for a different one. This is advisory, not a coercion rule
(D-26): the model MAY still return ASSERTS on an INCIDENTAL citation -- the docstring convention is
not literally enforced on every test, and E-56 does not apply outside C-14's own scope -- and no
mechanical rule downgrades a verdict for carrying `declared: false`. Changing C-10's text changes
`judge_prompt_sha256` (R-26) exactly as any other wording change does.
```

C-16 declared and declared_ratio in the JSON report (v1.14; R-39)

```
`speccheck.json` (C-07) gains, on every `tests[]` entry, `"declared": bool` per C-14 (true iff any
of the edge's citation lines is DECLARED), present after `"lines"` under every --judge mode
including none. `metrics` gains `"declared_ratio"`, defined in C-07 (the DECLARED share of the
test-kind, non-mapped citations of in-scope R/C/I/K/E ids in non-file-level cases), computed as
↪  every other ratio is
(Decimal, Q-009), null when the denominator is zero.
```

`schema_version` becomes "1.5". This contract is JSON-only: `SPEC_CONFORMANCE_REPORT.md` (C-08) is unchanged (v1.14 kept the report's cost zero under --judge none/mock).

C-17 Jev triage provider (v1.15; K-16, I-015)

```
SPECCHECK_JEV_URL      optional, default https://openrouter.ai/api/alpha/decisions
SPECCHECK_JEV_MODEL    optional, default ~typesafe/jev-latest  (OpenRouter's floating alias -- not
                       the pinned typesafe/jev-1.13 the calibration in
 ↪  docs/proposals/PROPOSAL_v1.16_jev_pre_triage.md
                       §1 was measured against; set it explicitly to hold the measured behaviour
 ↪  fixed)
SPECCHECK_JEV_API_KEY  required when the triage pass runs (--jev-pre-triage with --judge llm);
                       missing -> exit 2, message names the variable, value never echoed (E-21)
SPECCHECK_JEV_TIMEOUT  optional, seconds, default 30, integer 1..300 (else exit 2); per edge,
                       wall-clock, covering DNS, TCP, TLS, send and receive exactly as K-05 does
Read only when the triage pass runs. With --judge none|mock, or without --jev-pre-triage, none of
the four is read, no request is made, and the key is not required (K-16, I-006). The URL and model
MAY appear at INFO; the key MUST NOT appear anywhere, including DEBUG output and error messages
(redact to "***") -- I-007 applies to this provider exactly as it does to the judge.

Request -- exactly one per judge-eligible edge (I-010), at most --judge-concurrency in flight, no
retries, no batching (K-06):
  POST SPECCHECK_JEV_URL
  Headers: Content-Type: application/json ; Authorization: Bearer <SPECCHECK_JEV_API_KEY>
  Body (JSON, exactly these keys):
    { "model": <SPECCHECK_JEV_MODEL>,
      "state": <TEXT, built from the C-06 request object for this edge -- {id, statement, related,
 ↪  declared,
                (v1.16 D-28b: the C-06 object now carries `related`, R-38; the triage `state`
 ↪  carries it too).
               file, start, end, source} -- by exactly this template:
                 "Specification obligation {id}.\n\nStatement:\n{statement}\n\n"
                  "Related obligations:\n{related}\n\n"   # v1.16 D-28b: {related} = the related
 ↪  titles (R-38/D-28),
                  newline-joined, each whitespace-collapsed to 160 chars, a retired one with
 ↪  (retired), or a blank line when []
                 "Test file {file}, lines {start}-{end}:\n{source}"
               i.e. the task tools/judge_crosscheck_tasks.py builds, and the input the calibration
               figures of docs/proposals/PROPOSAL_v1.16_jev_pre_triage.md §1 were measured on>,
      "questions": { "verdict": { "type": "choice",
        "instructions": "Does any assertion in this test case check any clause of the statement?
 ↪  Choose exactly one option.",
          "criteria": {
            "ASSERTS":       "The test contains at least one assertion whose expected value or
 ↪  condition corresponds to a clause of the statement.",
            "EXECUTES_ONLY": "The test runs code the statement describes, but no assertion checks it
 ↪  (assertions absent, trivial, or about something else).",
            "UNRELATED":     "The test does not exercise any clause of the statement.",
            "UNKNOWN":       "Cannot decide from the source given. Prefer this over guessing." } } }
 ↪  }

Response: HTTP 200 with a JSON body; the answer is read from  answers.verdict  as
          {"choice": <label>, "probabilities": {<label>: <number>, ...}}   (a choice answer).
  The edge's CONFIDENCE is  p(e) := max(probabilities.values()) .
  A reply is USABLE iff the status is 200, the body parses as JSON, answers.verdict.choice is one
  of C-06's four verdict tokens, and answers.verdict.probabilities is a non-empty object whose
  values are all numbers. Anything else is a per-edge Jev FAILURE (E-59), not a run failure.

Ordering (K-16): edges are issued to the real judge in ascending p(e); ties are broken by
ascending id in C-07's id order (family R,C,I,K,E,T, then number); a FAILURE is ordered first.
```

Jev's choice, probabilities and any rationale are used for this order and nothing else: never validated against C-06, never recorded in either report, never the source of a Verdict (I-015). p(e) is not a metric and appears in no report field.

C-18 `explain` stdout contract (v1.17; R-40)

`explain <ID>` writes one trace for the named id to stdout and nothing else: no report file, no `--out`, no `schema_version` entry (§3.3, D-33). The section order, the headings and the two-space indentation below are pinned, so two runs over identical inputs are byte-identical (I-002, I-016):

```
ID <id>[ (RETIRED)][ (recorded)]
status: <STATUS> (<the C-05 step that set it>)
controls: <HAZ> (<hazard status>)[, <HAZ> (<hazard status>)...]
statement:
  <the id's statement, C-10/K-14, every line indented by two spaces>
sources:
  <file>:<line>
tests:
  <file>::<name> (<outcome>) [<verdict>]
    clause: <clause>
    rationale: <rationale>
impact (<depth>):
  <id> (depth <n>, via <src> -<kind>-> <dst>)
  <T-id> verifies <id>[, <id>...]
```

- **Status and reason.** `<STATUS>` is the C-05 status, and the parenthetical is C-05's own step (v1.23.1, F-711: the step numbers are C-05's): `RETIRED` → `step 1: struck through in the specification`; `UNCITED` → `step 2a: no source or test citation` (for a T id, `step 2b: no test citation`); `UNTESTED` → `step 2a: no test citation`; `UNVERIFIED` → `step 3: no citing test case has a result`; `FAILING` → `step 4: a citing test case failed or errored`; `SKIPPED` → `step 4: every citing test case with a result was skipped`; `PASSING` → `step 4: every citing test case with a result passed or was skipped`; `WEAKLY_PASSING` → `step 5: no judged edge asserts the statement`. The `RETIRED` marker in the `ID` line is appended for a retired id and the `(recorded)` marker for an R-35 RECORDED id; both may appear.
- **Controls (v1.25).** Printed only when the id carries a CONTROLS marker (C-27): its hazards in `SpecId.controls` order, each with its C-27 status computed over the whole specification, comma joined. `explain` takes no `--risk-file`, so a hazard's title is never shown here; an id with no marker has no `controls:` line, and its trace is byte-identical to v1.24's (I-021).
- **Statement.** `SpecId.text` verbatim (C-10: title, newline, body), each line prefixed by two spaces; a K-14 truncation marker is echoed as it stands.
- **Sources.** One line per source citation, ascending by (file, line), each two-space indented as `<file>:<line>`; the heading alone when the id has none (E-60's empty-block rule).
- **Tests.** One line per citing test case, ascending by (file, start): `<file>::<name> (<outcome>) [<verdict>]`, where `<outcome>` is `passed`, `failed`, `error`, `skipped` or `-` (U+2014: no result joined); a file-level case renders as `<file> (file-level)`. `<verdict>` is the C-06 token with `(coerced)` appended when the recorded verdict is coerced, and `not judged` when no verdict was recorded (under `--judge none`, and for any edge no provider judged; E-61). A `clause:` line is printed only when the recorded `clause` is non-empty, and a `rationale:` line only when a verdict was recorded, both four-space indented under their case. A case whose citation of this id is mapped (C-23) is rendered `<file>::<name> (<outcome>) [<verdict>] (mapped)`; a case with both a literal and a mapped citation of the id is not mapped (E-73).
- **Impact.** `impact (<depth>):` reuses C-12's `walk` from this id at `--depth` (default `1`, `0` = unbounded) and C-13's `reverify` set over `{id}` `walked`: one line per walked id in the walk's own order as `<id> (depth <n>, via <src> -<kind>-> <dst>)`, then one line per T id that verifies the id or a walked id as `<T-id> verifies <id>[, <id>...]` (the obligations it verifies among them, ascending); the heading alone when both are empty.
- **Exit.** `0` once the trace is written — nothing it renders is pass/fail, like `impact`; `2` for a usage error

(E-60 included); 3 for an input-contract violation (§5.4).
- **Determinism.** No timestamp, duration, absolute path or locale-dependent value appears in the trace; stdout is UTF-8 (R-29), and the trace is the only thing written to stdout by an `explain` run.

C-19 Help contract (v1.18; R-41)

Every argument definition of the four parsers (`verify` added in v1.25) except `-h` carries a non-empty help string. One flag's entry is one paragraph, in this order:

1. **Purpose** — one sentence saying what the flag does.
2. **Values** — for a finite set, the tokens in the flag's own usage-error spelling (`none, mock, or llm`; `auto, always, or never`; `INFO/DEBUG`; `SECONDS` or `N%`); for a bounded numeric value, its range (`a decimal in [0, 1]`, `an integer 1..32`, `an integer 0..999`).
3. **Default** — `default: <value>` for every optional flag that has one, including `check`'s absent-flag-only `src/tests` directory default and `impact`'s/`explain`'s absence of one.
4. **Preconditions** — an `ignored unless …` / `requires …` clause wherever §5.1's row carries one (the judge flags under `--judge llm`, `--judge-budget N%` under `--jev-pre-triage`, `--root` containment, E-58).

The flag's metavar MUST be the value vocabulary of §5.1's synopsis — `PATHS`, `FILE`, `DIR`, `IDS`, `MODE`, `FRACTION`, `N`, `SECONDS|N%`, `LEVEL`, and `NS` (v1.23) — never the argparse destination name. `--manifest` on `check` is a boolean and has no metavar; on `verify` it takes `FILE` (v1.25).

Each `--help` screen additionally carries:

- an **`environment:` block** in its epilog, one entry per variable the kernel reads: its name(s), the condition under which it is read, whether it is required, and its default; the names are exactly the `SPECCHECK_*` literals `src/speccheck/*.py` reads (T-98), plus one clause for `COLUMNS`, the only variable that changes a `--help` run's output (its line wrapping; D-41), and one for `SOURCE_DATE_EPOCH` on `check`'s screen: read only with `--manifest`, optional, a non-negative integer, no default (v1.25, C-25). No entry ever carries a value (R-23, I-007).
- an **exit-code epilog**: the §5.4 set (`0` conforming, `1` not conforming, `2` usage, `3` input contract) and the §5.1 summary line, so an operator or agent can construct and interpret a run from the help alone (D-40). `verify`'s screen (v1.25) states its own meanings instead: `0` every file matches, `1` a file is missing or differs, `2` usage, `3` malformed manifest, and the C-26 output lines.

The help's value tokens are the *validator's* tokens: for every flag with a finite accepted set, the tokens its help names equal the tokens its own usage error names (T-95), and the tokens are checked against the parser, not against this paragraph. Exact wording is not normative (D-38); the rendered bytes are pinned by T-97's golden at `COLUMNS=80`, and a wording change is a test diff, not a spec revision. `--help` itself is inert: it exits `0`, writes only to stdout, and reads no project file, no environment variable and no socket (I-017).

C-20 The lean adapter (v1.19; R-100)

`lean.py`, beside `swift.py`, splits a Lean source on declaration heads — lines matching `theorem`, `lemma`, `def`, `abbrev` or `instance` (each optionally `public`/`protected`-prefixed), one declaration per head — and, for each declaration, reads the spec ids from the doc-comment block ending within one line above the head: every `[RCIKE]-nn` id inside a bold span (`**C-02**`, `**R-11, I-003**`; a mixed list `**E-08, D-17, R-03**` contributes its R/C/I/K/E members only), with `(a)`-style parentheticals stripped. A directory named by `--proof` is scanned recursively for `*.lean`. A file whose declarations carry no tag contributes no citation; a file that is unreadable, not Lean source, or yields no parseable declaration head contributes a `notes` entry, not an error (E-62). The adapter never reads a declaration body — it is a citation scanner over doc comments, exactly as `swift.py` is a citation scanner over test-case delimiters, not a semantic check on what the declaration proves (§0 non-goal).

C-21 The proof manifest (v1.19; R-100)

{

```
  "build": {"exit": 0, "theorems_total": 66, "theorems_checked": 66, "errors": []},
  "theorems": [
    {"file": "HelloProof/Hello/Theorems.lean", "name": "rowInterrupt", "line": 105,
     "ids": ["C-04", "E-03", "R-09"], "status": "checked"}
  ],
  "deferred_to_pytest": [
    {"id": "R-09", "scope": "process side", "content": "real SIGINT delivery, plain 130 status",
     "tests": ["T-20"]}
  ]
}
```

speccheck reads exactly these three top-level keys and ignores every other one the file carries — a producer MAY write provenance keys (`spec`, `source`, `proof_repo`, `pytest`) for a human reader; they are not part of this contract, and neither is any extra key on a `theorems/` `deferred_to_pytest` entry (`speccheck/tools/proof_evidence.py` writes both today). `build.errors` is a list of `"<file>:<line>"` strings. A manifest theorem joins a proof citation (C-20) on name and file (line, where present; K-17): a citation without a matching manifest entry is `unknown`, a manifest entry without a citation is a `notes` entry. `--proof-results FILE` given but unreadable, not well-formed JSON, or lacking `build`/`theorems` → E-65.

C-22 Foreign reference and namespace (v1.23; R-101, R-102)

```
FOREIGN   :=
 ↪  (?<![A-Za-z0-9])(?P<ns>[A-Za-z][A-Za-z0-9_-]*):(?P<id>[RCIKET]-[0-9]{1,3})(?![A-Za-z0-9])
            with the added condition that `ns` does not match ^[RCIKET]-[0-9]{1,3}$.
            Recognition has no length cap (v1.23.1, F-712: with v1.23's {0,31} a 33-character
            label made no match and its id fell back to a local citation); the 32-character limit
            belongs to the --namespace VALUE only (R-102), so a longer `ns` is always foreign.
Evaluated on every line before the TOKEN rule (C-01), in SPEC.md statements and Affects cells as
well as in --src / --tests files. A TOKEN that is the `id` of a FOREIGN match whose `ns` is not
the --namespace value is a FOREIGN REFERENCE:
  - in a scanned source or test file: no citation (so no dangling or stale entry), recorded in
    the `foreign` array below, counted in one Note (E-67): `<N> foreign reference(s) ignored: `
 ↪  followed by the first three
    listed tokens, in `foreign` order, each `<ns>:<id> <file>:<line>`, comma-separated, and ` and
 ↪  <N-3> more`
    when N > 3;
  - in a declared statement or Affects cell: no C-12 edge and no Note (E-69); never listed.
With --namespace NS, a FOREIGN match whose `ns` == NS (case-sensitive) is not foreign: its TOKEN is
a local citation / edge target exactly as if the `NS:` were absent (E-68). A bare TOKEN (no `ns:`)
is always local, with or without --namespace (D-52). `speccheck:ignore` (C-01) applies to a
foreign reference as to any token on the line. Fenced code blocks in SPEC.md are not scanned for
statements (C-01), so a foreign token there is irrelevant.
JSON: top-level `foreign` = [ {"namespace","id","file","line"} ... ] sorted by
  (file, line, namespace, id); the key is present only when the array is non-empty.
```

C-23 The citation map (v1.23; R-103)

```
File: UTF-8 JSON, resolved inside --root (E-09), at most 2 MiB (K-19).
  {"schema_version": "1.0",
   "citations": [ {"id": "R-99", "file": "tests/test_core.py", "test": "test_zero_rate"}, ... ]}
  id    a bare TOKEN form (leading zeros do not matter; I-011); a `ns:`-prefixed id is E-70.
  file  a POSIX path relative to --root, exactly as it appears in `tests[].file`.
  test  the TestCase.name (C-03) to attribute to; "" selects the file-level case of `file`.
  Unknown top-level keys are ignored; an unknown or missing key in an entry is E-70.
Resolution (after the --tests scan, before C-05): the entry resolves when a TestCase with that
  `file` and `name` exists among the scanned cases (a file-level case always exists for a scanned
  file). Two cases with the same file and name (E-13 / E-27's ties) resolve to the one with the
  smallest `start`. Resolved -> Citation(id, file, line = case.start, kind "test", testcase,
  declared = true, mapped = true), unless a literal citation of the same (id, testcase) exists, in
  which case the entry adds nothing and the literal citation keeps mapped = false (E-73).
```

```
  An id the spec does not declare -> a dangling citation (R-07) at (file, case.start); a retired id
  -> a stale citation (R-15's stale rule) there; either entry carries `"mapped": true`, and the
  Markdown renders its location `<file> (map)`, so a reader looks in the map, not at that line
  (F-704). An unresolved entry (E-71) adds nothing.
  `test` matches `TestCase.name` exactly, and a Python method's name carries no class (C-03): two
  same-named methods of different classes in one file are not distinguishable by an entry, which
  resolves to the one with the smaller `start`; the other cannot be mapped (a stated limitation,
  F-716; cite it literally instead).
JSON: when --citation-map was given,
  top-level `citation_map` = {"file": <path as given>, "entries": N, "resolved": R, "unresolved":
↪   U}
    (R counts entries that resolved, E-73 duplicates included; R + U == N);
  every `tests[]` entry gains `"mapped": true|false` after `declared`; a mapped entry's `lines` is
↪   [];
  every `dangling` and `stale` entry gains `"mapped": true|false` after `line` (F-704).
  Without the flag neither key exists (I-019). The Markdown report (C-08) renders a mapped citation
↪   as
  `<file> (map)`. `schema_version` does not change: both keys are conditional (C-07).
Summary line (§5.1): gains `; citations: <M> mapped` after the judge-suffix when M > 0, where M is
  the number of Citations with mapped = true; absent when M == 0, even with the flag.
```

C-24 The JavaScript/TypeScript adapter (v1.22; R-104; moved from C-03 in v1.24)

```
JavaScript/TypeScript adapter (files ending .ts .tsx .mts .cts .js .jsx .mjs .cjs under a --tests
↪   root;
  R-104, D-56). The file is tokenized, not parsed: comments (`//`, `/* */`), string literals ('…',
↪   "…"),
  template literals (including `${…}` bodies, nested) and regular-expression literals are single
  opaque tokens, identifiers and one-character punctuation are the rest, and every rule below reads
  tokens, never raw text.
    REGEX vs DIVISION (F-708): a "/" that does not start a comment begins a regular-expression
                literal iff the previous significant token (comments skipped) is absent, or is
                one-character punctuation other than ")", "]" and "}", or is one of the
↪   identifiers
                return typeof instanceof in of new delete void throw case do else yield await;
                otherwise it is the division operator (one punctuation token). A regex literal
↪   ends
                at the next unescaped "/" outside a "[ … ]" class, then takes any following
                identifier characters as flags; a line end before that "/" makes it unterminated
                (E-74). A regex opening a statement right after "}" is misread as division; the
                worst cost is a parse fallback.
    CALL      := an identifier token `describe`, `suite`, `it` or `test` that is not preceded by a
↪   `.`
                token (so `pattern.test(x)` is not a call), followed by zero or more `.modifier`
                where modifier is one of only skip todo concurrent sequential fails failing
↪   shuffle
                each for runIf skipIf (any other modifier ends the match: it is not a call), then
↪   a `(`.
                After `each`, `for`, `runIf` or `skipIf` one further group is skipped first: a
                balanced `( … )`, or a template-literal table (`it.each\`…\`(…)`).
    TITLE     := the token right after that `(`, when it is a string literal or a template literal
↪   with
                no `${}`, and not empty. Any other first argument (a variable, an interpolated
                template) is DYNAMIC.
    CALL END  := the `)` token that balances the CALL's `(`.
  A test case is a CALL of `it` or `test` with a TITLE.
        name := the TITLES of every `describe`/`suite` CALL whose parentheses enclose it (outermost
                first) followed by its own, joined with " > " -- the string vitest writes as the
↪   JUnit
                `name` ("calc > inner (x) > subtracts [neg]"). A `describe` with a DYNAMIC title
```

```
               contributes nothing to the chain.
       classname := the file path relative to --root with "/" -> "." and the extension KEPT
               ("web/src/calc.test.ts" -> "web.src.calc.test.ts"; Python's drops it).
  span := the first line of the contiguous run of comment-only lines immediately above the CALL's
    first token (a line holding comment text and no code token; no blank line in the run) .. the
↪   line
    of its CALL END. A citation inside a span is attributed to that case; every other citation is
    file-level. A citation on a comment-only line is DECLARED (C-14); a comment trailing code is
↪   not.
  A CALL of `it`/`test` with a DYNAMIC title is not a case: its citations are file-level and the
↪   file
    gets one Note "undelimited tests in <path>: line N, line M" (E-75; E-43's rule). A `describe`
    with a DYNAMIC title adds its line to the same Note.
  A file whose parentheses do not balance, or whose block comment, string or template literal is
    unterminated, is one file-level case with Note `parse fallback: <path>` (E-74; E-12's rule).
  What the join sees (measured 2026-10-27, vitest 2.1.8, `vitest run --reporter=junit`): one
    <testcase classname="src/calc.test.ts" name="calc &gt; inner (x) &gt; subtracts [neg]"> per
    test, the classname the path RELATIVE TO VITEST'S ROOT (not necessarily --root), `describe`
    titles joined with " > ", an `it.each` row expanded to one <testcase> per row with its
    placeholders filled ("sums 1 and 2" for "sums %i and %i"), a skipped test carrying <skipped/>.
    jest-junit matches when configured `classNameTemplate: "{filepath}"`,
    `titleTemplate: "{classname} > {title}"` and `ancestorSeparator: " > "` (its defaults do not:
    classname "describe title", and nested describe titles joined by one space; F-707). Measured
    2026-10-27 (T-117, jest 30.5.2, jest-junit 17.0.0): under that configuration a nested test is
    `name="calc > inner (x) > subtracts [neg]"`, without `ancestorSeparator` it is
    `"calc inner (x) > subtracts [neg]"` (no join), and a test outside every `describe` is
    `" > top level"`, which C-04's leading-separator rule joins.
```

C-25 The provenance manifest (v1.25; R-106)

```
File: <out>/speccheck.manifest.json, written by `check --manifest` only (D-61), in the §3.1
  temp-and-rename sequence with the two reports. Serialized as speccheck.json is (C-07): UTF-8,
  two-space indent, "\n" line endings, a trailing newline, keys in exactly this order:
{
  "manifest_version": "1.0",
  "tool": {"name": "speccheck", "version": "<__version__>"},
  "options": {"judge": "none|mock|llm", "strict": false, "max_unknown": 0.2000,
              "namespace": null, "judge_model": null, "judge_prompt_sha256": null},
  "inputs": [ {"path": "SPEC.md", "role": "spec", "sha256": "<64 hex>", "bytes": 123456}, ... ],
  "tree_sha256": "<64 hex>",
  "reports": [ {"path": "out/SPEC_CONFORMANCE_REPORT.md", "sha256": "<64 hex>", "bytes": 2048},
               {"path": "out/speccheck.json",             "sha256": "<64 hex>", "bytes": 9000} ],
  "generated_at": "2026-11-05T00:00:00Z"            // only when SOURCE_DATE_EPOCH is set (D-62)
}
options   `namespace` is the --namespace value or null; `judge_model` is SPECCHECK_JUDGE_MODEL
          verbatim and `judge_prompt_sha256` is C-07's value, both null unless --judge llm;
          `max_unknown` is emitted as C-07 emits it (four places). No path flag appears here:
          every path is in `inputs`.
inputs    one entry per (file, role) the run read: role "spec" (--spec), "src" / "tests" /
          "proof" (each file a --src / --tests / --proof scan selected, after the C-03 exclusions,
          including one K-02/E-10 or E-29 skipped, hashed over its full bytes, because its Note
          is part of the report), "results", "citation_map", "proof_results", "risk_file".
          A file selected under two roles is listed once per role. A file excluded by C-03, or
          skipped as a symlink (E-11), was not read and is not listed. Sorted by (path, role),
          both by code point.
path      --root-relative, "/" separators (R-20), never absolute.
sha256    lower-case hex SHA-256 of the raw bytes read, before C-01's CRLF normalization: a CRLF
          twin of SPEC.md yields the same reports (I-002) and a different manifest.
bytes     the length of those bytes.
tree_sha256  SHA-256 of the UTF-8 text made of one line "<sha256>  <path>\n" (two spaces: the
```

```
          `shasum -a 256` line format) per DISTINCT path in `inputs`, in code-point order of path,
          so that `shasum -a 256 <files> | LC_ALL=C sort -k2 | shasum -a 256` reproduces it.
reports   the two report files of this run, Markdown first, paths --root-relative; hashes are of
          the bytes this run renamed into place.
generated_at  present only when SOURCE_DATE_EPOCH is set: its value, a non-negative decimal
          integer of seconds since the Unix epoch, as RFC 3339 UTC with a "Z" suffix and no
          fractional part. Any other value is E-85. No wall-clock time, host name, user name,
          duration or absolute path appears anywhere in the manifest (D-62).
Under --judge none and --judge mock the manifest is byte-identical for identical inputs (I-020).
Under --judge llm it records which reports were produced, not that a re-run reproduces them.
```

C-26 `verify` (v1.25; R-107)

```
speccheck verify --manifest FILE [--root DIR] [--verbose [INFO|DEBUG]]
  --manifest FILE  required; a C-25 manifest, resolved inside --root (E-09's rule; E-82 otherwise).
  --root DIR       default "."; every `path` in the manifest is resolved against it.
Reads the manifest, then every `inputs[].path` (each distinct path once) and every
 ↪  `reports[].path`,
  in that order, and compares SHA-256 and byte length with the recorded values. Re-hashes only:
  it does not parse SPEC.md, does not re-run `check`, and does not read `tool`, `options` or
  `generated_at` except to validate their presence (D-65). Writes no file; opens no socket.
stdout, one line each, ASCII except for the paths, which are UTF-8 (R-29):
  verify: missing <path>          the file does not exist or cannot be read
  verify: mismatch <path>         hash or length differs
    one such line per failing file, inputs first then reports, each in manifest order; then
  verify: FAILED - <k> of <N> files differ                                    -> exit 1
  or, when nothing failed, the single line
  verify: ok - <N> files match, tree <the first 12 hex digits of tree_sha256>      -> exit 0
  <N> = the number of distinct paths in `inputs` + the number of `reports` entries; <k> <= <N>.
  A path present in `inputs` under two roles is checked and counted once.
Faults: --manifest missing, unreadable or outside --root -> E-82 (exit 2); malformed -> E-83 (exit
 ↪  3).
  `tree_sha256` is not recomputed: it is a digest of the per-file hashes that are checked.
```

C-27 The CONTROLS marker and the hazard roll-up (v1.25; R-108)

```
CONTROLS := "*(controls:" WS? HAZID ( WS? "," WS? HAZID )* WS? ")*"
HAZID    := [A-Za-z0-9][A-Za-z0-9_.-]{0,63}            # at most 64 characters; case-sensitive
WS       := one or more spaces or tabs
Position (C-01): beginning at the token after the ID form, or after the RECORDED marker. One
  CONTROLS marker counts per declaration; a second one later in the decoration is decoration.
SpecId.controls := the HAZIDs, de-duplicated, sorted by code point. () on a T id (E-78) and
  when the text in that position begins "*(controls:" but fails the grammar (E-80).
HAZARD SET H := every HAZID in any SpecId.controls (retired ids included)   every id in the
  --risk-file (C-28).
For each h in H:  CTRL(h) := the NON-RETIRED declared ids whose controls contain h, in C-07 id
 ↪  order.
  UNCONTROLLED  if CTRL(h) is empty  (no marker names h, or every id naming it is retired)
  VERIFIED      else if every id in CTRL(h) has C-05 status PASSING   (WEAKLY_PASSING is not
 ↪  PASSING)
  NOT_VERIFIED  otherwise
  The statuses are computed after C-05 step 5, from the final statuses; nothing flows back.
HazardRecord := {"id": h, "title": the risk file's title, or null, "controls": CTRL(h), "status"}
JSON: per-id `controls` and top-level `hazards` (sorted by `id`, code point), per C-07's hazard
 ↪  keys.
Markdown: `## 10. Hazards` per C-08. Both present only when H is non-empty or --risk-file was
 ↪  given.
Summary line (§5.1): when `hazards` is present, the line ends "; hazards: <V>/<H> verified" - <V>
 ↪  the
  VERIFIED count, <H> = |H| - after the C-23 mapped suffix when both apply ("0/0" when --risk-file
```

```
  lists nothing and no marker exists).
--strict (R-15): an UNCONTROLLED hazard causes exit 1 (D-64); VERIFIED and NOT_VERIFIED add
  nothing, since a NOT_VERIFIED hazard has a non-PASSING in-scope control that R-15 already counts.
Notes: E-79 and E-81 at most once per run; E-78 and E-80 once per declaration they name.
  None of the four is emitted by `impact` (it ignores the marker, below).
`impact` parses the marker (it is C-01 grammar) and ignores it: no hazard appears in `impact.json`
  or `IMPACT_REPORT.md`, and a changed marker is not a C-13 change.
```

C-28 The risk file (v1.25; R-109)

```
--risk-file FILE  check only. UTF-8 JSON, resolved inside --root (E-09), excluded from every scan
  (C-03), at most 2 MiB (K-20). Read after extract-spec and before graph + status (§3.1).
  {"hazards": [ {"id": "HAZ-003", "title": "Wrong measurement shown"}, ... ]}
  id     a HAZID (C-27); required.
  title  a string; optional (null in the report when absent).
  Unknown keys, top-level or per entry, are ignored. Order in the file is free; the report sorts.
Meaning: the file lists exactly the hazards whose risk control is allocated to the software this
  SPEC.md specifies (an export of the user's ISO 14971 risk management file, filtered). Hazards
  controlled elsewhere (hardware, labelling, another software item) are left out of it; listing
  one makes it UNCONTROLLED here and fails --strict (E-81, D-64).
Faults: unreadable -> exit 2 (`--risk-file: cannot read <path>`); not JSON, `hazards` absent or not
  a list, an entry not an object, `id` missing or failing HAZID, `title` present and not a string,
  a duplicate `id`, or over K-20 -> E-84 (exit 3), before C-05 runs and before any report is
 ↪ written.
An empty list is valid: H is then the marker hazards alone, and `hazards` is present.
```

---

5. Interface specification

5.1 CLI (speccheck), the only surface

```
speccheck check --spec SPEC.md [--src PATHS]... [--tests PATHS]... [--results junit.xml]
                [--proof PATHS]... [--proof-results FILE] [--namespace NS] [--citation-map FILE]
                [--root DIR] [--out DIR] [--judge none|mock|llm] [--strict]
                [--judge-concurrency N] [--judge-budget SECONDS|N%] [--jev-pre-triage]
                [--max-unknown FRACTION] [--progress auto|always|never] [--verbose [INFO|DEBUG]]
                [--manifest] [--risk-file FILE]
speccheck impact --spec SPEC.md (--changed IDS | --against OLD_SPEC.md) [--src PATHS]... [--tests
 ↪ PATHS]...
                 [--namespace NS] [--citation-map FILE] [--root DIR] [--out DIR] [--depth N]
 ↪ [--verbose [INFO|DEBUG]]
speccheck explain ID --spec SPEC.md [--src PATHS]... [--tests PATHS]... [--results junit.xml]
                 [--namespace NS] [--citation-map FILE] [--root DIR] [--judge none|mock|llm]
 ↪ [--depth N] [--judge-concurrency N]
                 [--judge-budget SECONDS|N%] [--jev-pre-triage] [--max-unknown FRACTION]
                 [--progress auto|always|never] [--verbose [INFO|DEBUG]]
speccheck verify --manifest FILE [--root DIR] [--verbose [INFO|DEBUG]]
speccheck --self-check [--verbose [INFO|DEBUG]]
speccheck --version
speccheck --help
```

Subcommand / flag	Behavior	Exit
`check`	Run the §3.1 pipeline; write both reports; print the summary line.	`0` conforming, `1` not, `2` usage, `3` input contract
`--spec FILE`	Required on `check` and `explain` alike (v1.20; on `explain`, a missing flag is E-64's own message). Path to the specification; decoded as UTF-8 with `errors="replace"` (a Note is recorded if any byte was replaced). Missing flag or unreadable file → usage error.	`2`
`--src PATHS`	Repeatable; each value is a **PATHS list** — comma-separated files and/or directories per C-03, D-23. A resolved path is descended if a directory, scanned as one file otherwise; a segment inside `--root` that is neither a file nor a directory → usage error (E-52). Default: `src` if the directory exists, else none, only when the flag is absent.	`2`
`--tests PATHS`	As `--src` — a comma-separated list of files/directories (C-03, D-23) — but for test files and their case attribution; a directly-named `.swift` file's `MODULE` is its parent's last path component (D-18, D-23). Default: `tests` if the directory exists, else none, only when the flag is absent.	`2`
`--results FILE`	Optional JUnit XML. Absent → every cited ID is at most `UNVERIFIED`. Unreadable → usage error; malformed → E-05.	`2` / `3`
`--proof PATHS`	`check`-only (v1.19, R-100). Repeatable; each value a comma-separated PATHS list of Lean source files/directories, scanned per C-20 and resolved inside `--root` (E-09). Absent → no Lean file scanned, no `proof` key written (I-018).	`2`
`--proof-results FILE`	`check`-only (v1.19, C-21). A JSON proof manifest, read and joined to the C-20 citations (K-17); the checker never runs a prover. Absent → every citation's state is `unknown` (no manifest to confirm it). Unreadable, malformed, or missing `build/theorems` → E-65.	`2` / `3`
`--namespace NS`	`check`, `impact`, `explain` (v1.23, R-102). At most once (a second occurrence → exit `2`, `--namespace: given more than once`). One value matching `[A-Za-z][A-Za-z0-9_-]{0,31}` and not shaped like an ID, else exit `2`, message `--namespace: invalid namespace: <value>`. Tokens written `NS:ID` are then local citations (C-22, E-68). Absent → every `<ns>:`-prefixed token is foreign (R-101); a run with none is unchanged (I-019).	`2`
`--citation-map FILE`	`check`, `impact`, `explain` (v1.23, R-103, C-23). A JSON citation map, resolved inside `--root` (E-09), read after the `--tests` scan; each resolving entry is a mapped citation. Absent → no map is read and no `citation_map` key is written (I-019). Unreadable → E-70 (`2`); malformed, wrong `schema_version`, bad entry or over K-19 → E-70 (`3`).	`2` / `3`
`--root DIR`	Base for relative paths in reports; default `.`. Every path flag — `--spec`, `--src`, `--tests`, `--results`, `--out`, `--proof`, `--proof-results`, `--against`, `--citation-map`, `--risk-file`, and `verify`'s `--manifest` (v1.25) — MUST resolve inside it (E-09).	`2`

Subcommand / flag	Behavior	Exit
`--out DIR`	Where the two reports go; default `.`. MUST resolve inside `--root` (E-09; F-002). Created if missing; not writable → E-18.	2 / 3
`--judge MODE`	`none` (default), `mock`, `llm`. Any other value → usage error.	2
`--strict`	Apply R-15 and, with `--judge llm`, R-28.	—
`--max-unknown FRACTION`	Threshold for R-28; default `0.2`; a float in `[0, 1]`, else usage error. Accepted with any judge mode; consulted only under `--strict --judge llm`.	2
`--judge-concurrency N`	K-06; default `4`, integer `1..32`, else usage error. Ignored unless `--judge llm`.	2
`--judge-budget SECONDS|N%`	K-12; default `0` (unlimited). `SECONDS`: integer `0..86400`. `N%`: an integer `0..100` followed by `%`, accepted only with `--jev-pre-triage` under `--judge llm` (else E-58, exit 2). Any other value → usage error. The `SECONDS` form is ignored unless `--judge llm`; the `N%` form is not ignored anywhere — it either applies or is a usage error.	2
`--jev-pre-triage`	K-16 (v1.15); boolean, default off. Runs the C-17 triage pass and orders the judge's issue queue by it. Ignored unless `--judge llm`: under `--judge none`/`mock` no Jev request is made and no C-17 variable is read. Requires `SPECCHECK_JEV_API_KEY` when it runs (E-21).	2
`--progress MODE`	R-30, C-11. `auto` (default): draw the judge progress indicator iff `sys.stderr.isatty()` is true and verbosity is not `DEBUG`; `always`: draw it even when stderr is not a TTY (still suppressed under `--verbose DEBUG`, E-39); `never`: never draw it. Any other value → usage error. Ignored unless `--judge llm`.	2
`--verbose [LEVEL]`	§5.3. Bare = `INFO`. `LEVEL` other than `INFO`/`DEBUG` → usage error.	2
`--self-check`	Copy the packaged fixture `speccheck/_selfcheck/` (a byte-identical copy of the §9.8 golden fixture `fixtures/target/`, its `golden/` directory included; F-107) into a fresh temporary directory `<tmp>`; install a socket guard; then run, **in the same process**, exactly `check --spec SPEC.md --src src --tests tests --results junit.xml --judge mock --strict --root <tmp> --out <tmp>/out` with the working directory set to `<tmp>` (Q-003). The inner check is **expected** to exit `1` (the fixture has planted defects) and that exit code is not the self-check's result; compare `<tmp>/out/speccheck.json` and `<tmp>/out/SPEC_CONFORMANCE_REPORT.md` to `<tmp>/golden/` byte-for-byte; remove `<tmp>`; print `self-check: ok` when both match, else one line naming the first mismatch or failure (F-011).	0 / 1
`--version`	Print `speccheck <semver>` and exit.	0
`impact`	v1.13 (R-37, C-13). Run the §3.1 `impact` pipeline; write `impact.json` and `IMPACT_REPORT.md` under `--out`; print the C-13 summary line. Accepts `--spec`, `--src`, `--tests`, `--namespace`, `--citation-map` (v1.23), `--root`, `--out`, `--depth`, `--verbose` with the meanings above, except that `--src`/`--tests` have **no directory default** here: absent means not scanned (C-13), since the scan is optional for `impact`; no `--results`, no judge flags, no `--proof`/`--proof-results` (v1.19) — any of them → usage error (E-54).	0 written, 2 usage, 3 input contract
`--changed IDS`	`impact` only. Comma-separated ids (R/C/I/K/E/T or D), each declared in `--spec`; an undeclared one → E-53; empty → usage error. Mutually exclusive with `--against` (E-54).	2

Subcommand / flag	Behavior	Exit
`--against FILE`	`impact` only. A prior version of the spec, inside `--root` (E-09), parsed by C-01; the changed set is the C-13 diff. Unreadable → usage error; its E-01/E-02/E-03 → `3` with `--against:` prefixed. Mutually exclusive with `--changed` (E-54).	`2` / `3`
`--depth N`	`impact` (v1.13) and `explain` (v1.17). Integer `0..999`, default `1`; `0` = unbounded (C-13; for `explain`, the reach of C-18's `impact (<depth>):` section). Other values → usage error.	`2`
`explain ID`	v1.17 (R-40, C-18). Run the §3.1 pipeline over the same inputs as `check` and render the C-18 trace for the one id `ID` to stdout; write no report file. Accepts `--spec`, `--src`, `--tests`, `--results`, `--namespace`, `--citation-map` (v1.23), `--root`, `--judge`, `--judge-concurrency`, `--judge-budget`, `--jev-pre-triage`, `--max-unknown`, `--progress`, `--verbose` and `--depth` with the meanings above, and `ID` as a positional argument (one id per invocation, D-36). No `--out`, no `--strict`, no `--proof`, no `--proof-results` (v1.19): they are not defined on this subparser, so argparse rejects them as unrecognized arguments (E-54's pattern). An undeclared `ID` → E-60.	`0` rendered, `2` usage, `3` input contract
`--manifest`	`check`-only, boolean, default off (v1.25, R-106, C-25, D-61). Also write `speccheck.manifest.json` under `--out`: the tool version, options, and the SHA-256 and length of every file read and of both reports. Reads `SOURCE_DATE_EPOCH` when set (E-85). Given to `impact` or `explain` → usage error.	`2`
`--risk-file FILE`	`check`-only (v1.25, R-109, C-28). A JSON list of the hazards whose control is allocated to this software; every listed hazard no non-retired id controls is `UNCONTROLLED` and fails `--strict`. Absent → only CONTROLS markers (C-27) define hazards, and no `hazards` key exists without one (I-021). Unreadable → `2`; malformed, duplicate id, or over K-20 → E-84 (`3`). Given to `impact` or `explain` → usage error.	`2` / `3`
`verify`	v1.25 (R-107, C-26). Re-hash every file a C-25 manifest names and print `verify: ok - …` or one `missing/mismatch` line per failing file and `verify: FAILED - …`. Accepts `--manifest FILE` (required), `--root`, `--verbose`; any other flag → usage error. Writes nothing, reads no network.	`0` all match, `1` a file missing or differs, `2` usage (E-82), `3` malformed manifest (E-83)

Every row of this table is also stated, in the form C-19 pins, in the `--help` output of the subcommand that defines the flag (v1.18, R-41); the table stays normative and T-95 checks the values against the validator's own usage-error text, so the two cannot disagree about an accepted token.

The `--src`/`--tests` PATHS list (D-23). Each flag value is a comma-separated list, one list per occurrence. To build the effective list the extractor walks, each occurrence is split on the literal `,`, every resulting segment is whitespace-trimmed and empty segments dropped; the surviving paths are resolved inside `--root` (E-09) and deduplicated (I-012). Each resolved path is a directory (descended as in v1.11: K-02/K-03, E-10/E-11/E-13/E-29/E-30/E-33 apply) or a regular file (scanned as one, no descent); a segment that resolves inside `--root` but exists as neither is a usage error (E-52), replacing the former "non-directory → usage error". The full rule — including the absent-flag-only default, the directly-named-file Swift `MODULE`, and the determinism of the emitted order — is C-03 *PATHS*; the summary-line regex, both

report files, and the exit codes are otherwise unchanged for two inputs that name the same file set. Example: `--src a.py,b.py,src --tests tests/unit,tests/core` is equivalent to `--src a.py --src b.py --src src --tests tests/unit --tests tests/core` (the list is repeatable), and `--src a.py,,src,` trims and drops to `[a.py, src]`.

Summary line (stdout, exactly one line, only on exit 0/1):

```
speccheck: <STATUS> - <passing>/<in_scope> passing (<pct>%), <failing> failing, <skipped> skipped,
↪ <weak> weak, <unverified> unverified, <untested> untested, <uncited> uncited; <dangling>
↪ dangling, <stale> stale; judge=<mode><judge-suffix>
```

The line above is shown unwrapped on purpose: it is one physical line, ASCII only, terminated by a single `\n`, and written to stdout as UTF-8 regardless of locale (R-29; F-010). `<STATUS>` is `CONFORMING` (exit 0, and R-15's conditions hold), `UNPROVEN` (exit 0 and they do not — only possible without `--strict`; R-105, v1.24) or `NOT CONFORMING` (exit 1); `<pct>` is `conformance × 100` rounded half-even to one decimal; `<judge-suffix>` is empty unless R-28 forced exit 1, in which case it is `(unavailable)` or `(unknown_rate <rate> > max_unknown <max>)` with both numbers as in the JSON; when both R-28 reasons hold the suffix is `(unavailable)` (Q-004). After the judge-suffix, and only when at least one citation is mapped (C-23), the line ends `; citations: <M> mapped` (v1.23). After that, and only when the JSON `hazards` key is present (C-27), the line ends `; hazards: <V>/<H> verified` (v1.25). The seven counts after the ratio are the seven in-scope statuses of C-07 `by_status`, each exactly once (Q-005).

Regex the line MUST match (T-44):

```
^speccheck: (CONFORMING|NOT CONFORMING|UNPROVEN) - \d+/\d+ passing \(\d+\.\d%\), \d+ failing, \d+
↪ skipped, \d+ weak, \d+ unverified, \d+ untested, \d+ uncited; \d+ dangling, \d+ stale;
↪ judge=(none|mock|llm)( \(unavailable\)| \(unknown_rate \d\.\d{4} > max_unknown \d\.\d{4}\))?(;
↪ citations: \d+ mapped)?(; hazards: \d+/\d+ verified)?$
```

5.2 GUI

None. This system has no graphical surface, and O-3 explicitly leaves one out of v0.1.

5.3 Diagnostics and verbosity contract

- **Quiet by default.** With no `--verbose`, the process writes nothing to stderr on exit 0/1 and only the usage/contract error message on exit 2/3. Sole exception: the C-11 progress indicator under `--judge llm` (R-30), which is written directly to the stream, is redrawn in place, and is erased before the judge stage ends — so what *remains* on stderr at exit is still nothing (or only the error message). It is never routed through the logger.
- **Syntax.** `--verbose` alone ≡ `--verbose INFO`; `--verbose INFO` and `--verbose DEBUG` are accepted; any other value is a usage error (exit 2).
- **INFO** (stderr): one line per pipeline stage with counts and elapsed milliseconds (files scanned, IDs declared, citations found, test cases attributed, results joined, edges judged), the judge mode, and for the LLM judge the URL and model. INFO MUST NOT include file contents, statement text, judge prompts, judge responses, or the API key.
- **DEBUG** (stderr): INFO plus, per judged edge, the full `JudgeRequest` JSON as sent and the raw provider response as received, each prefixed `judge>` / `judge<` on its own line. The API key is redacted. DEBUG MUST NOT change stdout or either report file. Because these lines are emitted *during* the judge stage, the progress indicator is not drawn at DEBUG regardless of `--progress` (E-39); the per-edge lines are the progress record at that level.
- **Mechanism.** Python `logging` with a single stderr `StreamHandler`; logger name `speccheck`; format `%(levelname)s %(message)s`. Level is `ERROR` by default and `INFO`/`DEBUG` per flag, so nothing below ERROR reaches stderr without `--verbose` (F-014). The E-10..E-13 and E-27..E-30 Notes are logged at `INFO` (they are always in the report's Notes regardless of level); nothing is ever logged at `WARNING`; the exit-2/3 message is logged at `ERROR`. While the C-11 indicator is displayed, the logger emits nothing: the judge stage's INFO lines (judge mode, URL and model, the stage summary) are written after the erase (K-13; F-201).

5.4 Exit codes

Code	Meaning	Reports written?
0	CONFORMING, or UNPROVEN when R-15's conditions do not hold and --strict is off (R-105; the code is 0 either way): no in-scope ID is FAILING; and if --strict, every in-scope ID has status PASSING, dangling and stale are empty, no hazard is UNCONTROLLED (C-27; v1.25), and — with --judge llm only — judge_available is true and unknown_rate ≤ max_unknown (R-28). Beyond R-28, status is the only judge-related input (F-003).	yes
1	NOT CONFORMING: anything else the report can express	yes
2	Usage error (bad flag/value, missing required, path outside root, missing judge env)	no
3	Input contract violation — every E row whose Semantics column exits 3: E-01, E-02, E-03, E-05, E-18, E-65 (malformed manifest), E-70 (map shape faults), E-84 (malformed risk file; v1.25)	no

Any uncaught exception MUST also map to 3 with a one-line message; a traceback is printed only under --verbose DEBUG. KeyboardInterrupt is treated the same way, with the message interrupted (E-41).

impact (v1.13) uses 0 (both C-13 reports written; there is no 1: nothing it reports is pass/fail), 2 (usage, E-53, E-54), and 3 (E-01/E-02/E-03 on either spec, E-18, E-70); the same "reports written?" column applies.

explain (v1.17) uses 0 (the C-18 trace written; there is no 1: nothing it renders is pass/fail), 2 (usage, E-60, E-64) and 3 (E-01/E-02/E-03, E-05, E-70); it writes no file, so E-18 and the "reports written?" column do not apply to it.

verify (v1.25) uses 0 (every file the manifest names is present with its recorded hash and length), 1 (one or more is missing or differs; C-26), 2 (usage, E-82, and an invalid --root) and 3 (E-83); it writes no file, so the "reports written?" column does not apply to it. --manifest on check adds one usage fault (E-85) and no new exit code.

Since v1.18 this set is reachable from the CLI itself: every --help screen carries the exit-code epilog C-19 pins (D-40), naming the four codes and the summary line.

Precedence between 2 and 3 (v1.20, K-18). Flag, value, --root-containment and file-readability validation (the usage-fault class, 2) is completed in full before SPEC.md, the results file, the proof manifest, the citation map, the risk file (v1.25), the output directory, or (for impact) --against is parsed at all — so an invocation carrying both a usage fault and what would otherwise be an input-contract violation (3) exits 2; a run never reaches the contract check that would have failed. No other precedence exists between the two classes, since no other invocation can trigger both.

---

6. Invariants (must hold in every valid implementation)

ID	Invariant
I-001	**Read-only inputs.** No file under `--root` other than the two report files of the subcommand that ran, under `--out` (`check`: `speccheck.json`, `SPEC_CONFORMANCE_REPORT.md`, and `speccheck.manifest.json` when `--manifest` was given, v1.25; `impact`: `impact.json`, `IMPACT_REPORT.md`; v1.13) — and the `--out` directory itself, if it did not exist — is created, modified, or deleted by a run, and no temporary of §3.1 survives a run; no file outside `--root` is written at all. Sole exception: `--self-check` creates, uses, and removes one fresh temporary directory (F-011); it writes nowhere else. `verify` (v1.25) creates, modifies and deletes nothing at all.
I-002	**Determinism.** For `--judge none` and `--judge mock`, and for every `impact` run (v1.13), identical inputs (bytes of every file read, argv, env) produce byte-identical report files and summary line, on any OS. A `SPEC.md` that differs from another only in `\r\n` versus `\n` line endings counts as identical input (C-01 Lines rule; F-301).
I-003	**Exactly-once reporting.** Every declared ID (retired included) appears exactly once in `ids` and exactly once in the Markdown per-ID table; no undeclared ID ever appears there. A decision (C-01 (c)) is not an ID: it appears exactly once in `decisions` and never in `ids` (D-25).
I-004	**Downgrade-only judge.** For every ID, `status_with_judge` $\in$ `{status_without_judge}` $\cup$ (`{WEAKLY_PASSING}` iff `status_without_judge == PASSING`). Disabling the judge never changes any status except `WEAKLY_PASSING` $\rightarrow$ `PASSING`.
I-005	**Grounded verdicts, on both sides.** Every verdict in a report carries an `evidence` list (possibly empty), every `ASSERTS` carries ≥ 1 entry, and every evidence entry points to a line inside the judged test case's span in the judged file; and every `ASSERTS` or `EXECUTES_ONLY` carries a `clause` that is LOCATED in the judged ID's statement per K-15 (v1.9, R-34).
I-006	**Network boundary.** With `--judge none`/`mock`, no socket is opened for the lifetime of the process.
I-007	**Secret and payload hygiene.** The API key never appears in stdout, stderr, or either report. INFO never contains file contents, statements, prompts, or responses.
I-008	**Total metrics.** Every ratio is either a number in `[0, 1]` or `null`/`n/a`; no run raises on a zero denominator.
I-009	**Exit $\equiv$ report.** `exit_code` in `speccheck.json` equals the process exit status, and both are computable from the rest of the JSON plus `strict` by the §5.4 rule.
I-010	**One judge call per edge.** The judge is invoked at most once per (test case, ID) edge per run, and only for edges whose ID is `PASSING` after C-05 step 4, is not RECORDED (R-35), and whose test outcome is `passed`.
I-011	**Family-safe numbering.** ID normalization is injective within a family: `R-7`, `R-07`, `R-007` map to one ID; `R-07` and `C-07` never collide.
I-012	**One scan per physical file (D-23).** For the `--src` list, and separately for the `--tests` list, every physical file reached by the PATHS elements is scanned and produces citations at most once (C-03 PATHS deduplication): a file reached by two overlapping elements, or named directly and also covered by a directory element, is deduplicated by resolved path. Where a direct file and a directory element both cover a file, that file's recorded `--src`/`--tests` root — and therefore a directly-named `.swift` file's `MODULE` (D-18) — is the FIRST element in the effective-list order that covers it; the emitted citation order is by ascending resolved path, independent of the order in which paths were named. Two inputs that name the same physical file set produce byte-identical reports (I-002).

ID	Invariant
I-013	**The direct set is exact; depth is a prefix (v1.13).** In every `impact` report, an id is at depth 1 if and only if it is the `dst` of an `affects` edge whose `src` is a changed D id, or the `src` of a `depends_on` edge whose `dst` is a changed id, and it is not itself in the changed set; and for every $N \geq 1$ the `impact` list under `--depth N` is exactly the rows of the `--depth 0` list whose depth is $\leq N$, in the same order with the same `via`.
I-014	**`declared` is a pure function of the source and test trees (v1.14).** `Citation.declared` and every value derived from it (`JudgeRequest.declared`, `tests[].declared`, `declared_ratio`) depend only on the bytes of the files under `--src`/`--tests` and their citations; two runs with identical source and test trees produce identical `declared` values and `declared_ratio` regardless of `--judge` mode, `--results`, or network access — extending I-002's determinism guarantee to this fact.
I-015	**Jev stays advisory (v1.15).** For every edge, and under every combination of `--jev-pre-triage` and `--judge-budget`, the recorded `Verdict` (`verdict`, `clause`, `evidence`, `rationale`, `coerced`) is produced only by the configured `--judge` provider (C-06: `mock`, or the C-09 `llm` provider) or by an existing C-06 coercion rule. The C-17 triage provider's response is read only to compute K-16's order; it is never validated against C-06, never written to `speccheck.json` or `SPEC_CONFORMANCE_REPORT.md`, and never the source of a `Verdict`. Its request is not a judge call: I-010 is unchanged and applies to the judge alone, so a triaged run makes one C-17 request per judge-eligible edge and then one C-06 request per edge it issues.
I-016	**`explain` is additive and read-only (v1.17).** `explain` writes no report file, changes no status, verdict, citation, metric or note, and issues no judge call a `check` run over the same inputs would not issue (E-61). The facts it renders are the same `IdRecord`/edge/`Verdict` facts that run produces and the same `walk`/`reverify` facts `impact` produces (C-12, C-13), so a `check` and an `explain` over identical inputs agree on every status the trace shows, and no `--strict` gate moves.
I-017	**`--help` is inert (v1.18).** `speccheck --help`, `speccheck check --help`, `speccheck impact --help` and `speccheck explain --help` exit `0`, write only to stdout, and read no project file, no environment variable and no socket — naming the C-09/C-17 variables in the epilog is printing their names, not reading them. They therefore behave identically inside a project tree and in an empty directory, and the help text is a pure function of the installed version and the terminal width, which affects line wrapping only (C-19, T-96).
I-018	**Proof-input inertness (v1.19).** A `check` run without `--proof`/`--proof-results` writes `speccheck.json` and `SPEC_CONFORMANCE_REPORT.md` byte-identical to a v1.18 run on the same inputs: no `proof` key in the JSON (top-level or per-id), no proof column in the Markdown, no Lean file read, no exit-code difference (T-100).
I-019	**Namespace and map inertness (v1.23).** A `check` run given neither `--namespace` nor `--citation-map`, over inputs in which no scanned file and no declared statement or *Affects* cell contains a `<ns>:`-prefixed token, writes `speccheck.json` and `SPEC_CONFORMANCE_REPORT.md` byte-identical to a v1.22 run on the same inputs, prints the same summary line, and exits the same (T-109). The converse is deliberate: the same inputs *with* such a token change by exactly that token's citation or edge (E-67, E-69).
I-020	**Manifest determinism (v1.25).** I-002 extends to `speccheck.manifest.json`: for `--judge none` and `--judge mock`, identical inputs — the bytes of every file read, argv, and env, `SOURCE_DATE_EPOCH` included — produce a byte-identical manifest on any OS, and its `reports` hashes equal the SHA-256 of the two report files the same run left in `--out`. The manifest's `tool.version` makes it differ across tool versions; the reports do not (C-07).

ID	Invariant
I-021	**Provenance and hazard inertness (v1.25).** A `check` run given neither `--manifest` nor `--risk-file`, over a specification in which no declaration carries a CONTROLS marker (C-01, C-27), writes `speccheck.json` and `SPEC_CONFORMANCE_REPORT.md` byte-identical to a v1.24 run on the same inputs, writes no other file, prints the same summary line and exits the same; an `explain` trace of an id with no marker is byte-identical to v1.24's; `SOURCE_DATE_EPOCH` is read only with `--manifest`. With `--manifest` alone the two reports are still byte-identical to v1.24's, and only the third file is added.

---

7. Constraints (precise and measurable)

ID	Constraint
K-01	Usage errors exit `2`; input-contract violations exit `3`; conformance outcomes exit `0`/`1`; no other exit codes exist.
K-02	Files larger than 2 MiB (2,097,152 bytes) under `--src`/`--tests` are skipped with a Note (E-10); binary files (a `0x00` byte within the first 8192 bytes) are skipped silently (E-29); text files that are not valid UTF-8 are decoded with `errors="replace"` and noted (E-11).
K-03	Directories named `.git`, `.hg`, `.svn`, `node_modules`, `__pycache__`, `.venv`, `venv`, and any directory whose name starts with `.` are never descended into; symbolic links to files or directories are never followed (E-30).
K-04	ID numbers are 1–3 digits; a token with 4+ digits is not an ID.
K-05	LLM judge timeout is `SPECCHECK_JUDGE_TIMEOUT` seconds (default 30) per edge, wall-clock, measured from the moment the request is issued to the moment the full response body has been received, and covering every phase in between — DNS resolution, TCP connect, TLS handshake, request send, response receive (Q-010).
K-06	Exactly one HTTP request per judged edge; no retries, no batching. Requests MAY be issued concurrently, at most `--judge-concurrency` at a time (default `4`, integer `1..32`); output order is fixed by C-07 and MUST NOT depend on completion order (F-110).
K-12	`--judge-budget` has two forms (v1.15). `SECONDS` (default `0` = unlimited; integer `0..86400`): bounds the wall-clock spent in the judge stage, measured from the first request issued. The deadline is `start + budget`. A request is *started* when it is issued; no request is issued at or after the deadline; requests in flight at the deadline are allowed to complete (each still bounded by K-05) and their verdicts count. `N%` (an integer `0..100` followed by `%`; accepted only with `--jev-pre-triage` under `--judge llm`, else E-58): exactly $\lceil N/100 \times E \rceil$ of the E judge-eligible edges (I-010) are issued, in K-16's ascending-confidence order, and no other edge is issued, regardless of wall-clock time elapsed. Under both forms every edge not issued receives `UNKNOWN` with rationale `judge: budget` and `coerced: true`, and one Note records how many (F-110, Q-010). `SECONDS 0` is unlimited; `N%`'s `0%` issues none (a pure triage dry run) and its `100%` issues every eligible edge — the same report content as `SECONDS 0`, by an explicit count instead of a deadline that never arrives.

ID	Constraint
K-13	The progress indicator (C-11) is drawn once when the judge stage starts (with $d = 0$, `0:00 elapsed`); every determined verdict is reflected by a draw within 100 ms of its determination (a burst of verdicts MAY be coalesced into one draw); it is redrawn at least once per second of wall-clock while any request is in flight so that `elapsed` and `left` keep moving; and it is never drawn more than 10 times per second. Each draw and the erase is one atomic write, serialized across threads. The bar has exactly 20 cells. From the first draw to the erase nothing else is written to stderr (F-201). The indicator is erased exactly once, when the judge stage ends, after the final state has been drawn and before the `report` stage begins — before any INFO stage line, Note, error message, or the summary line is written.
K-14	A statement (C-01: title, newline, section body) is at most 16,384 bytes of UTF-8. A longer statement is truncated to the longest prefix of whole lines whose UTF-8 length, joined by `\n`, is $\leq$ 16,384 — the title line is always kept — followed by `\n` and the marker line `… (statement truncated by speccheck at K-14)` (U+2026, one space, then ASCII); the marker does not count toward the cap. The truncated text is what `SpecId.text`, the judge, and the JSON carry; `title` is unaffected; one Note `statement truncated at K-14: <ID>` is recorded per truncated ID (E-46). Table declarations never approach the cap. Reference points: the largest contract of the MonteCarloPi spec is 2.7 kB and C-03, at 8.4 kB when this row was written, grew with the Swift and JS/TS adapters and crossed the cap in v1.23.1 (its statement was truncated, with the K-14 Note, from `b6949a1` on); v1.24 moved the JS/TS adapter into C-24, and a contract nearing the cap should split the same way rather than cross it; the cap is a bound on the judge payload (about 4 k tokens per edge at most), not a working limit (D-20).
K-15	A `clause` is LOCATED when, after collapsing every run of whitespace in both strings to one space, trimming the clause (F-406), and cutting it to its first 280 characters, it is a case-sensitive substring of the statement and is at least 12 characters long — or the statement itself is shorter than 12 characters and the clause equals it (an empty statement is matched by an empty clause). The cut, not truncated-with-ellipsis, form is what is recorded (a prefix of a located excerpt is still located).
K-07	`rationale` $\leq$ 280 characters after truncation; single line (newlines replaced by spaces).
K-08	Kernel performance, measured on the reference machine named in `SPEC_BUILD_REPORT.md` (CPU model, RAM, OS, Python build, run in isolation): `check --judge mock` on the §9.8 golden fixture completes in $\leq$ 2 s wall-clock, and a generated 10,000-file tree with 100 declared IDs in $\leq$ 60 s. Both bounds are recorded, not CI-gated (F-017).
K-09	JSON output is UTF-8, `indent=2`, `ensure_ascii=False`, sorted per C-07 (not alphabetically), trailing newline; Markdown output is UTF-8 with `\n` line endings.
K-10	`--version` prints a PEP 440 version equal to the package metadata version.
K-11	`--max-unknown` defaults to `0.2`, accepts a decimal in `[0, 1]` (parsed as a Decimal from its literal text), and is compared to the quantized `unknown_rate` of C-07 (Decimal, 4 places, ROUND_HALF_EVEN; Q-009); equality passes; a `null` `unknown_rate` (zero judged edges) never exceeds it (F-105).

ID	Constraint
K-16	`--jev-pre-triage` (boolean, default off; ignored unless `--judge llm` — under `--judge none/mock` no C-17 request is made and no C-17 variable is read). Before any real-judge request is issued, the kernel MUST build one triage task per judge-eligible edge (I-010) in C-17's request shape — the C-06 request object for that edge, `{id, statement, related, declared, file, start, end, source}` (the `related` neighbourhood rides the triage `state` as it does the real-judge request, D-28b, R-38), rendered as C-17's `state` text — and send it to the C-17 endpoint at `--judge-concurrency`, with no retries (K-06). The edges are then handed to the otherwise unchanged real-judge issue loop (K-05, K-12, C-06) in ascending $p(e)$ (C-17), ties broken by ascending id in C-07's id order; a per-edge Jev failure is ordered first and counted in one Note (E-59) and never aborts the run. Jev's answer is discarded once the order is computed (I-015). Under `--judge-budget 0` the pass changes no report content — only the issue order, visible in the C-11 indicator — and one Note `jev-pre-triage had no effect: --judge-budget is unlimited` records that (D-30).
K-17	**Proof staleness (v1.19; the file match refined v1.19.1).** A manifest theorem (C-21) whose name matches a scanned declaration (C-20) in a *different* file or line is `stale`, surfaced like a stale citation; a scanned declaration absent from the manifest is `unknown`, never `failed`; a `failed` state carries the build error line from `build.errors`. The file comparison is exact-match first, then a path-suffix match (either side a suffix of the other): a citation's `file` is posix-relative to `--root` (R-20), while a manifest a foreign producer wrote commonly records `file` relative to its own `--proof` scan root instead — `proof/HelloProof/Hello/Theorems.lean` (citation) and `HelloProof/Hello/Theorems.lean` (manifest) name the same file and MUST NOT join as `stale` on that account alone. Stale/unknown/failed proof states appear in `notes` and the `proof_failed`/`proof_unknown` metrics and, per D-45, affect `--strict`.
K-18	**Exit-code precedence (v1.20).** When one invocation carries both a usage fault (§5.4 `2`: an undefined flag, an out-of-range or malformed value, a missing required flag, a path outside `--root`) and an input-contract violation (§5.4 `3`: E-01, E-02, E-03, E-05, E-18, and from v1.23 the shape faults of E-70), the usage fault is reported: flag, value and `--root`-containment validation — including that every file a flag names (`--spec`, `--results`, `--proof-results`, `--against`, `--citation-map`) exists and can be read as bytes, the "unreadable → `2`" halves of §5.1, E-65 and E-70 (v1.23.1, F-709: a readability check is a read, so v1.20's "before … is read at all" could not be met) — are completed before any of those files is *parsed* or the output directory is touched, so a run never reaches the contract check (E-01..E-03, E-05, E-65's and E-70's shape halves, E-18) that would have failed. No other precedence exists — the two are never both live.
K-19	**Citation-map size (v1.23).** A citation map (C-23) is at most 2 MiB, the K-02 file cap. Over it, E-70 (exit `3`), before any entry is resolved. (No entry-count limit: the smallest well-formed entry is about 40 bytes, so 2 MiB already bounds the count near 52,000.)
K-20	**Risk-file and hazard-id size (v1.25).** A risk file (C-28) is at most 2 MiB, the K-02 cap; over it, E-84 (exit `3`) before any entry is read. A HAZID (C-27) is at most 64 characters, so a longer one is malformed: in a marker, E-80; in a risk file, E-84. A manifest (C-25) read by `verify` has no size cap of its own beyond K-02's: over 2 MiB is E-83.

8. Edge cases and failure semantics

ID	Case	Semantics
E-01	Spec declares zero IDs (or all declared IDs are retired)	Exit 3, message `spec declares no in-scope IDs`; no reports written.
E-02	Same ID declared twice (both non-retired, or both retired); since v1.13 also a decision D-nn declared by two rows of the decision table (C-01 (c))	Exit 3, message names the ID and both lines.
E-03	ID declared retired and also declared non-retired (any order)	Exit 3, message names the ID and both lines (a spec must retire *or* keep an ID, not both).
E-04	ID declared inside a fenced code block in `SPEC.md`	Not a declaration; ignored silently (spec templates contain example IDs).
E-05	`--results` file is not well-formed XML, or root is neither `testsuites` nor `testsuite`, or a `<testcase>` lacks `name`	Exit 3, message `results: <reason>`.
E-06	Duplicate `(classname, name)` in results	Single outcome = worst of them (C-04); a Note records the count.
E-07	Result `<testcase>` joins no attributed test case	Listed under `unattributed_results`; not counted for any ID; exit unaffected.
E-08	Test case cites an ID but has no result (results absent, or case not run)	Case listed under that ID's `unrun`; ignored in C-05 step 4; if *no* citing case has a result → `UNVERIFIED`.
E-09	Any of `--spec`/`--src`/`--tests`/`--results`/`--proof` given/differs from its default, `--against` (v1.13), `--citation-map` (v1.23) or `--risk-file` (v1.25) — including any element of a `--src`/`--tests`/`--proof` PATHS list — resolves outside `--root` (after symlink resolution)	Exit 2, message `path outside --root: <path>`; when the path is given in resolved form (a symlink in it) the message ends `(resolves to <resolved path>)` so the cause is visible (v1.21). A PATHS element that is itself a symlink and resolves inside `--root` is used in resolved form and descended (a directory) or scanned (a file); it is not an E-30 skip (Q-007, D-23).
E-10	File > 2 MiB under a scan root	Skipped; Note `skipped 1 file over 2 MiB: <path>`.
E-11	A scanned file — or `SPEC.md` itself — is not valid UTF-8	Decoded with replacement; Note; citations on replaced bytes are impossible (token requires ASCII) so no false citations arise.
E-12	`.py` test file fails to parse	Whole file is one file-level case; Note `parse fallback: <path>`.
E-13	Test citation lands in a file-level case (module docstring, helper, fixture)	Attributed to the file-level case; that case has no JUnit outcome by construction (name `""`), so it can only ever contribute `UNVERIFIED`; the JSON carries `"name": ""` and the Markdown renders the label `(file)` (Q-011).
E-14	Judge provider unreachable, times out, or returns HTTP ≥ 400	Verdict `UNKNOWN`, `coerced: true`, rationale `judge: unavailable` / `judge: timeout` / `judge: http <code>`; `judge_available` set `false` if no call succeeded (every call failed, or — E-35 — none was issued; C-07 owns the rule); exit unaffected unless `--strict` (E-32).

ID	Case	Semantics
E-15	Judge returns non-JSON or JSON lacking `verdict`	`UNKNOWN`, `coerced: true`, rationale `judge: malformed response`.
E-16	Judge returns `ASSERTS` with no evidence, or evidence outside the span / in another file	`UNKNOWN`, `coerced: true`, rationale `judge: ungrounded`. When the same reply also fails K-15, E-48 wins (C-06 rule order, F-402). The raw answer is available only at DEBUG.
E-17	Judge returns a valid verdict for an ID whose test span contains no code at all (empty function)	Verdict recorded as returned; the kernel does not second-guess content, only grounding.
E-18	`--out` cannot be created, or a temporary or final report file cannot be written or renamed	Exit `3`; message `out: <reason>`; on exit neither final report from this run exists and no `.tmp` from this run remains. Because renames happen JSON-then-Markdown (§3.1), the only stale combination an operator can find is: a previous run's `SPEC_CONFORMANCE_REPORT.md` with no `speccheck.json` (F-106).
E-19	Both `--src` and `--tests` empty/absent	Run proceeds; every in-scope ID is `UNCITED`; exit `1` (or `0` only if `in_scope == 0`, which is E-01).
E-20	Dangling citation of a token that *looks* retired in code (`~~R-99~~` in a comment)	Strikethrough is only meaningful in `SPEC.md`; in code it is a plain citation of `R-99` → dangling.
E-21	`--judge llm` with a required env var missing	Exit `2`, message names the variable, key value never echoed.
E-22	Test file cites the same ID on several lines of one case	One edge; `lines` lists every line.
E-23	`--spec`, `--results`, or a report path lies under a `--src`/`--tests` root (e.g. `--src .`, or `--out` inside `src/`)	Those files are not scanned and produce no citations; no Note; a second run over the same inputs is byte-identical to the first (F-002).
E-24	JUnit `name` carries a parametrization suffix (`test_x[3-True]`), or several `<testcase>` join one case	Suffix stripped for the join; each raw result kept in `results`; the case's `outcome` is the worst of them (F-005).
E-25	A T id is cited in source files but by no test case	`UNCITED` (source citations are evidence only for family T); listed under `src` in the report (F-001).
E-26	An ID has judge verdicts of both `ASSERTS` and `EXECUTES_ONLY` on passed edges	Status `PASSING`; passes `--strict`; the `EXECUTES_ONLY` edge remains visible in §8 of the report (F-003).
E-27	A result's classname suffix-matches two test cases equally (`tests/a/test_core.py` and `tests/b/test_core.py`, result classname `test_core`)	Unattributed; one Note names the result and all candidates; neither case receives the outcome (F-006).

ID	Case	Semantics
E-28	A `test*` function (underscore or not) is a method of a class that is neither `Test*`-named nor a `*TestCase` subclass, or of a nested class	Not a test case; its citations are file-level; one Note per file lists the undelimited names (F-007, F-102).
E-29	A file under a scan root contains a `0x00` byte in its first 8192 bytes	Skipped silently: no citations, no test case, no Note (F-008).
E-30	A symbolic link (file or directory) is encountered *during descent* under a `--src`/`--tests` directory element	Not followed; one Note per run: `skipped N symlink(s)` (F-008). A symlink that is itself a PATHS element (on the command line) is E-09's case, not this one (Q-007).
E-31	A bold ID token appears in a table cell other than the first, or in a heading after other tokens	Not a declaration (F-009). It is also not a citation: `SPEC.md` is never scanned for citations.
E-32	`--strict --judge llm` and either `judge_available == false` or `unknown_rate > max_unknown`	Exit `1` even if every status is `PASSING`; `strict_judge_failure` set to `"unavailable"` when `judge_available == false` — whether the judge failed (E-14) or the eligible edges were never issued (E-35) — else `"unknown_rate"` when `unknown_rate > max_unknown`; when both hold, `"unavailable"` (F-012, Q-004; v1.20, amended). Summary line carries the matching suffix. Without `--strict`, or with `--judge none|mock`, no effect.
E-33	A line contains `speccheck:ignore`, or a file's first three lines contain `speccheck:ignore-file`	The line yields no citations / the file is not scanned; ignored files are counted in one Note (F-013).
E-34	A `.speccheck.json.*.tmp` or `.SPEC_CONFORMANCE_REPORT.md.*.tmp` from a killed run exists under `--out`, which lies under a scan root	Never scanned (C-03); deleted at §3.1 step 1; the run's reports are byte-identical to a run without the leftover (F-103).
E-35	`--judge llm` and the `--judge-budget` deadline passes with edges not yet started, or the `N%` form's issued count is reached	Each such edge is `UNKNOWN`, `coerced: true`, rationale `judge: budget`; in-flight requests complete and count; one Note `judge budget exhausted: N edge(s) unjudged`; `unknown_rate` counts them; R-28 applies as usual (F-110, Q-010). Under the `N%` form the unissued edges are the ones K-12's order left last, and they are determined before any request is issued — no wall-clock condition is involved.
E-36	`--judge llm` with zero eligible edges	No request is sent — to the judge, and (with `--jev-pre-triage`) to C-17 either; `judge_available` is `true`; `unknown_rate` is `null` and never trips R-28 (F-105).

ID	Case	Semantics
E-37	A `tests[]` entry whose case was not judged: judge disabled, or the ID not `PASSING` after C-05 step 4, or the ID RECORDED (R-35), or the outcome not `passed` — this row owns the condition; C-05, C-07 and T-34 cite it (F-403)	JSON `verdict` is `null` (key present); Markdown renders an em dash in the verdict position; the entry never appears in report §8 (Q-001).
E-38	Two or more Notes are produced in one run	Emitted in ascending Unicode code-point order of their full text, in both reports (Q-002).
E-39	`--judge llm` with stderr not a TTY (CI log, redirected file, pipe) under `--progress auto`; or `--verbose DEBUG` under any `--progress` value; or `--progress never`; or `--judge none|mock` under any `--progress` value	No progress bytes (neither draw nor erase) are written to stderr; the run is otherwise identical. `--progress always` overrides only the TTY test, never the DEBUG or judge-mode suppression (R-30).
E-40	The judge stage is cut short while the indicator is displayed: a provider raises out of the stage (should not happen — E-14..E-16 coerce), an uncaught exception, or an interrupt (E-41)	The erase sequence is written before the exit-3 message (§5.4, E-41); no partially drawn line remains on stderr. The erase MUST happen in a `finally`-equivalent path so it cannot be skipped (R-30, K-13).
E-41	`SIGINT` / `KeyboardInterrupt` at any stage	Exit 3 with the one-line message `interrupted`; the indicator, if displayed, is erased first (E-40); every §3.1 temporary of this run is removed and no report file from this run remains (a report file already renamed is removed, per §3.1's failure rule); judge requests in flight are **abandoned**, not awaited — the process MUST exit within 1 s of the interrupt regardless of K-05 — and edges not yet started are never started (v1.5). No other exit code is produced (K-01); a traceback is printed only under `--verbose DEBUG` (F-204, D-16).
E-42	A `.swift` test file whose brace depth goes negative on some line or is non-zero at end of file (counted per C-03, outside comments and string literals)	Whole file is one file-level case; Note `parse fallback: <path>` (R-31; the E-12 rule).
E-43	A `.swift` function named `test*` that is neither `@Test`-attributed nor a direct `test*` method of an `XCTestCase` class (a method of another type, a member of a nested class, a free function)	Not a test case; its citations are file-level; one Note per file lists the undelimited names as `<Chain.name>` (R-31; the E-28 rule).

ID	Case	Semantics
E-44	A declaring table row whose first cell begins with a bold ID form and continues with whitespace-separated decoration (`**K-07** **[port]**`)	Declares the ID; the decoration is dropped; a bold ID inside the decoration is not a declaration; a first cell where the form is followed by a non-whitespace character (`**K-07**x`) declares nothing (R-32).
E-45	Two Swift test cases with the same identifier and classname (overloads by parameter label)	A result with that `join_name` ties between them: unattributed, one Note names the result and both candidates (C-04; the E-27 rule).
E-46	A heading-declared ID whose statement (title plus section body) exceeds K-14; or whose section body is empty — the heading is followed only by blank lines before the next heading of the same or a higher level, or before end of file	Over the cap: truncated per K-14, marker line appended, one Note `statement truncated at K-14: <ID>`; `title` is unaffected and the Markdown table shows it. Empty body: the statement is the title alone, exactly as through v1.6; no Note. Empty title with a body (`### C-04` followed by text): the statement is the body alone (F-305). (R-33)
E-47	A heading-declared ID whose section body contains a deeper heading that itself declares an ID (`#### E-09 …` under `### C-01 …`, retired or not), or a table with declaring rows	Those inner declarations are parsed exactly as before, each with its own title, statement, and body; the outer ID's statement includes their text verbatim as context (the judge sees the sub-rows). A declaring heading's body ends only at a heading of level ≤ its own, so a deeper declaring heading never ends it (R-33, E-31).
E-48	Judge returns `ASSERTS` or `EXECUTES_ONLY` with a `clause` that is missing, empty, paraphrased, or shorter than 12 characters against a longer statement	`UNKNOWN`, `coerced: true`, rationale `judge: unlocated clause`; counted in `unknown_rate`, so a model that cannot quote the statement surfaces through R-28 rather than through silent mis-verdicts (R-34, K-15). The raw answer is available only at DEBUG.
E-49	Judge returns `UNRELATED` or `UNKNOWN` with a non-empty `clause`; or a `clause` that is present but not a JSON string (`null`, a number, an object); or a verdict that any C-06 rule coerces to `UNKNOWN`	Recorded with `clause ""`; a returned `UNRELATED`/`UNKNOWN` is not coerced and gets no Note; a non-string `clause` is treated as absent before the K-15 check, so on an `ASSERTS`/`EXECUTES_ONLY` it is E-48 (R-34, F-402).
E-50	The `*(recorded)*` marker on a declaration whose family is not T (`| **R-07** *(recorded)* |`)	The id is declared normally and is not RECORDED (`recorded: false`); one Note `recorded marker ignored on <ID>: not a T id` (R-35).
E-51	A RECORDED T id whose citing test case failed, was skipped, has no result, or does not exist	Status by C-05 steps 1–4 exactly as for any T id — `FAILING`, `SKIPPED`, `UNVERIFIED`, `UNCITED`; the marker exempts an id from the judge (step 5) and from nothing else, so a recorded test still has to be present and green (R-35, R-15).

ID	Case	Semantics
E-52	A `--src`/`--tests` PATHS element that resolves *inside* `--root` but exists there as neither a directory nor a regular file — a non-existent path, or a broken symlink to nowhere (E-09's containment check already passed)	Exit 2, message `--<flag>: no such file or directory: <segment>` (D-23). This replaces v1.11's "non-directory → usage error": a directory is descended, a regular file is scanned, and anything else is an error. No reports are written.
E-53	`impact --changed` names an id that is not declared in `--spec` (an undeclared conformance id, an unknown decision, or a token that is neither — `X-01`, `R-`, `1234`)	Exit 2, message `--changed: undeclared id: <element>` naming the first offending element in list order; no reports written (C-13).
E-54	`impact` given both `--changed` and `--against`, or neither; or given `--results`, any judge flag, or `--proof`/`--proof-results` (v1.19)	Exit 2, message `impact: exactly one of --changed, --against is required` (or `impact: --results is not accepted` / `impact: --judge is not accepted` / `impact: --proof is not accepted`); no reports written. An `--against` file that is unreadable or outside `--root` is the ordinary usage error (E-09); one that fails C-01 is exit `3` with `--against:` prefixed to the E-01/E-02/E-03 message.
E-55	A statement or an *Affects* cell names a retired id	The edge is recorded with `retired: true` (C-12); `impact` walks it like any other, and every report row for a retired id — changed, impact, re-verify — strikes through the ID cell; a retired id in `--changed` is accepted. No Note: retirement is declared, not accidental.
E-56	A citation of family R/C/I/K/E has no test-case span to check for declaration — a `"src"`-kind citation, or a `"test"`-kind citation attributed to a file-level case (no enclosing named test case delimited it)	Always `declared: false`; no Note (this is definitional, not an anomaly — there is no per-test declaration of intent to check a file or an undelimited case against).
E-57	A `related` edge — a C-12 `depends_on`, both directions (R-38, D-28) — names a retired id, and a `related` field is built for a judged edge	The retired neighbour is included in the edge's `related` with its `title` and `(retired)` appended: a test asserting the retired obligation is exactly the off-topic `ASSERTS` this change exists to catch. The retired target on its `affects`/`depends_on` edge is marked `retired: true` (E-55), and the `affects`/`depends_on` edge is still recorded; an id with no neighbour at all sends `related []`. Not a Note and not a status: it is how the judge's `related` neighbourhood (C-06's `related` field, v1.16; C-10) handles a retired target.

ID	Case	Semantics
E-58	`--judge-budget N%` given without `--jev-pre-triage`, or with `--jev-pre-triage` under `--judge none`/`--judge mock` (where K-16 ignores the flag, so there is no ordering for K-12 to issue in)	Exit 2, message naming both flags (`--judge-budget N% requires --jev-pre-triage with --judge llm`), before any request of either kind is issued and with no reports written; the message never contains a C-09 or C-17 key value (I-007). The `SECONDS` form is unaffected: it stays accepted-and-ignored unless `--judge llm`, exactly as before v1.15.
E-59	A C-17 triage request fails during `--jev-pre-triage`: non-200 status, transport error, timeout (`SPECCHECK_JEV_TIMEOUT`), non-JSON body, missing `answers.verdict`, a `choice` that is not one of C-06's four verdict tokens, or `probabilities` that are missing, empty, or non-numeric	That edge is ordered first (K-16) and one Note records how many edges this happened to; the run is not aborted, no edge is skipped because of it, and no edge's final `Verdict` is affected beyond its position in the issue order (I-015). A triage failure never sets `judge_available` false — that flag is the C-06 judge's fact (E-14) — and never enters `unknown_rate`.
E-60	`explain <ID>` names an id the spec does not declare	Exit 2, message `explain: undeclared id: <ID>` (the R-07 dangling wording), and no trace is written. An id that *is* declared but has no citation and no edge is not an error: its trace carries the C-05 status with its reason and an empty `sources:` and `tests:` block, and exits `0` (E-13/E-19 style).
E-61	`--judge` on `explain`	Carries `check`'s contract exactly: the same provider, the same I-010 eligibility, the same C-06 validation and coercion, and the same `--judge-concurrency` / `--judge-budget` / `--jev-pre-triage` semantics. A coerced or failed verdict is rendered verbatim (`UNKNOWN (coerced)` with its rationale, E-14/E-15); under `--judge none`, and for any edge no provider judged, the verdict reads `not judged`; `explain` neither issues nor omits a judge call `check` would not make.
E-62	An unreadable or non-Lean file in a `--proof` path (binary, or no declaration head parseable)	Recorded as a `notes` entry naming the file; MUST NOT be a usage error and MUST NOT change the exit code of a run whose ids are all otherwise decided (v1.19).
E-63	A proof state (`checked`/`failed`/`unknown`/`stale`) is computed for an id	It never changes that id's status: C-05 reads citations, tests, results and the judge only — a `checked` proof never promotes an `UNVERIFIED` id, and a `failed`/`unknown`/`stale` proof never demotes a `PASSING` one (R-06). `impact` MUST ignore proof citations: a theorem's file is not a scanned source file, so proof carries no blast radius (v1.19).
E-64	`explain` invoked without `--spec` (v1.20)	The flag is required on this subparser (D-47), and its synopsis carries no brackets (E-54's pattern). Exit `2`, message `explain: --spec is required`, and no trace is written; no `explain` stage runs.

ID	Case	Semantics
E-65	`--proof-results FILE` given but unreadable, not well-formed JSON, or lacking the `build`/`theorems` keys C-21 requires	Unreadable → exit 2, message `--proof-results: <reason>`; malformed JSON or missing required keys → exit 3, message `proof-results: <reason>` (the `--results`/E-05 pattern; v1.19). No report is written.
E-66	A table-declared row whose statement cell contains an odd number of unescaped `$` outside backtick spans and which has further cells — an unescaped `|` inside an inline-math span split the cell, and the rest of the formula was read as another column (v1.21); **or** (v1.24) a table-declared row inside a TABLE (C-01 (c)) with more cells than that TABLE's header row — an unescaped `|` anywhere split it	The statement is the cell as split (unchanged); one Note `statement may be cut at an unescaped '|': <ID>` per such row, emitted once when both conditions hold. The fix is `|` or a backtick span. (R-36)
E-67	A scanned `--src`/`--tests` file contains a foreign reference (C-22) — a TOKEN preceded by `<ns>:`, `<ns>` not the `--namespace` value	No citation; the token is listed in `foreign`; one Note `<N> foreign reference(s) ignored: <ns>:<id> <file>:<line>, …` (N the count of listed tokens; the first three in `foreign` order, then `and <N-3> more` when N > 3, so a reader sees what was dropped without opening the JSON). It is neither dangling nor stale. (R-101)
E-68	`--namespace NS` is given and a scanned file or a declared statement contains `NS:ID`	Read as the bare `ID`: a local citation / edge target, not listed in `foreign`. A bare `ID` is unaffected. A `--namespace` value outside `[A-Za-z][A-Za-z0-9_-]{0,31}` or shaped like an ID → exit 2, message `--namespace: invalid namespace: <value>`; a second `--namespace` → exit 2, message `--namespace: given more than once`; both before any file is read (K-18). (R-102)
E-69	A declared statement or an *Affects* cell contains a foreign reference	No C-12 edge and no Note for it; a bare token to the same id in the same statement still yields its edge. Foreign references in `SPEC.md` are never listed in `foreign`. (R-101, C-12)

ID	Case	Semantics
E-70	`--citation-map FILE` is unreadable, outside `--root`, not UTF-8 JSON, not an object, has a `schema_version` other than `"1.0"`, has no `citations` array, has an entry that is not an object with exactly the string keys `id`, `file`, `test` (a missing, extra or non-string key), has an `id` that is not a bare TOKEN form (a `ns:`-prefixed id included), or is over K-19's 2 MiB	Unreadable or outside `--root` → exit 2, message `--citation-map: <reason>` (E-09 for containment), decided with the other flags (K-18); every other fault → exit 3, message `citation-map: <reason>`, decided in the apply-map stage (§3.1). No report is written. (R-103, C-23, K-19)
E-71	A map entry's `file` and `test` match no scanned `TestCase` (the file was not scanned, or its test was renamed)	Adds nothing; one Note per entry `citation-map entry unresolved: <id> <file> <test>`; counted in `citation_map.unresolved`; the exit code is unchanged by it (D-55). (R-103)
E-72	A map entry resolves (C-23) and no literal citation of the same (id, testcase) exists	One Citation with `kind test`, `declared` true (D-57), `mapped` true, `line` the case's `start`; its `tests[]` entry has `mapped: true` and `lines: []`; the summary line gains `; citations: <M> mapped`. A mapped citation is judge-eligible and counts toward C-05 exactly as a literal one. (R-103, C-05)
E-73	A map entry resolves to a (id, testcase) already cited literally, or two entries name the same (id, testcase)	Adds no second Citation; the literal one stays `mapped: false`; the entry counts as resolved. A repeated entry counts as resolved each time. (R-103)
E-74	A JavaScript/TypeScript test file whose parentheses do not balance, or with an unterminated block comment, string or template literal	The whole file is one file-level case; one Note `parse fallback: <path>` (E-12's rule). (R-104)
E-75	An `it`/`test` call whose first argument is not a literal title (a variable, an interpolated template), or a `describe` with such a title	Not a case; its citations are file-level; one Note per file `undelimited tests in <path>: line N, …` (E-43's rule). A result for such a test is unattributed (E-07). (R-104)
E-76	A JavaScript/TypeScript case whose title carries `each` placeholders, and several results that fill them	All results join the one case as variants; its outcome is the worst of them (E-06). If the placeholders make two cases' patterns match the same result, the result ties and is unattributed (E-27). (R-104, C-04)

ID	Case	Semantics
E-77	A declared, non-retired id is the `src` or `dst` of no C-12 edge: no statement, row cell or *Affects* cell names it, and its own statement and row cells name no declared id (v1.24)	No edge, status, metric or exit-code change. One Note per run, when at least one id is isolated: `<N> declared id(s) touch no edge: <ID>, <ID>, <ID>` followed by `and <N-3> more` when N > 3, the ids in C-07 order. A retired id is never counted. `impact` does not emit it. (R-36, C-12, D-60)
E-78	A CONTROLS marker (C-27) on the declaration of a T id	The id is declared normally with `controls` empty, so no `controls` key and no hazard from this marker. One Note per such id: `controls marker ignored on <ID>: not an R/C/I/K/E id`. (R-108)
E-79	With `--risk-file`, a CONTROLS marker names a hazard the file does not list	The hazard is still in `hazards` (`title` null) with its C-27 status. One Note per run: `<N> hazard(s) not in the risk file: <HAZ>, <HAZ>, <HAZ>` followed by `and <N-3> more` when N > 3, hazards in code-point order. No exit-code effect, with or without `--strict`. Without `--risk-file` the Note is never emitted. (R-109)
E-80	The text in the CONTROLS position (C-01) begins `*(controls:` but does not match C-27's grammar: an empty list, a HAZID over 64 characters or with a forbidden character, a missing `)*`	The id is declared normally with `controls` empty; the text stays decoration (a row) or title text (a heading). One Note per such id: `malformed controls marker on <ID>`. (R-108, K-20)
E-81	A hazard in C-27's set H has no non-retired control: it is listed in `--risk-file` and no marker names it, or every id whose marker names it is retired	Status `UNCONTROLLED`. One Note per run: `<N> uncontrolled hazard(s): <HAZ>, …` shaped as E-79's. With `--strict`: exit `1` (R-15, D-64). Without `--strict`: the exit code is unchanged and, on exit `0`, the summary line's word is `UNPROVEN` (R-105). (R-109)
E-82	`verify` without `--manifest`, or with a `--manifest` path that does not exist, cannot be read, or resolves outside `--root`	Exit 2; message `verify: --manifest <reason>`; nothing on stdout. (R-107)
E-83	`verify` on a manifest that is over 2 MiB, not JSON, carries a `manifest_version` other than `"1.0"`, lacks a C-25 key, has an entry whose `sha256` is not 64 lower-case hex digits or whose `bytes` is not a non-negative integer, or names a `path` that is absolute or resolves outside `--root`	Exit 3; message `verify: malformed manifest: <reason>`; nothing on stdout; no input file is read (a manifest cannot direct a read outside the tree). (R-107, K-20)

ID	Case	Semantics
E-84	`--risk-file` is over 2 MiB, not JSON, lacks a `hazards` list, has an entry that is not an object, has an `id` missing or failing HAZID, a non-string `title`, or two entries with the same `id`	Exit 3; message `--risk-file: <reason>` (a duplicate names the id); no report is written. (R-109, C-28, K-20)
E-85	`check --manifest` with `SOURCE_DATE_EPOCH` set to anything but a non-negative decimal integer (empty included)	Exit 2; message `SOURCE_DATE_EPOCH: not a non-negative integer: <value>`; raised in the usage-fault class, so before any input is read (K-18). Without `--manifest` the variable is not read, whatever its value (I-021). (R-106, C-25)

---

9. Acceptance criteria, tests, and evals

Every test cites, in its docstring or a comment, its own T id and the R/C/I/K/E ids it proves, so that `speccheck` can check itself (§9.9; F-001) — and, since v1.14, that citation is what `declared` (C-14) checks mechanically: an id cited only elsewhere in a test's body is not this convention's declaration, whatever else it may be evidence of (R-39). Test inputs that contain the literal marker strings of C-01 (`speccheck:ignore`, `speccheck:ignore-file`) MUST live in data files under `tests/data/`, never inline in a test module, so self-application cannot ignore its own tests (F-109); likewise every test input containing a `<ns>:ID` foreign-reference form (C-22), so self-application and T-112 do not count the suite's own fixtures as foreign references (v1.23.1, F-713). Groups are ordered by pipeline stage.

9.1 Extraction (C-01, C-02)

ID	Test
T-01	Table-cell and heading declarations are both extracted with correct family, number, statement, and line; a `SPEC.md` containing invalid UTF-8 is decoded with replacement, its ASCII declarations are still found, and a Note is recorded. (R-01, E-11)
T-70	`| **K-07** **[port]** | s |` declares K-07 with statement `s`; `| **R-01** **R-02** | s |` declares R-01 only; `| **K-07**x | s |` and `| **K-07**, note | s |` declare nothing; the retired forms tolerate decoration the same way; heading declarations are unchanged. (R-32, C-01, E-44)
T-02	`R-7`, `R-07`, `R-007` in a spec are one ID reported as `R-07`; `I-5` is reported as `I-005`. (I-011, K-04)
T-03	A token with 4 digits (`R-1234`) and tokens adjacent to alphanumerics (`XR-07`, `R-07a`) are not IDs. (K-04, C-01)
T-04	Strikethrough declarations yield `retired=True`; a later plain declaration of the same ID exits 3. (R-02, E-03)
T-05	IDs inside fenced code blocks in `SPEC.md` are ignored, for both ``` and ~~~ fences, with and without an info string, and for an unclosed fence running to end of file; a backtick fence opened with an info string is closed by a bare backtick line, is not closed by a tilde line, and a candidate closing line carrying an info string does not close it. (E-04, C-01)

ID	Test
T-06	Duplicate declaration exits `3` and names both lines. (E-02)
T-55	A bold ID in a second cell, a heading with the ID after other tokens, a separator row, and a row with `|` and a backtick-quoted pipe in a cell are parsed per C-01: only first-cell/first-token forms declare; the statement is the second cell with escapes intact. (E-31, C-01)
T-72	A spec with `### C-01 Widget`, a fenced block containing `| **R-99** | x |` and a `# comment` line, a prose paragraph, a `#### note` sub-section holding a declaring `| **E-09** | … |` row, a `---`, then `### C-02 …`: C-01's `text` is `Widget`, a newline, and every line through the `---` with its indentation and trailing spaces intact (the `#### note` section included, C-02's heading excluded); C-01's `title` is `Widget`; R-99 is not declared (E-04) and E-09 is (E-47). A body that takes the statement over K-14 yields a `text` of whole lines $\leq$ 16,384 bytes plus the marker line and one Note naming the ID. A heading with an empty body, and a declaring heading on the last line of the file, each yield `text == title`; `### C-03` with no title and a body yields `text` equal to the body (F-305). The same spec saved with CRLF line endings yields a byte-identical `SpecIndex` and `speccheck.json`; a whitespace-only line after the body's last text line is dropped and one inside the body is kept (F-301). A `# four-space` line inside an indented block neither ends a body nor declares; a bare `###` ends a body; `### C-05 Title ###` has title `Title`; `###C-06 x` declares nothing (F-304). Property, for every ID of `fixtures/target/SPEC.md` and `fixtures/target-swift/SPEC.md`: `title` equals the second cell (table) or the heading remainder after the ID token (heading), whitespace-collapsed, as computed independently by the test; `text == title` for every table-declared ID and every heading-declared ID with an empty body; `text.startswith(title + "\n")` otherwise (F-302). (R-33, C-01, C-02, K-14, E-46, E-47)
T-07	A spec with no in-scope IDs exits `3` and writes no files. (E-01, I-001)
T-119	C-01 (a) with CommonMark backtick spans (v1.24): a row whose statement is `a fence (` ``` `), a table (first line contains `|`)` and continues past it has exactly two cells and its full text as the statement; a row ending in `` `x` `` then `|` carries no trailing `|` in its last cell; an unmatched backtick does not protect a later `|`, which splits; `` `a|b` `` stays one cell; the T-46/T-71 goldens are unchanged by this rule. (C-01, E-04, D-59)
T-120	In a two-column TABLE, `| **R-01** | a | b |` yields the E-66 Note and the statement `a`; the same row with the `|` inside `$..$` yields exactly one Note; a three-column TABLE's three-cell row yields none; a declared row outside any TABLE with an extra cell and no `$` yields none. (E-66, C-01)

9.2 Citation and attribution (C-03)

ID	Test
T-08	Source citations are recorded per (file, line) for tokens in code, comments, and strings alike. (R-03)

ID	Test
T-09	Python test citations inside `def test_*` and `class Test*` methods are attributed to the enclosing case with the correct `classname`, `start`, `end`. (R-04)
T-10	A citation in a module docstring or helper is attributed to the file-level case and reported with name `(file)`. (E-13)
T-11	An unparseable `.py` test file falls back to one file-level case and produces a Note. (E-12)
T-12	A test file in no adapter's language (`.go`, `.rb`) gets file-level attribution. (R-04, O-2)
T-13	Excluded directories (K-03), oversized / non-UTF-8 files (K-02), binary files, and symlinks are skipped or decoded exactly as specified: oversized and non-UTF-8 produce Notes, binary produces none, symlinks produce one Note per run and are never followed (a symlink cycle terminates). (E-10, E-11, E-29, E-30)
T-56	`test_*` and `testFoo` methods of a `*TestCase` subclass, `testFoo` methods of a `Test*` class, and `async def test_*` functions are delimited with correct spans; a module-level `testFoo` (no underscore) is not a test case; a `test*` method of an unrecognized class is not, its citations are file-level, and the file gets an `undelimited tests` Note. (E-28, C-03)
T-57	A line containing `speccheck:ignore` yields no citations but still counts toward its case's span; a file with `speccheck:ignore-file` on line 2 yields no citations and no test cases; a marker on line 4 has no file-level effect; markers inside `SPEC.md` change nothing; the Note counts ignored files. (R-27, E-33)
T-14	Several citations of one ID in one case yield one edge with all lines listed. (E-22)
T-65	Swift Testing attribution: a `.swift` file with a doc-commented `@Test func` at file scope, a `@Suite struct Outer` holding a `@Test("named") func`, a `@Test(arguments: [...])` attribute spread over two lines, a nested `@Suite struct Inner` with a `@Test func`, a plain helper, and a `func testHelper()` with no `@Test`, plus a `@testable import`: the cases are named by identifier, their classnames are `<Module>`, `<Module>.Outer`, `<Module>.Outer.Inner`, each span starts on the first doc-comment line above its attribute and ends on the function's closing brace; a citation on a doc-comment line, on an attribute continuation line and on a body line is attributed to that case; the helper's citation is file-level; the Note lists `Outer.testHelper` (E-43); `@testable` is not `@Test`. MODULE is the first path component under the `--tests` root, or the root's own name for a file directly under it. (R-31, C-03, E-43)
T-66	XCTest attribution: `final class LegacyTests: XCTestCase` with `func testAddition()` and `func helper()`, a class nested inside it with a `test*` method, and a `class Plain` (no `XCTestCase`) with `func testFoo()`: `testAddition` is a case named `testAddition` with classname `<Module>.LegacyTests` and a span from its doc comment to its closing brace; the nested and the `Plain` methods are undelimited with one Note naming `LegacyTests.Nested.testNested, Plain.testFoo`; `helper` produces no Note. (R-31, C-03, E-43)

ID	Test
T-67	A `.swift` file with an unclosed `{` is one file-level case with Note `parse fallback: <path>`; a file with a stray `}` likewise; a file whose only extra braces are inside `"{"`, `"""…}…"""` and `// }` delimits its cases correctly. (E-42, C-03)
T-85	DECLARED/INCIDENTAL classification (C-14): a citation on the case's own docstring line is DECLARED; a citation on a whole-line `#` comment inside the span is DECLARED; a citation inside a string literal, and one on a code line with a trailing comment (`x = 1  # R-02`), are INCIDENTAL; a citation attributed to the file-level fallback case is INCIDENTAL with no Note (E-56); a multi-line docstring whose citation is on its second line is DECLARED; the Swift adapter's doc-comment-line recognition (`///`, `/** … */`) classifies an equivalent Swift example the same way the Python path does. `declared` is present under `--judge none` and does not change with `--judge`. (R-39, C-03, C-14, E-56, I-014)

9.3 Results mapping (C-04)

ID	Test
T-15	`<testsuites>` and bare `<testsuite>` roots both parse; `failure`/`error`/`skipped` children map to the correct outcome. (R-05)
T-16	Classname join accepts both `tests.test_core` and `test_core` forms for `tests/test_core.py`, and rejects `unit_test_core` for `test_core`. (C-04)
T-58	With `tests/a/test_core.py` and `tests/b/test_core.py`, a result with classname `tests.a.test_core` joins `a` only; a result with classname `test_core` is unattributed with a Note naming both candidates. (E-27, C-04)
T-52	`test_x[3-True]` and `test_x[0-False]` both join `test_x`; `results` lists both with `param`; the case `outcome` is the worst of them; a name with no `[` is unchanged; everything from the first `[` is removed: `test_y[a][b]` → `test_y` and `test_x[list[int]]` → `test_x` with `param list[int]`; a result with empty `classname` joins the unique case with its `join_name` and is unattributed when two exist. (E-24, C-04, F-105)
T-68	A SwiftPM `junit-swift-testing.xml` with `freeFunction()` under `ProbeTests`, `named()`, `parameterized(x:)`, `twoArgs(a:b:)` and a `<skipped>` `disabledOne()` under `ProbeTests.Outer`, and `nested()` under `ProbeTests.Outer.Inner` joins each to its C-03 case with `param` null and the outcomes `passed` / `skipped`; an XCTest `<testcase classname="ProbeTests.LegacyTests" name="testAddition">` joins the XCTest case; results for overloads `f(a:)` and `f(b:)` are unattributed with one Note naming both candidates (E-45); `test_x[f(1)]` still strips from the first `[` and keeps `param f(1)`. (R-31, C-04, E-45)
T-17	Duplicate `(classname, name)` collapses to the worst outcome in the order error > failed > skipped > passed. (E-06)
T-18	Results for unknown cases are listed as unattributed and change no status. (E-07)

ID	Test
T-19	Malformed XML and a `<testcase>` without `name` each exit `3`. (E-05)

9.4 Status algorithm and metrics (C-05, C-07)

ID	Test
T-20	One fixture per deterministic status — `UNCITED`, `UNTESTED`, `UNVERIFIED`, `FAILING`, `SKIPPED`, `PASSING`, `RETIRED` — each asserting exactly that status; the `UNCITED` fixture has empty `--src` and `--tests` and exits `1`. (R-06, E-19)
T-21	An ID with one failed and three passed citing tests is `FAILING`; with all citing tests skipped is `SKIPPED`; with some skipped and some passed is `PASSING`. (C-05 step 4)
T-22	Citing cases absent from results are listed under `unrun` and do not affect the status. (E-08)
T-23	Dangling and stale citations are listed with file and line; strikethrough in code does not retire anything. (R-07, R-08, E-20)
T-24	`conformance`, per-family ratios, and counts match hand-computed values on the golden fixture; a family with zero in-scope IDs reports `null`/`n/a`, not `0.0`, and nothing raises. (R-09, I-008)
T-25	Retired IDs are excluded from every denominator and still appear in `ids` exactly once. (R-02, I-003)
T-53	A T id cited only in a source file is `UNCITED`; a T id cited by a passing test is `PASSING`; T ids are counted in `in_scope`, `conformance`, and `by_family.T`. (R-25, E-25)
T-77	`| **T-01** *(recorded)* | s |` declares T-01 with `recorded == True`, `title == "s"`; `### T-02 *(recorded)* Title` declares T-02 recorded with title `Title`; `| **T-03** *(Recorded)* |`, `| **T-04** (recorded) |` and `| **T-05** **[port]** *(recorded)* |` declare their ids not recorded; `| **R-01** *(recorded)* |` declares R-01 not recorded with the E-50 Note. With `--judge mock` and a call-counting stub under `--judge llm`: a recorded T id cited by a passing test with an assertion-free body is `PASSING` and its edge is never sent to the judge (`verdict` null, not counted in `unknown_rate`'s denominator or the C-11 total), while the same test cited by a non-recorded T id yields `WEAKLY_PASSING`; a recorded T id with a failing test is `FAILING`, with a skipped one `SKIPPED`, with no citing test `UNCITED` (E-51); with two recorded `PASSING` ids, one judged `PASSING` id and one `WEAKLY_PASSING` id, `judge_strength` is `1/2` (recorded ids outside the population) while `conformance` counts all three passing (F-405); the JSON carries `"recorded": true` after `"family"` and the Markdown ID cell reads `T-01 (recorded)` and a retired recorded id's `~~T-03~~ (recorded)`; `schema_version` is the C-07 value. (R-35, C-01, C-02, C-05, C-07, C-08, I-010, E-50, E-51)

9.5 Judge contract (C-06)

ID	Test
T-26	Mock judge returns `ASSERTS` with one evidence line per assertion token, and `EXECUTES_ONLY` with no evidence when none is present, including for an empty test body. (R-22, E-17)
T-69	The mock judge returns `ASSERTS` with the line as evidence for a Swift span containing `#expect(`, `#require(`, `XCTAssertEqual(`, `XCTFail(` or `Issue.record(` (one sub-test each), and `EXECUTES_ONLY` for a Swift body that only calls code (`_ = add(1, 2)`); likewise `ASSERTS` for a JavaScript/TypeScript span containing `expect(x).toBe(1)` or `expect.assertions(1)`, and `EXECUTES_ONLY` for one that only calls `add(1, 2)` (v1.23.1, F-706). (R-22, C-06, R-104)
T-27	A `PASSING` ID whose only passed edges are `EXECUTES_ONLY` becomes `WEAKLY_PASSING`; with any `ASSERTS` edge it stays `PASSING` and passes `--strict`; with only `UNKNOWN` it stays `PASSING`. (C-05 step 5, R-11, E-26)
T-28	Disabling the judge changes no status except `WEAKLY_PASSING` → `PASSING` (property test over random fixtures). (I-004)
T-29	A stub provider returning `ASSERTS` without evidence, evidence outside the span, or evidence in another file is coerced to `UNKNOWN` with `coerced: true` and rationale `judge: ungrounded`. (E-16, I-005)
T-30	Non-JSON, missing `verdict`, an unknown verdict string, a raised exception, and a timeout each yield `UNKNOWN` with the specified rationale. (E-14, E-15)
T-31	The judge is called exactly once per eligible edge and never for `FAILING`/`SKIPPED`/`UNVERIFIED`/`UNTESTED`/`UNCITED` IDs or for skipped/failed test outcomes (call-counting stub). (I-010)
T-32	Rationale over 280 chars is truncated to 277 + `...`; newlines become spaces. (K-07)
T-33	LLM provider sends exactly one `POST` per edge whose body is byte-for-byte the C-06 shape (`model`, `temperature: 0`, `max_tokens: 4000`, system message equal to the C-10 text, user message equal to the request JSON with line-numbered `source`), with the bearer header, honors the timeout, never retries, and issues at most `--judge-concurrency` requests at once while output order stays fixed (recorded HTTP stub with a concurrency counter); the URL, model, key and timeout are those of the C-09 variables. (K-05, K-06, R-26, C-09)
T-54	The provider reads `choices[0].message.content`, accepts a bare JSON object and one wrapped in a ```` ``` ````/```` ```json ```` fence, and treats a missing path, a non-200 status, or any other text as E-14/E-15; the report carries `judge_prompt_sha256` equal to the SHA-256 of `speccheck/judge_prompt.md`, and that file equals the C-10 text. (C-06, C-10, R-26)
T-74	Extends T-33: for a heading-declared ID with a body, the LLM request's user message carries `statement` equal to `SpecId.text` — title, newline, body with its fenced block and indentation, byte for byte — under the keys `{id, statement, related, declared, file, start, end, source}` (`declared` added in v1.14, C-15; `related` added in v1.16, R-38, D-28) and the system message that is the C-10 text; the C-10 file contains the any-clause rule added in v1.7 and `judge_prompt_sha256` is its SHA-256 (recorded HTTP stub). (R-33, C-06, C-15, C-10, R-26)

ID	Test
T-75	Clause grounding: a stub returning `ASSERTS` with a clause that is a verbatim excerpt of the statement differing only in line breaks and indentation is accepted and recorded with the collapsed excerpt; a 300-character excerpt is accepted and recorded cut to 280; a paraphrase, an 8-character fragment of a longer statement, a missing key, and `""` each yield `UNKNOWN`, `coerced: true`, `judge: unlocated clause`, for `ASSERTS` and for `EXECUTES_ONLY` alike; an `UNRELATED` with a clause is recorded with `""` and not coerced; `"clause": null` on an `UNRELATED` is `""` and not coerced, on an `ASSERTS` it is E-48; an `ASSERTS` whose clause is unlocated *and* whose evidence lies outside the span records `judge: unlocated clause` (rule order); a clause with a leading space that otherwise matches the statement's first words is LOCATED (K-15 trims); a statement of 9 characters is matched by itself and by nothing shorter; the mock judge's clause is the collapsed statement's first 280 characters on every edge of the golden fixture and equals `""` for an ID whose statement is `""`; the LLM request's system message equals the current C-10 text; the JSON verdict object's keys are `verdict, clause, evidence, rationale, coerced` in that order and `schema_version` is the C-07 value; report §8 shows the clause column with an em dash for `""`. (R-34, K-15, E-48, E-49, I-005, C-06, C-07, C-08, C-10, R-22)
T-83	For a judged edge, the C-06 request's user message now carries `related` — the whitespace-collapsed titles (each at most 160 characters) of the obligations the statement names and that name it, its own references first, at most eight in C-07 id order, a retired neighbour keeping its `(retired)` title (E-57), and the empty list when the id has no neighbour — a request-side field like `declared`: absent from `speccheck.json` and from the Markdown report and not inspected by any C-06 coercion rule or by C-08 (D-28); the C-10 system message contains the `related` rule. The triage `state` (C-17) carries the same `related` section (D-28b). (R-38, C-06, C-10, D-28, E-57)

ID	Test
T-89	Triage and the `N%` budget (v1.15): with `--judge llm --jev-pre-triage` over a stub fixture of 10 judge-eligible edges, a stub C-17 provider returning fixed confidences (one edge deliberately failing with HTTP 500) and a stub judge recording every edge it is asked about, `--judge-budget 30%` issues exactly $\lceil 0.30 \times 10 \rceil = 3$ edges — the three with the lowest $p(e)$, the failing edge first among them — in ascending-confidence order, and the other 7 are `UNKNOWN` with rationale `judge: budget` and `coerced: true`, with one Note reporting 7 and one reporting the single E-59 failure; `--judge-budget 100%` issues all 10; `--judge-budget 0%` issues none (judge call count 0) while still running the triage pass (10 C-17 requests, none of them in either report); the C-17 stub receives exactly one request per eligible edge, never more than `--judge-concurrency` at once, each carrying `state` equal to C-17's template for that edge — now including its `Related obligations:` section, the neighbourhood of that edge (D-28b, R-38, E-57) — and `questions.verdict.criteria` covering the four C-06 tokens, and none carrying `clause` or `evidence`; the same run under `--judge mock` (K-16 ignored) makes no C-17 request, needs no `SPECCHECK_JEV_API_KEY`, and leaves the golden reports byte-identical; with `--judge-budget 0` the pass reorders but changes no report content, and the D-30 Note is present; a stub that fails every C-17 call leaves `judge_available` and `unknown_rate` exactly as a triage-free run does. (K-16, K-12, C-17, I-015, E-35, E-59, I-006, D-30, D-32, D-28, R-38, E-57)

9.6 Reports and determinism (C-07, C-08)

ID	Test
T-34	`speccheck.json` validates against the C-07 shape: key order, sort orders, rounding, no absolute paths, no timestamps; `by_family` has all six keys and `by_status` all seven in-scope statuses on every fixture including single-family ones; `judge_available` is `null` for `none`, `true` for `mock`, `true` for `llm` with zero eligible edges, `false` for `llm` when every call fails; every `tests[]` entry carries the `verdict` key, `null` on a `FAILING` ID, on a skipped-outcome edge of a `PASSING` ID, on every edge of a RECORDED ID (E-37, R-35), and everywhere under `--judge none`; `tests[].name` is `""` for a file-level case; a fixture producing five Notes emits them in code-point order in both reports; every ratio has exactly four decimal places and a hand-picked tie value (e.g. 1/8 = 0.125 at 3 places, 0.00005 at 4) rounds half-even to the quantized Decimal, not to `round()`'s binary result. (R-13, R-20, K-09, C-07, E-37, E-38)
T-35	`SPEC_CONFORMANCE_REPORT.md` has the nine C-08 sections in order, the header's judge parenthetical reading `(available)`/`(unavailable)` for mock/llm and absent for none, one per-ID row per declared ID, retired rows with exactly the ID cell struck through, an em dash in the verdict position for every unjudged case and `(file)` for every file-level case, and every `file:line` in it also present in the JSON. (R-12, I-003, E-37, Q-011)

ID	Test
T-36	Two runs on the golden fixture with `--judge mock` produce byte-identical reports and summary lines; the same holds after copying the fixture to a different absolute path, and with `--src .` and `--out` inside the source root (the previous run's reports and the spec itself produce no citations), and with a planted `.speccheck.json.deadbeef.tmp` and `.SPEC_CONFORMANCE_REPORT.md.deadbeef.tmp` under `--out` before the run (never scanned, deleted by the run). (R-16, I-002, E-23, E-34)
T-37	Every metric in the report is recomputable from the report's own evidence table plus the results file (the test recomputes them independently). (R-24)
T-38	Only the two report files are created; input trees are byte-identical before and after (hash comparison). (R-19, I-001)
T-88	`speccheck.json` under `--judge none` and `--judge mock` on the golden fixture (T-46) carries `declared_ratio` in `metrics` and `"declared"` on every `tests[]` entry, `schema_version "1.5"`; the Markdown report is byte-identical to its pre-v1.14 form apart from what T-86 changes; `declared_ratio` is recomputable by hand from the evidence table exactly as R-24 requires of every other metric. (C-07, C-16, R-39)

9.7 CLI, exit codes, diagnostics, boundary (§5)

ID	Test
T-39	`exit_code` in JSON equals the process exit status for fixtures exercising `0`, `1` (non-strict), `1` (strict-only reasons: `UNVERIFIED`, `UNTESTED`, `UNCITED`, `SKIPPED`, `WEAKLY_PASSING`, dangling, stale), and `0` under `--strict` for a mixed `ASSERTS/EXECUTES_ONLY` ID. (R-14, R-15, I-009, E-26)
T-40	Missing `--spec`, bad `--judge`, bad `--verbose` value, any input or `--out` path outside `--root`, and missing judge env each exit `2` with the specified message; the API key value never appears in the message. (K-01, E-09, E-21, R-23, C-09)
T-104	A spec whose `R-04` statement reads `R-01..R-03`, `R-01 to R-03` and `R-01 through R-03`, with `R-01`, `R-02`, `R-03` declared: `R-04` has `depends_on` edges to all three; with `R-02` undeclared, edges to `R-01` and `R-03` only and no Note for `R-02`; `R-03..R-01`, a cross-family range, and a range wider than 100 name only their endpoints. (R-36, C-12; v1.21)
T-105	A table row `| **E-01** | condition | outcome naming R-02 |` yields a `depends_on` edge E-01 → R-02, while `E-01`'s `text` and `title` are `condition` alone. (R-36, C-12; v1.21)
T-106	A row `| **[R-05](#R-05)** | value $a \| b$ holds |` yields exactly one Note `statement may be cut at an unescaped '|': [R-05](#R-05)`; the same row with `|` or inside a backtick span, and a row with an even number of `$`, yield none. A symlink under `--root` pointing outside it, given as `--out`, exits `2` with a message starting `path outside --root:` and ending `(resolves to <the target>)`. (E-66, E-09; v1.21)

ID	Test
T-118	On a copy of `fixtures/target/` without `--results`, `check` prints `speccheck: UNPROVEN - 0/20 passing (0.0%), …` and exits `0`; the same run with `--strict` prints `NOT CONFORMING` and exits `1`; a project whose every in-scope id is `PASSING` with no dangling or stale citation prints `CONFORMING` with and without `--strict`; the same project plus one dangling citation prints `UNPROVEN` without `--strict`; in every case the Markdown §1 line equals stdout, the line matches the §5.1 regex, and `exit_code` in `speccheck.json` equals the process exit status. (R-105, §5.1, §5.4, C-08, R-14, I-009)
T-103	A run that carries both fault kinds — a `--src` element that is neither file nor directory (E-52) together with a malformed `--results` (E-05) — exits `2`, not `3`, and writes no report; the same two faults split across separate invocations exit `2` and `3` respectively, so the precedence is observable and not merely asserted; likewise an unreadable `--results` together with a spec that declares no in-scope id (E-01) exits `2` (v1.23.1, F-709). (K-18, §5.4; v1.20)
T-41	Bare `--verbose` ≡ `--verbose INFO`; INFO emits stage lines with counts and no statement text, file contents, or prompts; DEBUG emits `judge>`/`judge<` lines with the key redacted; stdout is the single summary line in all cases. (R-17, I-007)
T-42	With no `--verbose`, stderr is empty on exit 0/1 even when the run produced Notes; with `--verbose`, each Note appears once at `INFO`. (R-17, §5.3)
T-43	With `--judge none` and `--judge mock`, a socket-creation guard installed for the process lifetime is never triggered; `speccheck --self-check` passes from an installed wheel in an empty working directory, invokes the inner check in-process with exactly the §5.1 argument list (asserted via a recorded `Config`), prints `self-check: ok` although its inner check exits `1`, leaves no files behind, and writes nothing outside its temporary directory. (R-18, I-006, I-001, Q-003)
T-60	`speccheck/_selfcheck/` is byte-identical to `fixtures/target/` (recursive file-by-file comparison, `golden/` included); the test fails if either side changes without the other. (F-107, §10)
T-44	Summary line matches the §5.1 regex exactly, is pure ASCII, ends in a single `\n`, is emitted correctly under a C/POSIX locale and a `cp1252` stdout, carries all seven in-scope status counts including `skipped`, and its numbers equal the JSON metrics. (R-21, R-29, Q-005)
T-59	Under `--strict --judge llm` with a provider stub that fails every call — so that `judge_available` is `false` *and* `unknown_rate` is `1.0000 > max_unknown` — exit is `1`, `strict_judge_failure` is `"unavailable"` (not `"unknown_rate"`), and the summary line ends `judge=llm (unavailable)` (Q-004); with a stub yielding 3 `UNKNOWN` of 10 edges and `--max-unknown 0.2`, exit is `1` with `"unknown_rate"` and the numeric suffix; with `--max-unknown 0.3` exit is `0`; without `--strict` neither affects exit; with a fixture that has zero eligible edges, no request is sent, `unknown_rate` is `null`, and `--strict` exits `0`. (R-28, K-11, E-32, E-36)

ID	Test
T-101	Over a fixture whose `--judge-budget` is `0%` with `--jev-pre-triage` under `--judge llm` and at least one judge-eligible edge: the run issues no C-06 request, `speccheck.json`'s `judge_available` is `false` (never `null`), and every eligible edge's `verdict` is the E-35 `UNKNOWN` with rationale `judge: budget`; adding `--strict` exits `1` with `strict_judge_failure "unavailable"` and the summary-line suffix `(unavailable)`. A second run with the same `N%` but `N` above the eligible count leaves `judge_available true`. (C-07, E-32, E-35, R-28; v1.20)
T-61	With `--judge-budget 1`, `--judge-concurrency 2`, and a provider stub that sleeps 2 s per call over six edges, exactly the two requests issued before the deadline complete and are judged (in-flight requests are not cancelled); the remaining four are `UNKNOWN` with rationale `judge: budget` and `coerced: true`; the Note reports 4; `unknown_rate` includes them; with `--judge-budget 0` all six are judged. (K-12, E-35, Q-010)
T-62	With `--judge llm`, `--progress always`, a provider stub over six edges that sleeps 0.3 s per call, `--judge-concurrency 2`, and stderr captured: splitting the capture on `\r` yields only C-11 draw and erase sequences (after stripping each segment's trailing space padding, every draw matches the C-11 regex); the first draw reads `0/6 edges`, `0:00 elapsed`, `?:??`; `<done>` is non-decreasing across draws; a draw with `6/6` and a full 20-`#` bar exists and is the last draw; the capture ends with the erase sequence (`\r`, W spaces, `\r`) and no `\x1b` byte appears anywhere; every draw is padded to the width of the widest draw so far; the number of `#` in each draw equals $\lfloor 20d/6 \rfloor$ for its `<done>`; with a stub that sleeps 2.5 s on a single edge, at least two draws show `0/1` with distinct `elapsed` values (the 1 s tick); with 32 edges answered instantly at `--judge-concurrency 32`, no more than 10 draws occur in any one second and the final draw still shows `32/32` (coalescing); with `--verbose INFO` added, no `INFO` line occurs between the first draw and the erase and the judge stage's `INFO` lines follow the erase; stdout is the single summary line and both report files are byte-identical to a `--progress never` run over the same stub. (R-30, C-11, K-13)
T-63	No progress bytes reach stderr when: stderr is not a TTY under `--progress auto` (the default in a subprocess capture); `--progress never`; `--verbose DEBUG` with `--progress always`; `--judge mock` or `--judge none` with `--progress always`. With `--progress auto` and a stderr whose `isatty()` returns `True`, the indicator is drawn. `--progress` with any other value exits `2`. Under `--progress always` a stub that raises `KeyboardInterrupt` mid-stage leaves the erase sequence as the last stderr bytes before the `interrupted` message (E-41). (R-30, E-39, E-40, K-01)

ID	Test
T-64	A `KeyboardInterrupt` raised by a provider stub mid-judge, and one injected between §3.1 steps 3 and 4 (after the JSON rename, before the Markdown rename), each exit `3` with stderr exactly `ERROR interrupted` (plus a traceback only under `--verbose DEBUG`), write nothing to stdout, leave no `.tmp` and no `speccheck.json` from this run under `--out`, and leave a previous run's reports intact; the JSON `exit_code` is never consulted (no report exists). With six edges, `--judge-concurrency 2`, and a stub that sleeps 5 s per call, an interrupt delivered to the main thread 0.5 s in ends the run in under 2 s with at most three requests ever started (in-flight requests abandoned, queued edges never issued). (E-41, K-01, I-001, K-05, §3.1)
T-45	Unwritable `--out` exits `3` with nothing written; a failure injected after the JSON rename and before the Markdown rename exits `3` with no `.tmp` left, no `speccheck.json`, and the previous run's `SPEC_CONFORMANCE_REPORT.md` intact (the one stale combination E-18 permits); leftover temporaries from an earlier run are deleted first; on success no `.tmp` remains and the two temporaries used carried the same 8-hex-digit nonce. (E-18, I-001, §3.1)
T-50	`speccheck --version` prints `speccheck <version>` where `<version>` is PEP 440 and equals `importlib.metadata.version("speccheck")`. (K-10)
T-78	PATHS parsing (D-23, C-03): `--src a.py,b.py,src --tests tests/unit,tests/core` produces byte-identical reports to `--src a.py --src b.py --src src --tests tests/unit --tests tests/core` (one list repeatable occurrences); `--src ,,,a,` and `--src a ,, b ,` each yield the list `[a, b]` with no Note and no error; a `--src` or `--tests` whose only occurrence is empty (e.g. `--src \" \"`) yields the empty list, and the `src`/`tests` directory default (when it exists) applies only when the flag is *entirely* absent; a directly-named file is scanned as that one file with every per-file filter applied (K-02/E-10 oversized, E-11 UTF-8 replace, E-13 file-level case, E-29 binary, E-33 ignore files/markers), and a directory element is descended per K-03; a file covered by both a direct element and an overlapping directory element is scanned and produces citations exactly once (I-012), its recorded `--src`/`--tests` root — and hence a directly-named `.swift` file's `MODULE` — being the first covering element in list order, and two inputs naming the same physical file set yield byte-identical reports (I-002); a segment that resolves inside `--root` but exists as neither directory nor file exits `2` with message `--<flag>: no such file or directory: <segment>` and writes no reports (E-52), while a segment outside `--root` exits `2` per E-09. (R-03, R-04, C-03, I-012, D-23, E-52, E-09; the scan-filter half cross-references T-13)

ID	Test
T-90	`--judge-budget 30%` without `--jev-pre-triage` exits 2 with a message naming both flags, before any request of either kind is issued (both stubs record zero calls) and with no report written; so does `--judge-budget 30% --jev-pre-triage --judge mock`; `--judge-budget 30` (the `SECONDS` form) under `--judge mock` is still accepted and ignored, exactly as before v1.15; `--judge-budget 30% --jev-pre-triage --judge llm` with `SPECCHECK_JEV_API_KEY` unset exits 2 naming that variable; no C-09 or C-17 key value appears in any of these messages, and `--judge-budget 101%` and `--judge-budget %` each exit 2 as bad values. (E-58, E-21, K-12, K-16, I-007)

9.8 Golden fixture (end to end)

ID	Test
T-46	`fixtures/target/` — a small self-contained project with its own `SPEC.md` (≥ 12 IDs across all six families, 2 retired, one heading-declared contract with a multi-clause body ≥ 2,048 bytes per T-76, a §12 decision table per T-79), `src/`, `tests/`, and a checked-in `junit.xml` — contains planted defects: one `UNCITED` R, one `UNTESTED` C, one `UNVERIFIED` E, one `FAILING` T, one `SKIPPED` K, one `EXECUTES_ONLY`-only test, one dangling citation, one stale citation, one unattributed result, one file-level citation. The golden reports live in `fixtures/target/golden/` (outside every scan root). `speccheck check --spec SPEC.md --src src --tests tests --results junit.xml --judge mock --strict --root fixtures/target --out <fresh tmp>` produces files byte-identical to `golden/speccheck.json` and `golden/SPEC_CONFORMANCE_REPORT.md` and exits 1 (Q-003); `golden/` also holds the `impact` goldens of T-80. (all of §2)
T-47	Removing each planted defect in turn flips exactly the expected row and metric (one sub-test per defect). (R-06, R-24)
T-71	`fixtures/target-swift/` — a SwiftPM-shaped project with its own `SPEC.md` (≥ 10 IDs across all six families, one carrying `**[port]**` decoration), `Sources/`, `Tests/<Module>/` holding one Swift Testing file (nested suite, parameterized test, disabled test, doc-comment citations) and one XCTest file, and a checked-in `junit.xml` that concatenates the `junit-swift-testing.xml` SwiftPM wrote and an XCTest `<testsuite>` in SwiftPM's shape — contains planted defects: one `UNCITED` R, one `UNTESTED` C, one `UNVERIFIED` file-level citation, one `FAILING` T, one `SKIPPED` K (the disabled test), one `EXECUTES_ONLY`-only test, one undelimited `test*` helper (E-43), one unattributed result. With `--judge mock` both reports are byte-identical to `fixtures/target-swift/golden/`, and the summary line is exactly the one recorded in that golden. (R-31, R-32, R-16, R-24)

ID	Test
T-73	Both golden fixtures regenerated for v1.7 and, as properties of the current output: each `golden/speccheck.json` carries the C-07 `schema_version` and a `title` per ID; in `fixtures/target/golden/speccheck.json` C-01's `statement` is its `title`, a newline, and `The error message MUST name the dividend.` while C-02's (empty body) equals its `title`; in each `golden/SPEC_CONFORMANCE_REPORT.md` the Statement cell of every §3 row equals that ID's JSON `title`, contains no newline, and no row contains the K-14 marker; the set of IDs in the Markdown equals the set in the JSON; `speccheck/_selfcheck/` is updated in step (T-60) and `--self-check` prints `self-check: ok`. The one-time v1.6 → v1.7 golden diff (only `schema_version` and the `title` keys added, heading-declared `statement`s changed, Markdown unchanged) is recorded in `SPEC_BUILD_REPORT.md`, not asserted by a test (F-302). (R-33, C-07, C-08, T-46, T-71)
T-76	`fixtures/target/SPEC.md` declares one heading-declared contract whose SECTION BODY is at least 2,048 bytes and consists of a fenced code block pinning a struct with field comments followed by at least five numbered rules; `tests/` holds four tests that each assert exactly one of those rules or the struct's shape and cite the contract, one test that calls the code and asserts nothing about it while citing the contract, and one test that asserts a different ID's behaviour while citing the contract; the six are attributed to their own cases and appear in `golden/judge_labels.json` as `ASSERTS` ×4, `EXECUTES_ONLY` ×1, `UNRELATED` ×1; the label file has at least 37 entries of which at least 8 are downgraded (`EXECUTES_ONLY` or `UNRELATED`) on an adjacent edge — a test that cites an id, runs its code, and asserts only a fact about an id that id's *statement* names (the v1.16 adjacent subset, T-84; C-10's vocabulary makes such a test `EXECUTES_ONLY`, since it does exercise the statement's code) — and at least 6 are edges of an ID whose statement exceeds 2,048 bytes; under `--judge mock` the contract is `PASSING` and the T-46 goldens are byte-identical. (T-46, T-49, T-84, R-34, R-38, D-28)
T-86	`fixtures/target/` gains one test whose own docstring names a different id (its DECLARED citation) but whose body also cites an existing id only as fixture/example data (mirroring the real `R-01`/`R-02`/`R-03` pattern `JUDGE_CROSSCHECK_REPORT.md` §2b found): under `--judge none`, `speccheck.json` records `declared: false` for that second edge and the fixture's `declared_ratio` reflects it; `SPEC_CONFORMANCE_REPORT.md` is unchanged by this test (C-16 is JSON-only), and every other T-46 row stays byte-identical. (C-14, C-16, R-39, T-46)

9.9 Self-application (recorded, not gating)

ID	Test
T-48 *(recorded)*	`speccheck check --spec SPEC.md --src src --tests tests --results <this suite's junit.xml> --judge mock` on this repository reports every R/C/I/K/E/T ID in this document as `PASSING`. Recorded in `SPEC_BUILD_REPORT.md`; it is evidence *for* the build, not the proof of it — the proof is §9.1–§9.8. (R-24)

9.10 Performance (recorded)

ID	Test
T-51 *(recorded)*	A benchmark script (`tools/bench.py`, not collected by pytest) runs `check --judge mock` on the golden fixture and on the generated 10,000-file tree five times each and prints the median wall-clock; the medians, the reference-machine description, and the pass/fail against K-08 are recorded in `SPEC_BUILD_REPORT.md`. Not run in CI. (K-08)

9.11 LLM judge evaluation (opt-in, `--judge llm`, not run in CI)

ID	Test
T-49 *(recorded)*	On the golden fixture's judged edges (each hand-labeled `ASSERTS`/`EXECUTES_ONLY`/`UNRELATED`), the LLM judge's non-`UNKNOWN` verdicts agree with labels at ≥ 0.90 accuracy and `unknown_rate` ≤ 0.10 in **each** of three independent runs (no pooling; one failing run fails T-49); all three per-run figures are recorded with model name, date, and `judge_prompt_sha256`. The labels live in `fixtures/target/golden/judge_labels.json`, a JSON object mapping `"<ID> <file>::<name>"` (the judged edge) to its label, with at least 20 entries of which at least 6 are edges of a statement over 2,048 bytes (T-76), so that a judge which grades a body by its gist fails the run while one that locates the clause passes; the runner is `tools/eval_judge.py` (not collected by pytest), which runs the check three times and scores each run against the labels (F-210). A run counts toward T-49 only if its recorded `judge_prompt_sha256` equals the SHA-256 of the current C-10 text; a change to C-10, or to C-01's statement rule, therefore voids the recorded runs and requires three fresh ones, and the labels are re-read against the current statements before those runs (F-303). (R-10, R-26)
T-87 *(recorded)*	Re-run the three real edges named in `docs/proposals/PROPOSAL_v1.15_declared_vs_incidental_citations.md`'s evidence (`R-03/test_fenced_code_blocks_are_ignored`, `R-03/test_row_and_heading_grammar_edge_cases`, `R-02/test_judge_called_once_per_eligible_edge_only`, all recorded `ASSERTS` by `gpt-4o-mini` under the pre-v1.14 `C-10` text and `UNRELATED` by an independent second model) under the v1.14 `C-10` text with the same model, and record the three verdicts and `judge_prompt_sha256` in `SPEC_BUILD_REPORT.md`. Not gating: the claim this proposal makes is falsifiable, and this is the test of it — the honest possible outcomes are "moved on all three," "on some," or "on none," and D-26 is revisited if the answer is the last (§3 of that proposal). (R-39, C-15, D-26)

ID	Test
T-91 *(recorded)*	The calibration measurement of `docs/proposals/PROPOSAL_v1.16_jev_pre_triage.md` §1 re-run against this repository's own current `SPEC.md`/`src/tests`: a `check --judge llm --strict` run, its committed verdicts (recorded `UNKNOWN` excluded) bucketed by each edge's C-17 $p(e)$ against the C-17 model's own `answers.verdict.choice`, with the bucket sizes, the per-bucket agreement, and the C-17 model name, date and `judge_prompt_sha256` recorded in `SPEC_BUILD_REPORT.md`. Passes — non-gating, like T-49 and T-87 — when the buckets are monotonically non-increasing; a failure is a lead to re-examine D-30 and D-32, not a build break. The suite cites this id by a presence check that the recorded bucket table exists and has at least three rows. (K-16, C-17, D-32)
T-84 *(recorded)*	The v1.16 evidence (D-28b): the golden fixture's adjacent subset — the judged edges that cite an id but assert only a fact about an id that id's *statement* names — is measured to be not all `EXECUTES_ONLY`/`UNRELATED` (which would leave the neighbourhood change a no-op on this fixture); `judge_labels.json` holds at least 37 entries of which at least 8 are downgraded (`EXECUTES_ONLY` or `UNRELATED`) on an adjacent edge, mirroring T-76's extension, and the measured downgrades are recorded with `model`, `date` and `judge_prompt_sha256` in `SPEC_BUILD_REPORT.md`. Not gating: the honest outcomes are "moved", "on some", or "on none", and if the fixture's adjacent subset is all `EXE`/`UNREL` the fixture is grown, not the claim (T-49, T-87). (R-38, C-10, D-28, T-76, T-49)

9.12 Edges and impact (C-12, C-13; v1.13)

ID	Test
T-79	Edge and decision extraction on the golden fixture, whose `SPEC.md` gains a §12 decision table (header cells in an order other than the template's, so the *Affects* column is found by name) with at least three `D-nn` rows — one whose *Affects* cell names two declared ids, one naming an undeclared id, one naming the retired C-03 — plus one `D-nn` row outside that table and one bold `**D-09**` row inside it (both declare nothing): the regenerated `golden/speccheck.json` carries the C-07 `schema_version` and the `decisions` and `edges` arrays exactly as pinned; `ids`, every status, every metric, `dangling`, `stale`, `notes` other than the new C-12 Notes, and `SPEC_CONFORMANCE_REPORT.md` are byte-identical to their v1.12 values (T-73 property, extended); an obligation whose statement names an undeclared id yields the Note `edge to undeclared id: <X> -> <Y>` and no edge; a T naming a T is `depends_on`; an obligation naming its T and that T naming the obligation yield one `verifies` edge with the T as `src`; the *Affects* cell naming C-03 yields an edge with `retired: true` (E-55) and a `decisions[].affects` entry; `D-8` is reported as `D-08`; a spec whose decision table declares `D-01` twice exits `3` naming both lines (E-02); a spec with no decision table has `decisions: []`; a statement's self-reference and a repeated token add nothing; edges are in C-12 order on every fixture including one whose ids are declared out of order. (R-36, C-01 (c), C-02, C-07, C-12, E-02, E-55)
T-80	`speccheck impact --spec SPEC.md --changed K-02 --src src --tests tests --root fixtures/target --out <fresh tmp>` produces `impact.json` and `IMPACT_REPORT.md` byte-identical to `golden/impact.json` and `golden/IMPACT_REPORT.md`, exits `0`, and prints exactly one stdout line matching the C-13 regex whose five numbers equal the JSON's list lengths; the same command with `--depth 0` lists more rows in `impact`, its rows of depth ≤ 1 are the default run's rows in the same order with the same `via` (I-013), and the depth-cap Note is present in the default run and absent under `--depth 0`; two runs are byte-identical, and so is a run from a copy of the fixture at another absolute path (I-002); after a prior `check` into the same `--out`, an `impact` run leaves `speccheck.json` and `SPEC_CONFORMANCE_REPORT.md` byte-identical and writes only its two files, and `check` after `impact` leaves the two `impact` files untouched (I-001); `--changed D-01` reaches exactly D-01's *Affects* ids at depth 1 via `affects` edges; `--changed T-05` lists T-05 in re-verify with `verifies: []` and impacts none of the ids T-05 verifies; `--changed K-2` is accepted as `K-02` (I-011); without `--src` §4 reads `Not scanned.` and `recite` is `[]`; without `--tests test_cases` is `[]`; the retired C-03 in `--changed` is accepted and its rows are struck through (E-55); `--self-check` is unchanged and still compares only the two `check` goldens. (R-37, C-13, I-001, I-002, I-011, I-013, K-09, E-55)

ID	Test
T-81	`--against`: a copy of the fixture spec with one statement reworded, one id newly retired, one id removed, one id added, and one decision's *Affects* cell extended yields a changed set of exactly those five with the C-13 reasons, in C-12 id order, the removed id carrying its line in the `--against` file and appearing in no `impact` row; a byte-identical copy yields an empty changed set, every section `None.`, exit `0`, and the summary `speccheck impact: 0 changed, 0 impacted (depth 1), 0 to re-verify, 0 citations, 0 test cases`; an `--against` file with a duplicate declaration exits `3` with the E-02 message prefixed `--against:`; `--changed R-99`, `--changed X-01`, and `--changed ,` each exit `2` with the E-53 (or `--changed: no ids`) message; `--changed` with `--against`, neither, `--results junit.xml`, and `--judge mock` each exit `2` with the E-54 message; `--depth 1000`, `--depth -1`, `--depth x` exit `2`; in every exit-2/3 case no file is written under `--out` (I-001). (C-13, E-09, E-53, E-54, K-01, I-001)
T-82 *(recorded)*	The backtest `tools/impact_backtest.py` (D-24; not collected by pytest; needs `git` and this repository's history) on two ranges of `main` — spec `d170433^..c0a770a` built by `2a25569..2635298`, and spec `2635298..c1e3d87` built by `330dd4e..dfea1a6` — runs `impact --against` on the two spec versions and compares the predicted set at each depth cutoff $d \in \{1, 2, 3, \infty\}$ with the ids whose citations (per the built tree's `speccheck.json`) lie in the build range's touched hunks; recall and precision per cutoff, the miss list, and the chosen `--depth` default are recorded in `SPEC_BUILD_REPORT.md`; the bar is recall ≥ 0.80 at the chosen depth on both ranges, or every miss explained as an edge the prose does not and should not carry. The suite cites this id by a presence check that the script exists and `--help` exits `0`. (R-37, C-13, D-24)
T-121	A spec with an isolated R id, an isolated E id and a retired isolated C id yields one Note `2 declared id(s) touch no edge: R-.., E-..`, the retired id uncounted; adding a statement that names the R id removes it from the Note; with five isolated ids the Note lists three and ends `and 2 more`; the `fixtures/target` golden carries `3 declared id(s) touch no edge: R-03, K-01, E-01`; `impact.json` carries no such Note. (E-77, C-12, R-36, D-60)

9.13 `explain` (C-18; v1.17)

ID	Test
T-92	`speccheck explain R-01 --spec SPEC.md --src src --tests tests --results junit.xml --judge mock --root .` on a copy of the fixture prints exactly the C-18 trace for R-01 — `ID R-01`, `status: PASSING (step 4: every citing test case with a result passed or was skipped)`, `statement:`, its `sources:` line, its `tests:` case with outcome and `[ASSERTS]`, and the `impact (1):` section — exits `0`, writes nothing at all (the fixture copy is byte-identical before and after, T-38's rule; `explain` has no `--out`, D-33), and a second identical run is byte-identical; the status shown equals R-01's status in `golden/speccheck.json` from the same inputs (I-016, I-002 extended to this surface). (R-40, C-18, I-016)

ID	Test
T-93	`explain R-09 …` (an id `fixtures/target` does not declare) exits 2 with the E-60 message and writes no trace; `explain R-04` (the fixture's retired id) renders `ID R-04 (RETIRED)` with `status: RETIRED (step 1: struck through in the specification)` and exits 0; `explain C-02` (declared, cited in `src/` only) renders `status: UNTESTED (step 2a: no test citation)` with an empty `tests:` block and exits 0; an `UNVERIFIED` id's reason names `step 3`, and a case with an `<error>` result renders `(error)` (v1.23.1, F-711). (R-40, C-18, E-60)
T-102	`speccheck explain R-01` with no `--spec` exits 2 with the E-64 message and writes no trace; `speccheck explain --help`'s synopsis shows `--spec` unbracketed and its entry carries the C-19 §3 `default:`/precondition clause the `check` entry carries; the rendered `explain` help golden is updated for that one entry. (C-19, E-64, R-41; v1.20)
T-94 *(recorded)*	Under `--judge llm`, `explain <a judge-eligible id>` renders the C-06 verdict with its `clause:` and `rationale:` lines; under `--judge none` the same id's edge lines read `not judged`; in both, the id's status in the trace equals its status in `speccheck.json` from the same inputs (I-016). Recorded in `SPEC_BUILD_REPORT.md`; not gating, like T-49 — it exercises a live model. (R-40, C-18, E-61)

9.14 CLI help (C-19; v1.18)

ID	Test
T-95	For every flag whose accepted set is finite (`--judge`, `--progress`, `--verbose`, `--max-unknown`, `--judge-concurrency`, `--judge-budget`, `--depth`), the value tokens and range text named in its `--help` entry equal those named in the message the same flag's own usage error prints (e.g. `check --judge bogus` → `expected none, mock, or llm`, exit 2), and every token the help names is accepted: a run over the §9.8 fixture carrying that token exits other than 2, while the control token `bogus` exits 2. (R-41, C-19)
T-96	Every argument definition of the four parsers except `-h` carries a non-empty help string; each flag's rendered entry names a metavar from C-19's vocabulary and, where the flag has a default, that default; and each of the four `--help` screens, run from an empty temporary directory with the R-18 socket guard installed, exits 0, writes a non-empty screen to stdout and nothing to stderr (I-017). (R-41, C-19, I-017)
T-97	With `COLUMNS=80` fixed, the four `--help` screens render byte-stable text compared against checked-in goldens under `tests/data/help/` (not §9.8 report goldens: C-19's wording is not normative, D-38), so a dropped value token, a dropped default, a new flag without help, or an unintended rewording is a visible diff. (R-41, C-19, D-39)

ID	Test
T-98	The `environment:` block names exactly the variables the kernel reads and no others: scanning `src/speccheck/*.py` for string literals matching `SPECCHECK_[A-Z_]+` yields the same set the rendered `check --help` block lists (8 — `SPECCHECK_JUDGE_URL/_MODEL/_API_KEY/_TIMEOUT`, `SPECCHECK_JEV_URL/_MODEL/_API_KEY/_TIMEOUT`), each entry naming its read condition and requiredness, with `COLUMNS` documented beside them (D-41); a variable read by the kernel but absent from the block, or named in the block but read by no code, fails the test; and no `_API_KEY` value ever appears in any help screen or usage-error message (R-23, I-007). (R-41, C-19)

9.15 Proof evidence (C-20, C-21, K-17, E-62, E-63, E-65; v1.19)

ID	Test
T-99	Over the §9.8 golden fixture extended with a minimal Lean proof (three declarations, two tagged, one untagged) and a two-entry manifest: `check --spec … --src … --tests … --results … --proof <fixture lean> --proof-results <fixture manifest>` exits as the golden pins; the JSON carries, for the tagged declarations, `proof` citations with state `checked`, for the untagged declaration no citation, for the manifest-failed entry state `failed` with the error line, for the cited-but-unmanifested entry `unknown`; the untagged file with no parseable declaration head yields a `notes` entry, not an error (E-62); the Markdown shows the proof column; `grep -c "proof"` on both goldens is the pinned count; a `--proof-results` file that is unreadable, not well-formed JSON, or missing `build/theorems` exits `2`/`3` per E-65; and a run over the `hello_world_deepseek` sidecar's evidence (`proof` + `build/proof/proof-results.json`, both produced by `speccheck/tools/proof_evidence.py`, under `--root .`) exits `0` with every dual id showing state `checked` — a cross-repository integration recorded, not pooled (R-24); `impact --spec … --proof proof` and `explain R-01 --spec … --proof-results build/proof/proof-results.json` each exit `2` with the amended E-54 message, naming `--proof`. (R-100, C-20, C-21, K-17, E-62, E-65, E-54)
T-100	Byte-identity under flag absence: `check` over the unextended §9.8 fixture, with no `--proof`/`--proof-results`, reproduces both v1.18 goldens byte for byte (I-018), and the rendered `check --help` (T-97's golden) shows exactly two new flag rows, each with C-19-compliant help. (I-018, C-19)

9.16 JavaScript/TypeScript adapter (C-03, C-04, E-74, E-75, E-76; v1.22)

ID	Test
T-113	A vitest-style file with nested `describe`s, `it.each`, `it.skip`, a regex literal containing `)` and a `/** … */` block above a test: the cases are exactly those calls, named `outer > inner > title`, with the comment block inside the span and the call's closing-paren line as its end; whole-line comment lines are flagged and code lines are not. REGEX vs DIVISION (F-708): `f(/a)b/)`, `return /c)/`, `x = /d[/)]/g` and a `/e)/` at the start of a file each read as one regex token (the `)` inside does not count toward balance), while `expect(a) / 2` and `total / count / 2` read as division; a file mixing them delimits its cases exactly as the rule predicts. (R-104, C-24)
T-114	A non-literal title and an interpolated template are listed as `line N` and are not cases; `re.test(x)` is not a call; a file with an unclosed `(` raises the parse error that becomes `parse fallback: <path>`. (E-74, E-75)
T-115	Attribution: a citation in a case's span attaches to it; classname is the path with `/` → `.` and the extension kept; a comment-only citation is DECLARED, a trailing comment is not; a citation on the second line of a multi-line `/* … */` comment inside the span is DECLARED (C-14's JS/TS bullet, F-705); a citation outside every case is file-level. (R-104, C-24, C-14)
T-117 *(recorded)*	One real `jest` + `jest-junit` run over a file with a nested `describe` (two levels) and a test outside every `describe`, configured with C-03's three options, checked in as a fixture string: every `<testcase>` joins its C-03 case and none is unattributed — the top-level one through C-04's leading-separator rule; the jest and jest-junit versions and the date are recorded in `SPEC_BUILD_REPORT.md`. Until it is recorded, the jest path is documented, not claimed (C-03, F-707). (R-104, C-24, C-04, D-56)
T-116	End to end over the `vitest run --reporter=junit` output measured 2026-10-27 (checked in as the test's fixture string): path classname relative to vitest's root joins under `--root`, titles containing `[` and `(` join, an `it.each` title with two filled results joins as one case, and the skipped test's id is `SKIPPED`; no result is unattributed. (R-104, C-24, C-04, E-76)

9.17 Namespaces and the citation map (C-22, C-23, I-019, K-19, E-67..E-73; v1.23)

ID	Test
T-107	A test docstring naming `platform:R-99` against a spec declaring `R-99` leaves `R-99 UNCITED`, lists the token in `foreign` with its file and line, and emits the Note `1 foreign reference(s) ignored: platform:R-99 <file>:<line>` naming the docstring's file and line; with four such tokens the Note lists three and ends `and 1 more`; with `--namespace platform` the same project reports `R-99 PASSING`, `foreign` is absent and no Note is emitted; `impact` and `explain` over the same inputs agree. Two ids joined by a colon in a test comment (the first one an ID-shaped label) are both local citations. A namespace longer than 32 characters (`regulated_platform_core_services_v2_extra:R-99`) is foreign, not local (F-712), while `--namespace` with a 33-character value exits `2`. (R-101, R-102, E-67, E-68, C-22)
T-108	A declared statement containing `platform:R-99` yields no `depends_on` edge to `R-99` and no Note, while a bare `R-99` in the same statement still yields it; an *Affects* cell likewise; `foreign` does not list either. A range `platform:R-01..R-05` with R-01..R-05 declared yields one edge, to R-05 (F-716). (R-101, E-69, C-12)
T-109	Inertness: the §9.8 golden fixture, the Swift fixture and `--self-check` outputs are byte-identical to v1.22's, the summary line and the exit code included. (I-019)
T-110	A map resolving a Python test case, a file-level case and a JavaScript case reproduces the statuses of the same project with literal citations; each mapped `tests[]` entry has `mapped: true` and `lines: []`; the summary line ends `; citations: 3 mapped`; the Markdown renders `<file> (map)`; under `--judge mock` the mapped edge is judged and its request carries `declared: true` (D-57); `explain` renders `(mapped)`; `impact` lists the mapped test as a re-cite site; a literal duplicate stays `mapped: false` and does not raise the count; an undeclared id is dangling and a retired one stale, at the case's start line, each entry carrying `mapped: true` while literal dangling/stale entries carry `mapped: false` (F-704); `declared_ratio` is the same with and without the map (mapped citations are outside it, F-703); and the same run with the map file placed under `--tests` produces byte-identical reports — the map is never scanned (F-702), nor is a `--proof-results` manifest under `--tests`. (R-103, C-23, C-03, C-07, E-72, E-73)
T-111	Map faults: an unresolved entry (Note, `citation_map.unresolved`, exit code unchanged), malformed JSON, wrong `schema_version`, an entry with a missing or extra key, a `ns:`-prefixed id, a map over 2 MiB, a map path outside `--root` and an unreadable one each exit as E-70 specifies and write no report; an invalid `--namespace` exits `2` before any file is read. (E-70, E-71, E-68, K-19)
T-112 *(recorded)*	`check --strict` over this repository and over at least two further real projects (named in the run's record), before and after v1.23: ids, statuses and exit code identical, and the number and first three of the foreign references found recorded (expected `0` here). Recorded, not gating: it measures D-51's claim that default-on recognition moves nothing in a real project, and a non-zero count in another project is the evidence to reopen D-51. (R-101, D-51)

9.18 Provenance manifest and hazards (C-25..C-28, I-020, I-021, K-20, E-78..E-85; v1.25)

ID	Test
T-122	On the §9.8 golden fixture, `check --judge mock --manifest` writes `speccheck.manifest.json` whose keys and order match C-25; every `inputs[].sha256` and `bytes` equals `hashlib.sha256` and `len` of that file's bytes on disk, and the list holds exactly the spec, every scanned file and the results file, each with its role, sorted by (path, role); `tree_sha256` equals the value recomputed from the `shasum` line format; `reports[]` hashes and lengths equal the two report files written; `tool.version` equals `__version__`; the two reports are byte-identical to the same run without `--manifest`; the manifest is not scanned on a second run with `--out` inside `--tests`. (R-106, C-25, C-03, I-021)
T-123	Two `check --judge mock --manifest` runs over identical inputs give byte-identical manifests; with `SOURCE_DATE_EPOCH=1793836800` the manifest ends with `generated_at 2026-11-05T00:00:00Z`, and without the variable the key is absent; `SOURCE_DATE_EPOCH=abc` and `SOURCE_DATE_EPOCH=` each exit `2` with E-85's message and write nothing, while `SOURCE_DATE_EPOCH=abc` without `--manifest` changes nothing; a CRLF twin of the spec gives byte-identical reports and a different `inputs` hash for the spec; a failure injected before the manifest rename leaves none of the three files (§3.1). (I-020, C-25, E-85, I-001)
T-124	`verify` against the T-122 manifest on the untouched tree prints exactly `verify: ok - <N> files match, tree <12 hex>` and exits `0`; after one byte of a scanned test file changes it prints `verify: mismatch <that path>` then `verify: FAILED - 1 of <N> files differ` and exits `1`; after `SPEC_CONFORMANCE_REPORT.md` is deleted it prints `verify: missing <path>`; a missing `--manifest` and one outside `--root` exit `2` (E-82); a non-JSON manifest, a `manifest_version` of `"2.0"`, a missing `reports` key, and an entry with path `../x` each exit `3` (E-83) without reading any input; no file under `--root` changes (hash comparison) and no socket opens (socket guard). (R-107, C-26, E-82, E-83, I-001)
T-125	The v1.25 evidence as a test: on a copy of the golden fixture, `check --judge mock --manifest` runs; one line is appended to a `--src` file; a second `check` without `--manifest` writes reports byte-identical to the first run's (the v1.24 behaviour, kept); `verify` against the first manifest exits `1` and its only `mismatch` line names that `--src` file. (R-106, R-107)

ID	Test
T-126	A fixture whose `R-01 *(controls: HAZ-2, HAZ-1)*` is PASSING, `C-01 *(controls: HAZ-2)*` is FAILING, retired `~~R-02~~ *(controls: HAZ-5)*`, and heading `### [C-02](#C-02) *(controls: HAZ-3)* Title` is PASSING: `HAZ-1` and `HAZ-3` are `VERIFIED`, `HAZ-2` `NOT_VERIFIED`, `HAZ-5` `UNCONTROLLED` with E-81's Note; `[R-01](#R-01)`'s `controls` is `["HAZ-1", "HAZ-2"]`, `[C-02](#C-02)`'s title is `Title`, and only marked ids carry the key; `| **[R-03](#R-03)** *(recorded)* *(controls: HAZ-4)* |` names `HAZ-4`; the marker on `[T-01](#T-01)` gives E-78's Note and `*(controls: )*` and a 65-character HAZID give E-80's; under `--judge mock` a WEAKLY_PASSING control makes its hazard `NOT_VERIFIED`; every id status, metric, edge and the Markdown §3 table are byte-identical to the same fixture without markers; `hazards` and `## 10. Hazards` render per C-07/C-08; `explain [R-01](#R-01)` prints `controls: HAZ-1 (VERIFIED), HAZ-2 (NOT_VERIFIED)` after its `status:` line; `impact` output is unchanged by the markers. (R-108, C-01, C-02, C-27, E-78, E-80, E-81, C-18)
T-127	With `--risk-file` listing `HAZ-1` (titled), `HAZ-2` and `HAZ-9` over T-126's fixture minus the retired id: `HAZ-9` is `UNCONTROLLED` with E-81's Note; `--strict` exits `1` although every in-scope id is PASSING; without `--strict` the run exits `0` and prints `UNPROVEN`; `HAZ-1`'s title appears in both reports; a marker naming `HAZ-7` gives E-79's Note and no exit change; an empty `hazards` list over a marker-free spec yields `hazards: []` and `; hazards: 0/0 verified`; a duplicate id, a bad HAZID, a non-string title, a missing `hazards` key and a 2 MiB + 1 byte file each exit `3` with E-84's message and no report; an unreadable file and one outside `--root` exit `2`; `--risk-file` given to `impact` or `explain` exits `2`; the file is never scanned for citations. (R-109, R-15, R-105, C-28, E-79, E-81, E-84, K-20, E-09)
T-128	I-021: on the §9.8 golden fixture, the Swift fixture and every fixture of T-109, a v1.25 `check` with neither new flag over a marker-free spec reproduces the committed v1.24 goldens byte for byte, with the same summary line and exit code, and creates no third file; `--self-check` passes unchanged. (I-021)
T-129	With hazards present the summary line ends `; hazards: 1/2 verified`, matches the §5.1 regex (T-44), is ASCII, and follows `; citations: <M> mapped` when a citation map also resolves; the regex still matches every v1.24 line; C-08's §1 verdict line carries the same suffix; the `check --help` screen names `--manifest`, `--risk-file` and `SOURCE_DATE_EPOCH`, and `verify --help` names `--manifest FILE` and its four exit codes (C-19, T-97's goldens regenerated). (C-27, §5.1, C-19, R-41)

---

10. Dependencies and environment

- **Runtime:** Python 3.12; `uv` for environment and lockfile. Install: `uv sync --extra dev`; with the model judge: `uv sync --extra dev --extra llm`.
- **Kernel dependencies:** none beyond the standard library (`re`, `ast`, `xml.etree.ElementTree`, `json`, `argparse`, `pathlib`, `logging`, `hashlib`). The lean adapter (`lean.py`, v1.19, C-20) is the same line-based, standard-library-only pattern as `swift.py` — no Lean toolchain is required to run `check` itself,

since `--proof-results` is read, never executed (C-21).

- **Package data:** `speccheck/judge_prompt.md` (C-10) and `speccheck/_selfcheck/` (a byte-identical copy of `fixtures/target/` including its `golden/` directory, kept in sync by T-60; F-011, F-107), both included in the wheel.
- **Golden fixture layout:** `fixtures/target/{SPEC.md, src/, tests/, junit.xml, golden/speccheck.json, golden/SPEC_CONFORMANCE_REPORT.md, golden/impact.json, golden/IMPACT_REPORT.md, proof/, proof-results.json}` (`impact.json`/`IMPACT_REPORT.md` since v1.13, T-80; `proof/` — a three-declaration Lean fixture — and `proof-results.json` — its two-entry manifest — since v1.19, T-99); `golden/` is never a scan root and is never written to by a run (Q-003).
- **`[llm]` extra:** `httpx` (HTTP client with per-request timeout). No provider SDK; the wire format is C-06. The C-17 triage pass uses the same client and the same extra.
- **`[dev]` extra:** `pytest`, `pytest-cov`, `hypothesis` (T-28), `ruff`.
- **Environment variables:** C-09 (only with `--judge llm`), C-17 (only with `--jev-pre-triage` under `--judge llm`; v1.15), and `SOURCE_DATE_EPOCH` (only with `check --manifest`; v1.25, C-25, E-85). No configuration files are read.
- **Host prerequisites:** none. No network access is required except with `--judge llm`, and — when `--jev-pre-triage` is given — the C-17 triage pass that flag enables. The kernel never invokes `git` or any subprocess (D-24).
- **Help goldens (T-97, v1.18):** `tests/data/help/{speccheck,check,impact,explain,verify}_help.txt` (`verify` from v1.25) — the C-19 screens rendered at `COLUMNS=80`, checked in as test data (not a §9.8 report golden; D-39).
- **Tools (not collected by pytest, not part of the wheel):** `tools/bench.py` (T-51), `tools/eval_judge.py` (T-49), `tools/sync_selfcheck.py` (T-60), `tools/impact_backtest.py` (T-82; the only one that needs `git` and the repository history; D-24), `tools/proof_evidence.py` (v1.19; a standalone sidecar — reads a project's own `proof/` lake project and test suite, writes the C-21 manifest `--proof-results` reads; not part of the kernel and not required to run `check`, T-99's cross-repository integration case).
- **Commands:**

```
uv sync --extra dev
uv run python -m pytest tests -q --junitxml=junit.xml     # the §9 suite; junit.xml feeds T-48
uv run ruff check src tests
uv run speccheck --self-check
uv run speccheck check --spec SPEC.md --src src --tests tests --results junit.xml --judge mock
uv run speccheck impact --spec SPEC.md --changed K-15 --src src --tests tests               #
 ↪  v1.13; or --against <prior SPEC.md>
uv run speccheck explain R-01 --spec SPEC.md --src src --tests tests --results junit.xml --judge
 ↪  mock   # v1.17; stdout only
uv run speccheck check --spec SPEC.md --src src --tests tests --results junit.xml --proof proof
 ↪  --proof-results build/proof/proof-results.json --judge mock   # v1.19
uv run speccheck check --spec SPEC.md --src src --tests tests --results junit.xml --judge mock
 ↪  --manifest --out build/rec   # v1.25; then:
uv run speccheck verify --manifest build/rec/speccheck.manifest.json                        #
 ↪  v1.25; exit 0 = nothing changed
```

- **Optional items:**
  - **O-1** LLM judge provider (`--judge llm`, `[llm]` extra). Specified fully in C-06/C-09; gated by flag and extra. The C-17 triage provider (`--jev-pre-triage`, `[llm]` extra, v1.15) is the same kind of item: fully specified in C-17/K-16, gated by flag and extra, and never the source of a status (I-015).
  - **O-2** Additional language adapters for test-case attribution (Go `func Test*`, and others). Not in v0.1; the fallback path is what v0.1 ships and tests (T-12). Swift joined the shipped adapters in v1.6 (R-31, C-03) and JavaScript/TypeScript in v1.22 (R-104, C-03); Go remains optional.
  - **O-3** Any non-CLI surface (GUI, HTTP, editor plugin). Not in v0.1 and not designed for.

---

11. Traceability matrix (id → where realized)

Until the build exists, "where realized" names the component the §1/§4 design assigns; `spec-build` replaces it with the real module and the real test.

Spec id	Where realized (component / module)	Verified by
R-01	`extract.py` (declaration scan, UTF-8 replace)	T-01, T-05, T-46, T-72
R-02	`extract.py` (retired flag), `graph.py` (denominators)	T-04, T-25, T-46
R-03	`extract.py` (source scan, PATHS list parsing per C-03, C-03 exclusions, binary/symlink filters)	T-08, T-13, T-36, T-46, T-78
R-04	`attribute.py` (Python adapter, Swift adapter, fallback, C-03 exclusions), `cli.py` (PATHS list parsing)	T-09, T-10, T-11, T-12, T-36, T-56, T-46, T-65, T-66, T-67, T-78
R-05	`results.py`	T-15, T-16, T-52, T-58, T-46, T-68
R-06	`graph.py` (C-05 steps 1–4)	T-20, T-21, T-47, T-46
R-07	`graph.py` (dangling)	T-23, T-46
R-08	`graph.py` (stale)	T-23, T-46
R-09	`graph.py` (metrics)	T-24, T-46
R-10	`judge.py`, `judge_mock.py`, `judge_llm.py` (passed-outcome edges only)	T-26, T-31, T-33, T-49, T-46
R-11	`graph.py` (C-05 step 5)	T-27, T-28, T-46
R-12	`report.py` (Markdown)	T-35, T-46
R-13	`report.py` (JSON)	T-34, T-46
R-14	`cli.py` (exit mapping)	T-39, T-46
R-15	`cli.py` (`--strict`, status-only)	T-39, T-27, T-46, T-127
R-16	all kernel modules (sorted, no volatile fields)	T-36
R-17	`cli.py` (logging setup, level `ERROR` default)	T-41, T-42
R-18	`cli.py` (`--self-check`, pinned in-process invocation), `speccheck/_selfcheck/`, `judge_llm.py` import isolation	T-43, T-60
R-19	`report.py` (only writer, temp-and-rename), `cli.py`	T-38, T-43, T-45
R-20	`extract.py`, `report.py` (path normalization)	T-34, T-36
R-21	`cli.py` (summary line)	T-44, T-59, T-118
R-22	`judge_mock.py` (Python, Swift and JS/TS assertion tokens; prefix clause)	T-26, T-69, T-75
R-23	`judge_llm.py`, `cli.py` (env validation, redaction)	T-40, T-41
R-24	`report.py` (evidence table completeness)	T-37, T-48
R-25	`graph.py` (C-05 step 2b)	T-53
R-26	`judge_llm.py` (body, response path, prompt hash), `speccheck/judge_prompt.md`	T-33, T-54, T-74
R-27	`extract.py` (ignore markers)	T-57
R-28	`cli.py` (`--strict` judge gate), `report.py` (`strict_judge_failure`)	T-59
R-29	`cli.py` (summary line encoding)	T-44
R-30	`cli.py` (`--progress` resolution, TTY test), `judge.py` (progress callback in `run_judge`)	T-62, T-63
R-31	`attribute.py` (Swift adapter: type/func lines, attribute block, brace spans, MODULE), `results.py` (signature strip), `judge_mock.py` (Swift tokens)	T-65, T-66, T-67, T-68, T-69, T-71
R-32	`extract.py` (first-cell prefix rule)	T-70, T-71

Spec id	Where realized (component / module)	Verified by
R-33	`extract.py` (heading title, SECTION BODY, K-14 cap and Note), `report.py` (`title` key; Markdown renders `title`), `judge_llm.py` (statement passthrough)	T-72, T-73, T-74
R-34	`judge.py` (clause validation, K-15 matcher, E-48/E-49 coercion), `judge_llm.py` (`clause` in the reply), `judge_mock.py` (prefix clause), `report.py` (`clause` key, §8 column), `speccheck/judge_prompt.md`	T-75, T-76, T-49
R-35	`extract.py` (RECORDED marker, `SpecId.recorded`), `graph.py` (step 5 skip, eligible edges), `report.py` (`recorded` key, ID cell)	T-77
R-36	`extract.py` (decision-table scan, `Decision`, `Edge`, C-12 edge builder and Notes), `report.py` (`decisions`, `edges`)	T-79
R-37	`impact.py` (changed set, `--against` diff, walk, re-verify, re-cite), `cli.py` (`impact` subcommand), `report.py` (C-13 renderers)	T-80, T-81, T-82
R-38	`judge_llm.py` (builds the `related` neighbourhood into the C-06 request: the statement's own references first, then those that name it, at most eight, each a 160-char whitespace-collapsed title, a retired neighbour tagged `(retired)` (E-57), the empty list when none), `jev.py` (the D-28b `Related obligations:` section in the C-17 `state` template), `speccheck/judge_prompt.md` (the `related` field and its rule, C-10) — a request-side field: not in `speccheck.json`, C-16, or C-08	T-83, T-84
R-39	`attribute.py` (docstring-span and whole-line-comment detection, Python, Swift and JS/TS; mapped citations DECLARED), `report.py` (`declared`, `declared_ratio`)	T-85, T-86, T-88
R-40	`cli.py` (the `explain` subparser, `ExplainConfig`, the render wiring), `explain.py` (the C-18 trace renderer)	T-92, T-93, T-94
R-41	`cli.py` (the help strings, metavars and epilogs of the four parsers)	T-129, T-95, T-96, T-97, T-98, T-102
R-100	`lean.py` (new adapter), `cli.py` (`--proof` parsing)	T-99
R-101	`extract.py` (foreign pass before the TOKEN rule; `citations_in_file`, `targets_of`), `cli.py`	T-107, T-108, T-112
R-102	`cli.py` (`--namespace` on `check`/`impact`/`explain`), `extract.py`	T-107, T-111
R-103	`cli.py` (`--citation-map`), `attribute.py` (`Citation.mapped`), `report.py` (`mapped`, `citation_map`, summary suffix, `(map)`), `explain.py` (`(mapped)`)	T-110, T-111
R-104	`jsts.py` (tokenizer, CALL/TITLE/span rules), `attribute.py` (dispatch, classname), `results.py` (path classname, case join_name, templated titles)	T-113, T-114, T-115, T-116, T-117
R-105	`report.py` (`summary_line`: the `UNPROVEN` word from R-15's conditions)	T-118
R-106	`report.py` / a new `manifest.py` (render after both reports, hashes of the bytes read, the third temp-and-rename), `cli.py` (`--manifest`)	T-122, T-123, T-125
R-107	`cli.py` (the `verify` parser), `manifest.py` (re-hash, C-26 lines, exit codes)	T-124, T-125
R-108	`extract.py` (CONTROLS marker → `SpecId.controls`), `graph.py` (hazard roll-up after C-05 step 5)	T-126
R-109	`cli.py` (`--risk-file` read and validation), `graph.py` (`UNCONTROLLED`, R-15's hazard clause)	T-127

Spec id	Where realized (component / module)	Verified by
C-01	`extract.py` (`ID_RE`, fence tracker, row/heading parsers incl. first-cell decoration, RECORDED marker and section bodies, decision rows (c), ignore markers)	T-126, T-01, T-02, T-03, T-04, T-05, T-55, T-57, T-70, T-72, T-77, T-79, T-119
C-02	`extract.py` (`SpecId` with `title`, `text` and `recorded`, `Decision`, `Edge`, `SpecIndex`)	T-01, T-06, T-72, T-77, T-79
C-03	`attribute.py` (`TestCase`, `Citation` incl. `declared`, `test*` methods, Swift adapter, JavaScript/TypeScript adapter), `jsts.py`, `extract.py` (exclusions incl. temporaries, binary, symlinks), `cli.py` (PATHS list parsing; D-23)	T-09, T-10, T-13, T-14, T-36, T-56, T-65, T-66, T-67, T-78, T-85, T-110, T-113, T-114, T-115
C-04	`results.py` (two-step `join_name` on results and cases, path classnames, templated titles)	T-15, T-16, T-17, T-18, T-19, T-52, T-58, T-68, T-116, T-117
C-05	`graph.py` (`IdStatus`, `compute_status`, recorded skip in step 5)	T-20, T-21, T-27, T-53, T-77
C-06	`judge.py` (`JudgeRequest` with numbered `source`, full statement, `declared`, and `related` (R-38), `Verdict` with `clause`, validation incl. K-15), providers; `judge_mock.py` tokens	T-26, T-29, T-30, T-32, T-33, T-54, T-69, T-74, T-75, T-83, T-87
C-07	`report.py` (`to_json`; `title`; `recorded`; `clause`; `decisions`; `edges`; `declared`; `declared_ratio`; Decimal quantization; `verdict: null`; Note order; `judge_available` total over the eligible-but-unissued case, v1.20)	T-126, T-127, T-34, T-37, T-59, T-73, T-75, T-77, T-79, T-86, T-88, T-101, T-110
C-08	`report.py` (`to_markdown`; Statement cell from `title`; `(recorded)` ID cell; §8 Clause column; em dash and `(file)` renderings)	T-126, T-127, T-35, T-73, T-75, T-77
C-09	`judge_llm.py` (`from_env`)	T-33, T-40
C-10	`speccheck/judge_prompt.md` (incl. `declared` and `related` fields, the skepticism rule, and the `related` rule (R-38)), `judge_llm.py` (system message)	T-33, T-54, T-74, T-75, T-83, T-84, T-87
C-11	`judge.py` (progress line rendering, draw/erase sequences)	T-62
C-12	`extract.py` (token pass over `SpecId.text` and `SpecId.extra`, ranges, direction rules, order, undeclared-target Notes, foreign endpoints)	T-79, T-104, T-105, T-108, T-121
C-13	`impact.py` (changed set and reasons, breadth-first walk, `via`, depth cap, REVERIFY, RECITE, TEST_CASES), `report.py` (`impact.json`, `IMPACT_REPORT.md`, summary line)	T-80, T-81
C-14	`attribute.py` (docstring-span and whole-line-comment classification, Python, Swift and JS/TS; the mapped-citation rule)	T-85, T-86, T-110, T-115
C-15	`judge.py` (`JudgeRequest.declared`), `judge_llm.py` (`declared` in the request body), `speccheck/judge_prompt.md` (field definition, skepticism rule)	T-87
C-16	`report.py` (`tests[].declared`, `metrics.declared_ratio`)	T-86, T-88
C-17	`jev.py` (the four C-17 variables, the request body and `state` template — now including the D-28b `Related obligations:` section, R-38 — the `answers.verdict` parse and `p(e)`), `cli.py` (variable validation and redaction)	T-89, T-90
C-18	`explain.py` (the trace renderer: section order, headings, the C-05 reason, the verdict lines, the impact section)	T-126, T-92, T-93, T-94

Spec id	Where realized (component / module)	Verified by
C-19	`cli.py` (every argument definition's help, the C-19 metavar vocabulary, the `environment:` and exit-code epilogs)	T-129, T-95, T-96, T-97, T-98, T-102
C-20	`lean.py` (declaration split, doc-comment tag extraction)	T-99
C-21	`lean.py`/`cli.py` (manifest read and join)	T-99
C-22	`extract.py` (`FOREIGN` pass, `foreign` list), `cli.py`	T-107, T-108, T-111
C-23	`cli.py` (map read, validation, resolution), `attribute.py` (`Citation.mapped`), `report.py`	T-110, T-111
C-24	`jsts.py` (tokenizer, REGEX vs DIVISION, CALL/TITLE/span rules, `each` patterns), `attribute.py` (dispatch, classname)	T-113, T-114, T-115, T-116, T-117
C-25	`manifest.py` (key order, `inputs` roles and sort, `tree_sha256`, `generated_at`)	T-122, T-123
C-26	`manifest.py` (`verify`: read order, `missing`/`mismatch` lines, counts)	T-124
C-27	`extract.py` (grammar), `graph.py` (H, CTRL, statuses), `report.py` (`controls`, `hazards`, `## 10. Hazards`, summary suffix), `explain.py` (`controls:` line)	T-126, T-127, T-129
C-28	`cli.py` (risk-file shape, faults, scan exclusion)	T-127
I-001	`report.py` (temp-and-rename, interrupt cleanup), `cli.py`	T-123, T-124, T-07, T-38, T-43, T-45, T-60, T-64
I-002	kernel modules	T-36
I-003	`report.py`	T-25, T-35
I-004	`graph.py`	T-27, T-28
I-005	`judge.py` (validation: evidence in span, clause located)	T-29, T-75
I-006	`cli.py`, module import layout	T-43
I-007	`cli.py` (logging filter), `judge_llm.py` (redaction)	T-40, T-41
I-008	`graph.py` (metrics)	T-24
I-009	`cli.py`, `report.py`	T-39
I-010	`graph.py` (edge selection: PASSING, passed, not recorded), `judge.py`	T-31, T-77
I-011	`extract.py` (normalization)	T-02
I-012	`extract.py` (deduplicate by resolved path, first-seen `scan_root`, emit by ascending path)	T-78
I-013	`impact.py` (breadth-first order, first-reached depth, smallest `via`)	T-80
I-014	`attribute.py` (`declared` is a pure function of the source/test trees), `report.py` (`declared_ratio`)	T-85, T-88
I-015	`jev.py` (answer discarded after the order is computed), `report.py` (no triage field in either report)	T-89
I-016	`cli.py` (the explain path writes no report and carries the judge contract unchanged), `explain.py` (reads only pipeline facts)	T-92, T-94
I-017	`cli.py` (no help path reads a file, an environment variable or a socket)	T-96
I-018	`cli.py`, `report.py` (conditional `proof` key, absent-flag byte-identity)	T-100
I-019	`extract.py`, `cli.py`, `report.py` (every addition conditional on its input)	T-109
I-020	`manifest.py` (no time, host or absolute path; `SOURCE_DATE_EPOCH` only)	T-123
I-021	`cli.py`, `report.py`, `explain.py` (every v1.25 addition conditional on its flag or marker)	T-128
K-01	`cli.py`	T-39, T-40, T-19, T-64

Spec id	Where realized (component / module)	Verified by
K-02	`extract.py` (file filters, binary rule)	T-13
K-03	`extract.py` (walker, no symlinks)	T-13
K-04	`extract.py` (`ID_RE`)	T-03
K-05	`judge_llm.py` (full-lifecycle timeout)	T-33
K-06	`judge_llm.py` (concurrency), `cli.py` (`--judge-concurrency`)	T-33
K-07	`judge.py`	T-32
K-08	kernel (performance)	T-51
K-09	`report.py`	T-34
K-10	`cli.py` (`--version`)	T-50
K-11	`cli.py` (`--max-unknown`, Decimal comparison, `null` rule)	T-59
K-12	`judge.py` (budget deadline, in-flight completion, `N%` issued count), `cli.py` (`--judge-budget`)	T-61, T-89, T-90
K-13	`judge.py` (redraw on completion, 1 s ticker, single erase)	T-62
K-14	`extract.py` (statement cap at line boundary, marker line, Note)	T-72
K-15	`judge.py` (whitespace-collapsed substring matcher, 12/280 bounds)	T-75
K-16	`jev.py` (triage pass over the `related`-bearing `state` (D-28b), confidence ordering, D-30 Note), `judge.py` (issue order), `cli.py` (`--jev-pre-triage`)	T-89, T-90
K-17	`lean.py` (join on name/file/line; stale/unknown/failed states)	T-99
K-18	`cli.py` (validation stage order: flags/values/containment before spec/results/out reads)	T-103
K-19	`cli.py` (map size check)	T-111
K-20	`cli.py` (risk-file size), `extract.py` (HAZID length), `manifest.py` (manifest size in `verify`)	T-124, T-126, T-127
E-01	`extract.py`, `cli.py`	T-07
E-02	`extract.py`	T-06
E-03	`extract.py`	T-04
E-04	`extract.py` (fence tracking)	T-05
E-05	`results.py`	T-19
E-06	`results.py`	T-17
E-07	`results.py`, `report.py`	T-18
E-08	`graph.py` (`unrun`)	T-22
E-09	`cli.py` (path validation incl. `--out`; a PATHS element resolved and used, incl. neither-file-nor-directory → E-52; `resolves to` suffix)	T-40, T-78, T-99, T-106
E-10	`extract.py`	T-13
E-11	`extract.py` (scan roots and `SPEC.md`)	T-13, T-01
E-12	`attribute.py`	T-11
E-13	`attribute.py`, `report.py`	T-10
E-14	`judge.py`, `judge_llm.py`	T-30, T-33
E-15	`judge.py`	T-30
E-16	`judge.py`	T-29
E-17	`judge.py`	T-26
E-18	`report.py` (write/rename/cleanup, JSON-then-Markdown)	T-45
E-19	`graph.py`, `cli.py`	T-20, T-39
E-20	`extract.py` (strikethrough only in spec)	T-23
E-21	`cli.py`	T-40
E-22	`attribute.py`	T-14
E-23	`extract.py` (exclusions)	T-36
E-24	`results.py` (join_name, worst-of)	T-52
E-25	`graph.py` (step 2b)	T-53
E-26	`graph.py` (step 5), `cli.py` (`--strict`)	T-27, T-39

Spec id	Where realized (component / module)	Verified by
E-27	`results.py` (longest-suffix join)	T-58
E-28	`attribute.py` (class recognition, `test*` methods)	T-56
E-29	`extract.py` (binary rule)	T-13
E-30	`extract.py` (walker; descent-time symlinks only)	T-13
E-31	`extract.py` (row/heading parsers)	T-55
E-32	`cli.py` (R-28 gate, `unavailable` precedence, extended to E-35 v1.20)	T-59, T-101
E-33	`extract.py` (ignore markers)	T-57
E-34	`extract.py` (temporary exclusion), `report.py` (leftover cleanup)	T-36, T-45
E-35	`judge.py` (budget, both forms)	T-61, T-89
E-36	`judge.py`, `cli.py` (R-28 with `null`)	T-59
E-37	`graph.py` (eligibility), `report.py` (`verdict: null`, em-dash rendering)	T-34, T-35, T-77
E-38	`report.py` (Note ordering)	T-34
E-39	`cli.py` (`--progress` gating)	T-63
E-40	`judge.py` (erase in `finally`)	T-63
E-41	`cli.py` (`KeyboardInterrupt` → exit 3), `report.py` (cleanup), `judge.py` (erase; timed polling of futures, `abort` flag), `judge_llm.py` (abortable deadline wait, daemon transport thread)	T-63, T-64
E-42	`attribute.py` (Swift brace fallback)	T-67
E-43	`attribute.py` (Swift undelimited `test*`)	T-65, T-66, T-71
E-44	`extract.py` (first-cell decoration)	T-70, T-71
E-45	`results.py` (tie → unattributed)	T-68
E-46	`extract.py` (cap and Note; empty body → title)	T-72
E-47	`extract.py` (inner declarations parsed; body ends only at level ≤ own)	T-72
E-48	`judge.py` (unlocated clause → `UNKNOWN`)	T-75
E-49	`judge.py` (clause blanked for `UNRELATED` / `UNKNOWN`, non-string clause, coerced verdicts)	T-75
E-50	`extract.py` (marker on a non-T id → Note)	T-77
E-51	`graph.py` (steps 1–4 unchanged for recorded ids)	T-77
E-52	`cli.py` (a PATHS element inside `--root` but neither a directory nor a file → usage 2)	T-78
E-53	`cli.py` / `impact.py` (`--changed` element validation and message)	T-81
E-54	`cli.py` (`impact` flag exclusivity; rejected `check`-only flags; `--against` errors)	T-81, T-99
E-55	`extract.py` (`Edge.retired`), `report.py` (struck ID cells in the impact report)	T-79, T-80
E-56	`attribute.py` (`declared: false` for a `"src"`-kind or file-level citation)	T-85, T-86
E-57	`judge_llm.py` / `jev.py` (a retired id that appears in a `related` neighbourhood keeps its `title` with `(retired)` appended; the `affects`/`depends_on` edge to it is still marked `retired` per E-55)	T-83
E-58	`cli.py` (`--judge-budget N%` requires `--jev-pre-triage` with `--judge llm`; message names both flags)	T-90
E-59	`jev.py` (per-edge failure ordered first, counted in one Note; `judge_available`/`unknown_rate` untouched)	T-89
E-60	`cli.py` (the `explain` id validation and its message)	T-93
E-61	`cli.py` (the judge flags carried over), `explain.py` (verdict rendering incl. `coerced` and `not judged`)	T-94
E-62	`lean.py` (unreadable/non-Lean file → notes entry)	T-99

Spec id	Where realized (component / module)	Verified by
E-63	`graph.py` (C-05 untouched by proof), `impact.py` (proof edges ignored)	T-99
E-64	`cli.py` (`explain`'s required `--spec`, its message)	T-102
E-65	`cli.py` (`--proof-results` read/validation)	T-99
E-66	`extract.py` (`_odd_math_delimiters` and the header cell count, Note in `parse_spec`)	T-106, T-120
E-67	`extract.py` (`foreign` list, Note), `report.py`	T-107
E-68	`extract.py`, `cli.py` (`--namespace` validation)	T-107, T-111
E-69	`extract.py` (`targets_of`)	T-108
E-70	`cli.py` (map read and validation)	T-111
E-71	`cli.py` (map resolution, Note), `report.py`	T-110, T-111
E-72	`cli.py` (map resolution), `report.py`, `explain.py`	T-110
E-73	`cli.py` (map resolution)	T-110
E-74	`jsts.py` (`JsParseError`), `attribute.py` (`parse fallback` Note)	T-114
E-75	`jsts.py` (`undelimited`), `attribute.py` (Note)	T-114
E-76	`jsts.py` (`each_pattern`), `results.py` (templated join)	T-116
E-77	`extract.py` (isolated ids after the C-12 pass, one Note)	T-121
E-78	`extract.py` (marker on a T id → Note)	T-126
E-79	`graph.py` (hazard not in the risk file → Note)	T-127
E-80	`extract.py` (malformed marker → Note)	T-126
E-81	`graph.py` (`UNCONTROLLED`, Note), `cli.py` (strict exit)	T-126, T-127
E-82	`cli.py` (`verify --manifest` usage faults)	T-124
E-83	`manifest.py` (manifest validation before any read)	T-124
E-84	`cli.py` (risk-file validation)	T-127
E-85	`cli.py` (`SOURCE_DATE_EPOCH` validation, usage class)	T-123

---

12. Open questions and decisions to confirm

Every row below is a decision the specification's author made on the requester's behalf because `one_sentence_prompt.md` did not settle it. The normative rows assume the default; the reviews checked that the default is precise, not that it is what the requester wanted. Rows are the `D-nn` family (decisions to confirm). A row marked `confirm` is ratified by a human changing its status to `confirmed v1.n`; a row the requester overturns becomes a `fix(speccheck):` change with a version bump. Since v1.13 this table is machine-read: it is the C-01 (c) decision table, and each row's *Affects* cell is the normative list of the ids the decision touches — the source of the `affects` edges (C-12) that `impact` walks. An id a decision affects and the cell omits is a defect the T-82 backtest can find.

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-01	Implementation stack	Python 3.12 + `uv`, standard-library kernel, one HTTP client behind an extra	Go or Rust (single static binary; stack-neutral repos would not need a Python runtime); TypeScript (nearest to most agent tooling)	§10, C-02/C-03 type pins, K-10	requester / confirm

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-02	Test-result input contract	JUnit XML, consumed; the tool never runs tests	run the suite itself (simpler for users, but executes untrusted code and ties the tool to one runner); pytest's own JSON report (richer, Python-only)	R-05, C-04, non-goal "no test execution"	requester / confirm
D-03	What counts as a citation	a literal ID token anywhere in a file, with `speccheck:ignore` opt-outs	a structured annotation (`@spec R-07`, a decorator, a registry) — precise but requires adoption; docstring-only citations — misses comments	C-01, R-03, R-04, R-27, the §0 limitation	requester / confirm
D-04	Family-T semantics	option (a): T ids are in scope, cited by their own test, and count in `conformance`	option (b): T ids declared but out of scope, reported separately — leaves `conformance` about R/C/I/K/E only	R-25, C-05 step 2b, C-07 metrics, T-53	requester / confirm (F-001 chose (a) on the reviewer's recommendation)
D-05	Mixed judge verdicts under `--strict`	an ID with both `ASSERTS` and `EXECUTES_ONLY` edges is `PASSING` and passes strict	a `MIXED` status that fails strict — stricter, noisier	R-15, C-05 step 5, E-26	requester / confirm (F-003)
D-06	Judge question and vocabulary	one question per edge — does this test *assert* the behavior? — with `ASSERTS` / `EXECUTES_ONLY` / `UNRELATED` / `UNKNOWN`; from v1.7 answered at the granularity of a clause: a statement with several clauses is asserted when any one of them is (C-10, R-33)	a graded strength score (rejected: a number the model made up); judging code rather than tests (rejected: semantic analysis is a non-goal)	C-06, C-10, I-004, I-005	requester / confirm

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-07	Judge request parameters	`temperature 0`, `max_tokens 4000` (v1.2; was 400), 30 s timeout, concurrency 4, budget unlimited, `--max-unknown 0.2`	any of these numbers; `max_tokens 400` was rejected after the first T-49 runs because thinking models (`qwen3:8b`, `gemma4:latest`) spend the budget on reasoning and are truncated before the verdict JSON, driving `unknown_rate` above `--max-unknown`; at 4000 both pass T-49 three runs out of three. The other numbers remain defaults with no data behind them	C-06, K-05, K-06, K-11, K-12	requester / `max_tokens` confirmed; the rest remain confirm

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-08	Judge instruction text and model	the C-10 text as written (v1.9: clause-first question, `clause` field); no default model — the operator's `SPECCHECK_JUDGE_MODEL`	any rewording; the text's only evidence is T-49	C-10, R-26, T-49, T-76	requester / confirm — re-opened again in v1.9: the text changed (R-34); confirm after three fresh T-49 runs over the T-76 label set with each candidate model, and record both. Measured 2026-09-18 under the v1.8 text: `openai/gpt-4o-mini` (fast; graded long bodies by their gist) and `google/gemini-3.8-flash` (located clauses, stricter, 4–8× the wall-clock, needs `--judge-concurrency` 8) — both passed the nine-label T-49 at 1.000, which is why T-76 exists
D-09	Input limits	files over 2 MiB skipped; NUL byte in the first 8 KiB means binary; IDs 1–3 digits	larger cap, or none; MIME sniffing; 4-digit IDs	K-02, K-04, E-10, E-29	requester / confirm
D-10	LLM endpoint shape	an OpenAI-compatible `chat/completions` body, provider-agnostic	a specific vendor SDK (simpler, vendor-locked); a local-model-only path	C-06, C-09	requester / confirm
D-11	Fixture and self-check layout	`fixtures/target/` with `golden/`, byte-copied into the wheel as `speccheck/_selfcheck/`	package only the goldens and regenerate the fixture; skip the self-check entirely	§10, T-43, T-46, T-60	requester / confirm

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-12	Report file names and location	`SPEC_CONFORMANCE_REPORT.md` and `speccheck.json` under `--out`, default `.`	a `reports/` directory; a single JSON with Markdown derived by a separate renderer	§3.3, C-07, C-08, E-18	requester / confirm
D-13	Python test-case delimitation	`def test_*` at module level; `test*` methods in `Test*` classes and `*TestCase` subclasses; nested classes excluded	honor `pytest.ini` `python_functions` / `python_classes`; collect via pytest itself (rejected: runs code)	C-03, E-28, T-56	requester / confirm (F-007, F-102)
D-14	Reference machine for K-08	named in `SPEC_BUILD_REPORT.md` at build time	a CI runner with a generous bound; no performance constraint at all	K-08, T-51	build owner / open
D-15	Progress indicator shape and gating	a single in-place ASCII line on stderr (`#/-` bar of 20 cells, done/total, elapsed, ETA per C-11), padded with spaces and redrawn with a bare `\r` (no terminal escapes; v1.4, F-205), on by default only when stderr is a TTY and not at DEBUG, erased when the judge stage ends, `--progress auto|always|never` to override; LLM judge only	leaving the final line on screen (rejected: §5.3 quiet-by-default would then have a visible exception at exit); a per-edge log line instead of a bar (rejected: that is what `--verbose DEBUG` already is); Unicode block characters (rejected: R-29 locale reasoning); a bar for `--judge mock` too (rejected: mock is sub-second, K-08); a spinner without ETA (rejected: the requester's complaint is duration, so ETA is the useful number)	R-30, C-11, K-13, E-39, E-40, §5.1, §5.3	requester / confirm (the *existence* of the bar is the requester's ask of 2026-09-13; its shape is the author's default)
D-16	Exit code on interrupt	`SIGINT`/`KeyboardInterrupt` exits `3` with message `interrupted`, keeping K-01's closed set `{0,1,2,3}`; temporaries and any already-renamed report removed	the shell convention `130` (rejected by default: it widens K-01 and every CI wrapper that switches on the code; easy to adopt if the requester prefers it); leaving Python's default (rejected: traceback, exit `1`, indistinguishable from `NOT CONFORMING`)	E-41, E-40, §5.4, K-01, I-001, T-64	requester / confirm (F-204)

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-17	How Swift test files are delimited	by lines and brace counting inside `attribute.py` (C-03 TYPE LINE / FUNC LINE / ATTRIBUTE BLOCK / SPAN END rules; comments and string literals excluded from the count), so the kernel stays standard-library and needs no toolchain at check time	a real parser (`swift-syntax` has no Python binding; `swiftc -dump-parse` or `swift test list` need a Swift toolchain where the checker runs and give names but not line spans); treating `.swift` as file-level as before (the citations then never join — the defect that motivated v1.6)	R-31, C-03, E-42, E-43, T-65..T-67	requester / confirm
D-18	Where the Swift `MODULE` in a classname comes from	the first path component of the file under its `--tests` root, or the root's last component for a file directly under it, or — a file named *directly* as a `--tests` PATHS element (D-23) — its parent directory's last path component (SwiftPM's `Tests/<Target>/` layout)	parse `Package.swift` for target names and paths (a second grammar); a `--swift-module` flag (one more thing to get wrong); match on the type chain alone ignoring the module (ambiguous across targets)	C-03, T-65, T-68, T-78	requester / confirm
D-19	How a Swift Testing result name (`twoArgs(a:b:)`) is joined	C-04 strips the signature and joins on the bare identifier; overloads by label tie and are unattributed (E-45)	reconstruct the label signature in the adapter and join exactly (correct for overloads, but default arguments, `_` labels, generics and `inout` all need parsing to get right); join on signature when present and fall back to identifier (two rules where one suffices)	C-04, E-45, T-68	requester / confirm

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-20	What a heading-declared ID's statement contains	the title plus the whole section body, fenced code blocks included, capped at 16,384 bytes (R-33, C-01 (b), K-14) — the pinned shape is the contract, and it is what a literal judge needs	prose only, code blocks dropped (smaller; rejected because MonteCarloPi's C-02 would still be judged on comment-free prose); the first fenced block only (loses prose clauses such as "throws on the first tick", and bullet-pinned shapes such as C-05's reservoir); the body to the judge but the title as JSON `statement` (two notions of "statement" in one tool; the report could not show what the judge saw); letting the judge fetch context itself (nondeterministic, provider-specific, against the §0 boundary). The cap's value is part of this decision: the proposal drafted 8,192 bytes, which would have truncated this document's own C-03 (8.4 kB) in T-48 and handed the judge a Swift-less C-03; the requester raised it to 16,384 on 2026-09-18 so no known contract is cut	R-33, C-01, C-02, C-06, C-07, C-10, K-14, E-46, E-47, T-72..T-74	requester / confirmed v1.7 (code blocks included, cap 16,384; 2026-09-18)

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-21	How a verdict is tied to the statement	the judge quotes the clause it judged against and the kernel requires it to be LOCATED (whitespace-collapsed substring, 12..280 characters; K-15), coercing an unlocated `ASSERTS`/`EXECUTES_ONLY` to `UNKNOWN` — the same grounding pattern as evidence lines (R-34, E-48)	prompt rewording only, no `clause` field (smaller; unverifiable — the kernel cannot tell a located match from a gist, and the report cannot show which clause was judged); sending only a "relevant" slice of the body (the kernel cannot know which clause a test is about; D-20's rejected alternative); voting across repeated calls (K-06: one request per edge; hides instability); fuzzy clause matching (arguable at the boundary; "quote it" means verbatim, and a model that cannot is the finding)	R-34, C-06, C-07, C-08, C-10, K-15, E-48, E-49, I-005, T-75	requester / confirmed v1.9 (2026-09-18: "apply proposal 1.9 in full")
D-22	Recorded-not-gating T ids under an honest judge	v1.10: a `*(recorded)*` marker in the T row's ID cell (C-01) makes the id RECORDED — its citing test still has to exist and pass (C-05 steps 1–4, E-51), but its edges are never judged (step 5, I-010), so a presence check is what it is and the strict LLM gate stays about assertions. This document marks T-48, T-49 and T-51	no rule (v1.9's state: an honest judge marks the presence check `EXECUTES_ONLY` and `--strict --judge llm` exits 1 on this repository by design — `gemini-3.8-flash` did so to T-48 on 2026-09-18); a stated assertion for the presence check (fragile: what a presence check can honestly assert differs per recorded test); excluding family T from the judge stage entirely (too broad: T ids realized by real tests benefit from the judge)	R-35, C-01, C-02, C-05 step 5, I-010, C-07, C-08, E-50, E-51, T-77	requester / confirmed v1.10 (2026-09-18: "let's do D-22")

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-23	`--src`/`--tests` accept a comma-separated list of files and/or directories	each occurrence is split on the literal `,`, every resulting segment is whitespace-trimmed and empty segments are dropped (no Note, no error); the surviving paths are resolved inside `--root` and deduplicated by resolved path — a file reached by more than one element is scanned and cites exactly once (I-012), the emits are in ascending-path order (I-002), and a file's recorded `--src`/`--tests` root, and thus a directly-named `.swift` file's `MODULE` (D-18), is fixed by the first-seen covering element; a directory element is descended and a regular file is scanned as one with every per-file filter applied; the default `src`/`tests` (the directory if it exists) applies only when the flag is entirely absent	a strict "any empty or whitespace-only segment is a usage error" policy (rejected: a trailing `--src a,` or a defensive `a,,b` is common and should not fail a run); extending the same list to `--results` (rejected for D-23: `--results` stays one JUnit file — a documented possible later step); a shared path-list primitive for every path flag, including `--out` (rejected: `--out` is a single destination, and a list of output directories is odd); letting a merely present-but-empty flag fall back to the directory default (rejected: an explicitly-given empty list is the operator's statement "no trees", which is E-19)	R-03, R-04, C-03, §5.1, E-09, E-30, E-52, I-012, I-002, T-40, T-78, D-18	requester / confirmed 2026-09-19 ("trim + drop empties", scoped to `--src`/`--tests`)

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-24	Where the change-impact backtest lives	`tools/impact_backtest.py`, a script outside the kernel that shells out to `git` for two spec versions and a build range, calls `speccheck impact --against`, and scores the prediction against the touched citations (T-82). The kernel exposes only the pure function of files (`--against FILE`) and never invokes `git` or any subprocess.	a kernel `--since REF` that reads the prior spec from `git` (one command for the operator, but the kernel gains a subprocess, a history-dependent input, and a failure mode per git state; I-001/I-002's "same bytes in, same bytes out" would need a git-state clause)	R-37, C-13, §3.1, §10, T-82, I-002	requester / confirmed 2026-09-19 (`docs/proposals/PROPOSAL_v1` §5)
D-25	How decision rows enter the JSON	a separate `decisions` array (C-07) under a declaration-only family D (C-01 (c)): never in `ids`, never a citation target, never in `by_status`, `by_family`, `conformance`, `dangling`, or `stale`; I-003 and every existing golden §3 row are untouched	a seventh family in `ids` with a status of its own (uniform with the other families, but every denominator, the T-46 goldens, and the C-08 table change, `by_family` needs a seventh key, and `UNCITED`/`UNTESTED` are meaningless for a decision)	R-36, C-01, C-02, C-07, C-12, I-003, T-79	requester / confirmed 2026-09-19 (`docs/proposals/PROPOSAL_v1` §5)

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-26	Whether `declared: false` changes the judge's verdict on its own	advisory only (Part B as written): `C-10`'s instruction text tells the model `declared` and asks it not to credit `ASSERTS` on an INCIDENTAL citation without checking the assertion's subject, but no C-06 rule coerces the verdict — preserves the model's judgment for the real, undocumented-but-correct cases; T-87 measures whether the text alone is enough	hard coercion: `declared: false` forces `ASSERTS` to `EXECUTES_ONLY` automatically, no model discretion — stronger and fully deterministic, but risks false downgrades on tests that simply predate or don't follow the docstring convention perfectly; kept as the fallback if T-87 shows the advisory text moves nothing	C-06, C-10, C-15, T-87	requester / confirmed 2026-09-20 (`docs/proposals/PROPOSAL_v1` §5)
D-27	How a citation line is classified DECLARED	whole-line-comment detection: the first non-whitespace character on the citation's own line is `#` (Python) or the line is a doc-comment line by the Swift adapter's existing recognition (C-14) — no new parsing machinery, matches every real DECLARED example found while drafting the proposal; v1.23.1 extends it to the JS/TS adapter's comment-only lines, which that adapter's tokens already identify (F-705)	full token-level comment detection (via `tokenize`, distinguishing a trailing comment from code): more precise (would credit `x = 1 # R-02` as DECLARED) but needs column-accurate token classification the citation scanner does not carry today, for a case that did not appear once in the real evidence read for the proposal; left for a later proposal if it proves necessary	C-03, C-14, T-85, T-115	requester / confirmed 2026-09-20 (`docs/proposals/PROPOSAL_v1` §5)

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-28	How wide the judge request's `related` neighbourhood reaches, and whether it rides the Jev triage `state` (this row carries both: the reach decision and its sub-decision, recorded here as **D-28b**)	**(a)** `related` is the titles of the obligations this statement names **and** of those that name it (C-12 `depends_on`, both directions), the statement's own references first, at most eight total, each a whitespace-collapsed title tail-truncated to 160 characters with …, `[]` when the id has no neighbour; a retired neighbour keeps its title with `(retired)` appended (E-57). `(b)` **D-28b**: `related` is included in the C-17 triage `state` (not stripped) — Jev ranks the exact request the real judge gets — at a cost of roughly doubling the per-edge triage input, which is cents (§3 of `docs/proposals/PROPOSAL_v1_14_obligation_aware_judge.md`).	**(a-alt)** own references only, uncapped: smaller on heavily-depended-on ids (C-01 and C-07 have dozens of dependents) but misses the reverse-direction case the §4 flags — a test that cites E-09 yet asserts E-52's message is the same off-topic `ASSERTS` the other way round. **(b-alt)** strip `related` from the triage `state`: cheaper (no neighbourhood pass) at the price of a weaker proxy for "least-sure-about"; kept as the fallback if a T-89 run shows triage ranking is insensitive to the neighbourhood. Advisory: the `related` rule in C-10 tells the model not to credit an `ASSERTS` that matches only a related obligation, but no C-06 rule coerces a violation.	R-38, C-06, C-10, C-16, C-17, K-16, I-015, E-57, T-83, T-84, T-89	requester / confirmed 2026-09-20 (reach — both directions, own-first, cap 8 — per `docs/proposals/PROPOSAL_v1` §5; the D-28b sub-decision — include in the triage `state` — per that proposal's §7 addendum; both the recommended options)
D-29	How the triage provider is configured	dedicated variables, `SPECCHECK_JEV_URL` / `SPECCHECK_JEV_MODEL` / `SPECCHECK_JEV_API_KEY` / `SPECCHECK_JEV_TIMEOUT` (C-17), read only when the triage pass runs	reusing `OPENROUTER_API_KEY`, as `tools/jev_client.py` already does — one fewer variable to set, but it couples the kernel's own configuration to a `tools/`-script naming choice and cannot be scoped or rotated independently of any other OpenRouter use	C-17, K-16, §10	requester / confirmed 2026-09-20 (`docs/proposals/PROPOSAL_v1` §5)

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-30	Whether `--jev-pre-triage` with `--judge-budget 0` complains	yes — one informational Note, `jev-pre-triage had no effect: --judge-budget is unlimited` (K-16). The combination pays Jev's latency and dollar cost for zero effect on the report, and a Note costs nothing while catching a likely mistake, consistent with this project's Notes-not-errors style	silently proceeding — no new Note text, but the one combination that buys nothing looks exactly like the one that buys something	K-16, K-12, E-59, T-89	requester / confirmed 2026-09-20 (`docs/proposals/PROPOSAL_v1` §5)
D-31	What a per-edge Jev failure during triage does	orders that edge first (as least confident) and records one Note (E-59); the run is never aborted. This session's own `deepseek-v4.1-flash` self-check run hit 110/656 (16.8%) real-provider coercions — 65 timeouts, 42 malformed responses — so "Jev could not answer" is expected to happen, and treating it as "assume this edge needs the real judge" is the safe default for a signal that sometimes fails	aborting the whole run on the first Jev failure — simpler and louder, but fragile against exactly the provider flakiness this project has already measured in a sibling judge model; it would also let a triage-only fault destroy a run that would otherwise have completed	E-59, K-16, T-89	requester / confirmed 2026-09-20 (`docs/proposals/PROPOSAL_v1` §5)
D-32	How `N%` rounds	up: $\lceil N/100 \times E \rceil$ of the E eligible edges (K-12), so any nonzero percentage issues at least one edge to the real judge	floor or round-to-nearest — floor silently zeroes the real judge at low percentages on a small edge count (1% of 50 edges floors to 0, a pure triage run that does not say so)	K-12, T-89	requester / confirmed 2026-09-20 (`docs/proposals/PROPOSAL_v1` §5)

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-33	`explain`'s output surface (v1.17)	**stdout only**: no `--out`, no durable file, no `schema_version` entry, no golden pair — the structured view of the same facts already exists as `speccheck.json`, and the narrative is what this command adds (C-18, §3.3)	a durable `EXPLAIN_<ID>.md` + `explain.json` pair matching the two-files-per-subcommand pattern — CI-friendlier, but it adds a durable-artifact contract, a golden pair to regress, and a third pair that must obey §3.1's temp-and-rename rule, while duplicating `speccheck.json`	R-40, C-18, §3.3, I-001, T-92	requester / confirmed 2026-09-20 (`docs/proposals/PROPOSAL_v1` §5, recommended branch)
D-34	Whether the trace carries the upward `impact` section (v1.17)	**include** it, at `--depth 1` by default: the forensic complement to the downward trace — *why this id is weak* beside *what a change to it might touch* — and it costs nothing, since C-12's `walk` and C-13's `reverify` already exist (C-18)	downward-only (the pure "why" trace), which would leave the reader to run `impact` separately for the other half	R-40, C-18, C-13, T-92	requester / confirmed 2026-09-20 (`docs/proposals/PROPOSAL_v1` §5, recommended branch)
D-35	Where `explain`'s facts come from (v1.17)	**recompute** from the same pipeline inputs `check` uses (`--spec`/`--src`/`--test`/`--results`, judge contract carried over, E-61): deterministic, byte-reproducible, able to show a live verdict under `--judge llm`, and no second input contract	a `--from speccheck.json` fast-path — instant, but it would report-as-input surface §0's boundary cautions against and cannot show a verdict a `none`/`mock` run never recorded	R-40, C-18, I-016, §0, T-92	requester / confirmed 2026-09-20 (`docs/proposals/PROPOSAL_v1` §5, recommended branch)
D-36	How many ids one `explain` invocation takes (v1.17)	**one id per invocation**, positional; an undeclared id is exit 2 per E-60 and a retired id is rendered, not an error; a shell loop drives many ids	accepting a list of ids and printing one block each — convenient, but it would need a second output format and its own exit-code rule (does one undeclared id fail the whole run?)	R-40, C-18, E-60, K-01, T-93	requester / confirmed 2026-09-20 (`docs/proposals/PROPOSAL_v1` §5, recommended branch)

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-37	How much a flag's help entry carries (v1.18)	**the full entry** — purpose, values, default and preconditions (C-19)	values and default only (shorter, but it leaves E-58's `N%` companion requirement and E-09's containment rule undocumented — the two failures §1 shows rejecting a well-formed command); values only (the literal ask, and the help still cannot be used to construct a judge invocation)	R-41, C-19, T-95, T-96	requester / confirmed 2026-09-20 (`docs/proposals/PROPOSAL_v1` §5, recommended branch)
D-38	Whether the exact help wording is normative (v1.18)	**content-required, wording free** — C-19 pins what each entry must carry and the metavar vocabulary; T-97's golden in `tests/data/help/` makes every wording change visible without a spec revision	exact strings normative in `SPEC.md` (maximum stability for consumers that parse the help, at a spec revision and version bump per typo, and ~50 lines of CLI prose duplicating §5.1)	R-41, C-19, T-97, §10	requester / confirmed 2026-09-20 (`docs/proposals/PROPOSAL_v1` §5, recommended branch)
D-39	How the help is guarded (v1.18)	**round-trip equality plus width-pinned goldens** — T-95 (the tokens are true), T-96 (every definition has help), T-97 (the bytes, at `COLUMNS=80`)	round-trip equality only (no golden churn on rewording, but a silently reduced entry then passes as long as its remaining tokens are correct)	R-41, C-19, T-95, T-96, T-97	requester / confirmed 2026-09-20 (`docs/proposals/PROPOSAL_v1` §5, recommended branch)
D-40	What the screens carry beyond the per-flag entries (v1.18)	**per-flag help plus an exit-code and summary-line epilog** on every `--help` (the codes are the second thing an operator or agent needs, and they are printed nowhere the CLI can be asked about; §5.4 is a stable K-01-pinned set)	strictly per-flag help (smallest surface, no fourth copy of §5.4); per-flag help plus a machine-readable `--help-json` (exact for a parsing agent, but a new interface, a new contract, and a second rendering of the same facts to keep equal)	R-41, C-19, §5.4, §5.1, T-96, T-97	requester / confirmed 2026-09-20 (`docs/proposals/PROPOSAL_v1` §5, recommended branch)

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-41	What the `environment:` block lists (v1.18)	**the eight `SPECCHECK_*` names with read condition, requiredness and default, plus one clause for `COLUMNS`** — the only variable that changes a `--help` run's own output, and the one T-97's golden pins	the eight names only (`COLUMNS` stays undocumented and T-97's fixed width unexplained); also documenting `install.sh`'s `~/.config/speccheck/judge.env` and its shell-rc `source` line (helpful for a first-time installer, but no kernel code reads that file — it is the installer's shell convenience)	R-41, C-19, T-97, T-98	requester / confirmed 2026-09-20 (`docs/proposals/PROPOSAL_v1` §5, recommended branch)
D-42	Where C-06's wrong variable name is fixed (v1.18)	**in this version's own fold** — `SPECCHECK_LLMMODEL` becomes `SPECCHECK_JUDGE_MODEL`, matching C-06's own request block two lines above it, C-09's block and `judge_llm.py`'s `ENV_MODEL`; the proposal's recommended branch (ride along in the still-uncommitted v1.16 fold) is no longer available because v1.16 and v1.17 have landed, so the fix lands here	leaving it (the name stays wrong and contradicts C-06's own block; T-98 still passes, because it binds the help to the code, not to that sentence)	R-41, C-19, C-06, C-09, T-98	requester / confirmed 2026-09-20 (`docs/proposals/PROPOSAL_v1` §5; branch adapted — see the v1.18 history row)
D-43	Proof manifest granularity (v1.19)	**per-declaration**: name + line + `checked`/`failed`, one entry per declaration (C-21) — it is the granularity the build tool emits and the per-id table needs; a single failed declaration still names itself	per-file check status — cheaper manifest, but a one-declaration failure fails every id the file tags, and T-99's pinned per-declaration states lose their referent	C-21, K-17, T-99	requester / confirmed v1.19 (`docs/proposals/PROPOSAL_v1` §5, recommended branch, 2026-09-24)

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-44	Whether `speccheck.json`'s `proof` key is conditional (v1.19)	**conditional**: the top-level `proof` object and the per-id `proof` field exist only when `--proof`/`--proof-results` were given; `schema_version` unchanged; absent-flag output byte-identical to v1.18 (I-018, T-100) — zero golden churn and zero consumer impact for the existing user base; a missing key unambiguously means "no proof input"	always-present `proof: null` — a stable key set, but every existing golden and every downstream consumer of the current shape must change, and `schema_version` must bump	C-07, I-018, T-100	requester / confirmed v1.19 (`docs/proposals/PROPOSAL_v1` §5, recommended branch, 2026-09-24)
D-45	Whether a `failed` proof trips `--strict` (v1.19)	**only when `--proof-results` was given and is readable**: the gate fails on a broken proof it was explicitly handed, while a missing manifest keeps the gate exactly as robust as v1.18 — a repo that never had a proof is never punished for the absence of one, and a proof failure is a defect of the proof repo, not the checked system	proof states strictly advisory in all modes — maximally decoupled, but "did the formal half break?" never answers through `--strict`	K-17, E-63, §5.4	requester / confirmed v1.19 (`docs/proposals/PROPOSAL_v1` §5, recommended branch, 2026-09-24)

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-49	Exact JSON shape of `proof` (v1.19, amended v1.19.1)	the confirming proposal pins conditional presence (D-44) and says the per-id field is "shaped like `tests`" but not its literal keys; this fold pins: per-id `proof` is an array of `{name, file, line, state, error}` (`state` ∈ `{checked, failed, unknown, stale}`; `error` non-null only when `failed`; an id with `--proof` given but no citation has `proof: []`; key omitted per-id only when `--proof` was not given); the top-level `proof` is `{"build": {…} | null}`, present whenever `--proof` **or** `--proof-results` was given (amended v1.19.1: `proof_from_spec`'s modeling of this row found a run given only `--proof-results` read the manifest and wrote no `proof` key at all under the original `--proof`-only gate — F-504, `docs/reviews/SPEC_MODEL_FINDINGS.md`), echoing C-21's `build` verbatim when `--proof-results` was given, else `null`	a fourth literal `"absent"` instead of an empty array — rejected: every other evidence array in this schema (`src`, `tests`) uses emptiness for "no citation", and a fifth token here would be the only exception; no top-level `proof` object at all, aggregate build info left to a walk of `ids` — rejected: `judge/judge_available` already set the precedent that aggregate run configuration belongs at the top; (v1.19.1) leaving the `--proof`-only gate and documenting `--proof-results` alone as an intentional no-op — rejected: a flag silently producing no observable effect at all is worse than widening its gate by one clause	C-07, C-21, K-17	requester / confirmed v1.19 (2026-09-24); amended v1.19.1 (2026-09-24, F-504)

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-50	Whether a table row's non-statement cells feed C-12 edges, and whether ranges expand (v1.21)	**edges only**: the other cells are read for edges (so an edge-case row's outcome column reaches `impact`) but `text`, `title`, the judged statement and the C-13 diff are unchanged; a range names the declared ids between its endpoints	widen `text` to every cell (changes every judged statement and C-13's changed set) / leave ranges as two endpoints	C-12, R-36, E-66, T-104, T-105	requester / confirm (letter from a regulated-domain user, 2026-10-14)
D-56	How JavaScript/TypeScript test files are delimited, and which runners' JUnit the join supports (v1.22)	by tokens inside `jsts.py` (comments, strings, templates and regex literals opaque; call parentheses balanced), so the kernel stays standard-library and needs no Node toolchain at check time; the join targets `vitest --reporter=junit`'s measured shape and jest-junit under the three-option configuration C-03 names (documented, not measured, until T-117 records a run; F-707)	a real parser (`typescript`, `@babel/parser` need Node where the checker runs); regex or line matching like the Swift adapter (a call's closing line is not recoverable from lines alone once strings and arrow bodies nest); also supporting jest-junit's default `classname`/`name` (it carries no file path, so nothing could be joined to a file)	R-104, C-03, C-24, C-04, E-74..E-76, T-113..T-117	requester / confirm
D-51	Whether foreign references are recognised by default (v1.23)	**by default**, no flag needed: the false-green in `docs/proposals/PROPOSAL_v1.23_namespaces_and_citation_maps.md` §1 A is silent, and this repository's own scan finds no code or test using the old reading (T-112)	only when `--namespace` is given (no behaviour change for anyone who does not know to ask)	R-101, C-22, I-019, T-107, T-112	requester / confirmed v1.23 (proposal §5, recommended branch, 2026-10-27)

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-52	What a bare ID means once `--namespace` is given	**still local**: no spec has to be rewritten to adopt the flag	a bare ID becomes foreign (stricter, but every existing citation must be rewritten)	R-102, E-68, T-107	requester / confirmed v1.23 (recommended branch)
D-53	The citation map's file format	**JSON** (matches C-21; test names contain commas and brackets)	CSV (hand-editable in a spreadsheet, brittle on those names)	R-103, C-23, E-70	requester / confirmed v1.23 (recommended branch)
D-54	Where a mapped citation is marked	**per-test in the JSON and Markdown, in the `explain` trace and in the summary line**: the one cost of the map is that the citation is not in the code, so the report says so where it is read	in the JSON only (quieter summary line, easier to miss)	R-103, C-23, E-72, T-110	requester / confirmed v1.23 (recommended branch)
D-55	Whether an unresolved map entry can fail the gate	**no**: it is a Note, and a stale entry lowers that id's status anyway, which `--strict` already gates	an unresolved entry fails `--strict` (stricter, but a renamed test would break a gate for a reason `--strict` already measures)	R-103, E-71, T-111	requester / confirmed v1.23 (recommended branch)
D-57	Whether a mapped citation is DECLARED (C-14) for the judge (v1.23)	**yes**: the map is the owner's statement of which test proves which id — the same intent a docstring citation states — so the judge's `declared` context is true (E-72); `mapped` is the separate label for where the statement lives	`declared: false`, treating it as INCIDENTAL (it understates a deliberate map and teaches the judge to distrust the one citation an owner wrote on purpose)	R-103, C-10, C-14, C-16, C-23, E-72, T-110	requester / confirmed v1.23 (review F-610, recommended branch); amended v1.23.1 (F-703, F-714): the rule is stated in C-14 and in C-10's field definition, and mapped citations are outside `declared_ratio`

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-58	How a non-strict run that would fail `--strict` is worded (v1.24)	**a distinct first word, `UNPROVEN`**, on exit `0` without `--strict` when R-15's conditions do not hold; the exit code is unchanged, since non-strict runs are used as a trend, not a gate	keep `CONFORMING` with a suffix naming the unproven count (no parser change, but `CONFORMING` still prints over 0 % passing); a threshold on the passing share (arbitrary, moves the false green); exit `1` (breaks the trend use and K-01's meanings)	R-105, R-21, R-14, C-08, T-44, T-118	requester / confirmed v1.24 (`docs/proposals/PROPOSAL_v1` §5, recommended branch, 2026-10-27)
D-59	What C-01 (a)'s "backtick span" is (v1.24)	**the CommonMark code span** (a run of N backticks closed by the next run of exactly N): it fixed the one silent truncation found in 14 real specs (`mdv*`'s R-24, ten copies) and 32 rows whose closing `|` leaked into the statement, and changed no fixture row	keep the single-backtick toggle and rely on E-66's Note (R-24 would stay cut until edited); split the way GFM does, inside code spans too (contradicts C-01's stated rule)	C-01, E-66, T-119, T-120	requester / confirmed v1.24 (`docs/proposals/PROPOSAL_v1` §5, recommended branch, 2026-10-27)
D-60	Whether ids that no edge touches are reported, and how (v1.24)	**one default-on Note, aggregated** (E-77): 0–2 per real spec measured, every hit a real gap (this document's T-42, `gpu-quicksort-revisited`'s E-17)	the same check behind an opt-in `--lint` flag (no golden churn, but the authors who need it would not run it); a Note per id that names no other id (51 of this document's 300, all correct leaf rows)	E-77, C-12, R-36, T-121	requester / confirmed v1.24 (`docs/proposals/PROPOSAL_v1` §5, recommended branch, 2026-10-27)
D-61	Whether the provenance manifest is written always or on request (v1.25)	**opt-in, `--manifest`**: users who file no records see nothing new, and I-021 can be stated exactly	always written (one flag fewer to forget in a regulated pipeline, but a third file in every `--out` and T-38's "only the two report files" rewritten for everyone)	R-106, C-25, I-001, I-021, T-122, T-128	requester / confirmed v1.25, 2026-11-05 (`docs/proposals/PROPOSAL_v1` §5)

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-62	Whether the manifest carries an operator and a wall-clock time (v1.25)	**neither**: `generated_at` only from `SOURCE_DATE_EPOCH`, so the manifest stays deterministic (I-020); attribution and signature belong to the user's quality system or a signed commit, where 21 CFR Part 11 can actually be met	an `--operator NAME` flag and a wall-clock timestamp (one self-contained file, but neither value is authenticated and the manifest stops reproducing)	R-106, C-25, I-020, E-85, T-123	requester / confirmed v1.25, 2026-11-05 (`docs/proposals/PROPOSAL_v1` §5)
D-63	How a requirement is bound to the hazards it controls (v1.25)	**a `*(controls: …)*` DECORATION marker** on the controlling row or heading: no new id family, hazard ids keep the risk file's own shape, and C-01 already parses that position for RECORDED	a hazard family `H-nn` in C-01 (first-class rows, at the cost of `by_family`'s six keys, every golden, and renumbering the user's hazards); widening C-22's FOREIGN grammar (a foreign reference yields no edge by design, and a wider id grammar would match prose such as `ISO:13485`)	R-108, C-01, C-02, C-27, E-78, E-80, T-126	requester / confirmed v1.25, 2026-11-05 (`docs/proposals/PROPOSAL_v1` §5)
D-64	Whether an `UNCONTROLLED` hazard fails `--strict` (v1.25)	**yes**: the risk file lists exactly the hazards allocated to this software, so one with no control row is the gap ISO 14971 asks to close, and a Note alone would let a `CONFORMING` gate hide it	a Note only (no new R-15 condition, but a green strict gate with a hazard nobody controls)	R-15, R-109, C-27, C-28, E-81, T-127	requester / confirmed v1.25, 2026-11-05 (`docs/proposals/PROPOSAL_v1` §5)
D-65	Whether `verify` re-hashes or re-runs (v1.25)	**re-hashes only**: simple and exact, works for `--judge llm` runs too, and an auditor who wants a re-run already has `check` followed by `verify`	`verify --recompute` for `none`/`mock` manifests (proves the reports follow from the inputs, but needs re-runnable argv in C-25 and a temporary `--out`)	R-107, C-26, T-124	requester / confirmed v1.25, 2026-11-05 (`docs/proposals/PROPOSAL_v1` §5)

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-46	Whether `judge_available` becomes boolean and total over the eligible-but-unissued case (v1.20)	**boolean, total**: `false` whenever at least one edge was judge-eligible and no call succeeded, whether every call failed (E-14) or none was issued (E-35) — the field is what R-28 gates on, and "no usable judge" is one operational fact however it arose; T-101 measures both budget forms	leave C-07's two clauses and patch only E-32's reason — smaller diff, but `speccheck.json` keeps a field the contract doesn't fully determine for this cell	C-07, E-32, E-35, R-28, T-101	requester / confirmed v1.20 (`docs/proposals/PROPOSAL_v1` §5, recommended branch, 2026-09-24)
D-47	Whether `--spec` becomes genuinely required on `explain` (v1.20)	**required**, synopsis brackets dropped — states existing behaviour (the flag table already said `Required.`; there was already no defined way to run `explain` without a spec) rather than inventing a new one; T-102 pins it	make `--spec` optional with a stated default — smaller for a quick `explain R-01`, but invents a fallback file the spec has never had and adds a second place the input spec is chosen	R-40, C-18, C-19, E-64, T-102	requester / confirmed v1.20 (`docs/proposals/PROPOSAL_v1` §5, recommended branch, 2026-09-24)
D-48	Whether exit-code precedence (usage fault vs. input-contract violation) is pinned as a normative `K` row (v1.20)	**pinned**, K-18, beside K-01's exit set — one sentence, checkable by T-103	leave §5.4 silent — two faithful implementations could then exit `2` or `3` on the same invocation	K-01, K-18, §5.4, T-103	requester / confirmed v1.20 (`docs/proposals/PROPOSAL_v1` §5, recommended branch, 2026-09-24)

None of the first fourteen was raised as a question before v1.1; each was decided and reviewed for precision only. That is the defect this section corrects: a specification can be implementation-grade and still not be what was asked for.

D-28 (and its sub-decision D-28b), E-57 and R-38 — the obligation-aware judge of `docs/proposals/PROPOSAL_v1.14_obligat` — are now declared by this v1.16 fold, discharging the reservation that held them in v1.15; D-29..D-32 and E-58..E-59 sit in the v1.15 Jev rows. The reservation that stood for the still-pending `docs/proposals/PROPOSAL_v1.17_explain_id.md` in v1.16 is discharged: v1.17 declares R-40, C-18, I-016, E-60, E-61 and T-92..T-94 for it. D-43..D-45 — the proof-parameter feature of `docs/proposals/PROPOSAL_v1.19_proof_parameter.md` — are now declared by this v1.19 fold, discharging the reservation that held R-100, C-20, C-21, I-018, K-17, E-62, E-63, T-99, T-100; D-49 — the JSON shape left unpinned by that proposal — was confirmed separately, 2026-09-24, then

amended the same day in the v1.19.1 fold once `proof_from_spec/`'s model of it found F-504. D-46..D-48 — the three determinacy fixes of `docs/proposals/PROPOSAL_v1.20_spec_determinacy.md` — are now declared by this v1.20 fold, discharging the reservation that held K-18, E-64, T-101..T-103. `docs/proposals/PROPOSAL_obligation_census.md` remains pending and reserves no ids. Every gap in a family not held by such a pending proposal is an error, and the proposal that owns a held id will declare it when it lands.

Revision history

Version	Change
v0.1	Initial draft from `one_sentence_prompt.md`, for `spec-review`.
v0.2	P0 of `docs/reviews/SPEC_REVIEW_REPORT.md` applied: F-001 T-family semantics (C-01, C-05 step 2b, R-25, E-25, T-53, T-48); F-002 scan exclusions and `--out` inside `--root` (C-03, §5.1, E-09, E-23, T-36); F-003 status is the sole input to `--strict` (R-15, §5.4, C-05, E-26, T-27, T-39); F-004 LLM body, response path, and normative instruction text (C-06, C-10, R-26, C-07 `judge_prompt_sha256`, T-33, T-54); F-005 parametrized JUnit names (C-04, C-07 `results`, E-24, T-52). P1 (F-006..F-013) and P2 (F-014..F-017) remain open.
v0.3	P1 of `docs/reviews/SPEC_REVIEW_REPORT.md` applied: F-006 longest-suffix classname join (C-04, E-27, T-58); F-007 `*TestCase` classes and `async def` (C-03, E-28, T-56); F-008 binary rule and no symlinks (C-03, K-02, K-03, E-29, E-30, T-13); F-009 row/cell/heading declaration grammar (C-01, E-31, T-55); F-010 one-line ASCII summary with regex, UTF-8 stdout (§5.1, R-29, T-44); F-011 packaged self-check fixture in a temp dir, I-001 exemption, §9.8 reference (§5.1, I-001, §10, T-43); F-012 `--strict --judge llm` requires an available judge and `unknown_rate` $\leq$ `--max-unknown` (R-28, K-11, C-07, §5.4, E-32, T-59); F-013 `speccheck:ignore` markers and the stated literal-citation limitation (§0, C-01, R-27, E-33, T-57). P2 (F-014..F-017) remains open.
v0.4	P2 of `docs/reviews/SPEC_REVIEW_REPORT.md` applied: F-014 logging default `ERROR`, Notes at `INFO`, nothing at `WARNING` (§5.3, T-42); F-015 in-memory render, temp-and-rename dual write, cleanup on failure, `--out` creation permitted (§3.1, §3.3, I-001, E-18, T-45); F-016 `judge_available` null/true/false rule, `by_family` always six keys, `by_status` seven in-scope statuses, ID cell struck (C-07, C-08, T-34, T-35); F-017 R-24 scoped to deterministic statuses plus recorded verdicts, T-49 per-run not pooled, K-08/T-51 recorded on a named reference machine via `tools/bench.py`, `SPEC.md` decoded UTF-8-with-replacement (R-24, T-49, K-08, T-51, §5.1, E-11, T-01). No findings open.
v0.5	`FINAL_SPEC_REVIEW_REPORT.md` P1 and P2 applied: F-101 `join_name` strips from the first `[`, T-52 corrected; F-102 `test*` methods (no underscore) in recognized classes (C-03, E-28, T-56); F-103 temporaries get a per-run nonce, are excluded from scans, and leftovers are deleted first (§3.1, C-03, E-34, T-36, T-45); F-104 R-10 passed-outcome clause, R-19 self-check exemption; F-105 empty `classname` join and `null unknown_rate` (C-04, K-11, E-36, T-52, T-59); F-106 JSON-then-Markdown rename order and the one permitted stale combination (§3.1, E-18, T-45); F-107 inner self-check exit expectation, `_selfcheck/` byte-identical to `fixtures/target/`, drift guard T-60 (§5.1, §10); F-108 fence definition and fallback `classname` (C-01, C-03, T-05); F-109 line-numbered `source` in the judge request, marker fixtures in data files (C-06, C-10, §9); F-110 `--judge-concurrency` and `--judge-budget` (K-06, K-12, §5.1, E-35, T-33, T-61). No findings open.

Version	Change
v1.0	All eleven findings of the independent `SPEC_v0.5_REVIEW_REPORT_by_QWEN.md` applied (cited as Q-nnn): Q-001 `verdict` key always present, `null` when unjudged, em dash in Markdown (C-07, C-08, E-37, T-34, T-35); Q-002 `notes` sorted by code point (C-07, E-38, T-34); Q-003 exact in-process `--self-check` invocation with `--results`, goldens under `fixtures/target/golden/` (§5.1, §10, T-43, T-46, T-60); Q-004 `unavailable` takes precedence over `unknown_rate` (C-07, §5.1, E-32, T-59); Q-005 `skipped` count added to the summary line and regex (§5.1, T-44); Q-006 "first declaration wins" removed (C-01); Q-007 symlinked command-line roots resolved and descended, descent-time symlinks skipped (C-03, E-09, E-30); Q-008 fence closing pinned — same marker character, no info string, no cross-type closure (C-01, T-05); Q-009 ratios as quantized Decimals emitted with exactly four decimals, K-11 compares the quantized value (C-07, K-11, T-34); Q-010 K-05 timeout spans the full request lifecycle, K-12 deadline semantics with in-flight completion (K-05, K-12, E-35, T-61); Q-011 `name` is `""` in JSON, `(file)` is Markdown-only, R-13 concerns data (C-07, C-08, E-13, T-35). No findings open.
v1.1	Added §12, *Open questions and decisions to confirm*: fourteen decisions the author took by default on the requester's behalf (stack, results contract, citation rule, T-family option, mixed-verdict strict rule, judge question and parameters, instruction text, input limits, endpoint shape, fixture layout, report names, test delimitation, reference machine), each with its default, the alternatives rejected, the IDs it affects, and a `confirm` / `open` status. No normative row changed. The `spec-writing` skill now requires this section.
v1.2	D-07 partly resolved by the first T-49 runs: `max_tokens` raised from 400 to 4000 in the judge request body (C-06), the wire-format test updated to match (T-33), and the D-07 row records the evidence. Thinking models truncated at 400 before emitting the verdict JSON; at 4000 `qwen3:8b` and `gemma4:latest` pass T-49 three runs out of three. No other normative row changed.
v1.3	Judge-stage progress indicator, requested on 2026-09-13 because `--judge llm` runs take minutes: R-30 (requirement), C-11 (the one-line ASCII format, draw/erase byte sequences, regex, k and ETA formulas), K-13 (redraw cadence, 20 cells, single erase before the report stage), E-39 (no bytes when not a TTY, at DEBUG, under `--progress never`, or with `--judge none|mock`), E-40 (erase on interrupt or failure), `--progress auto|always|never` in §5.1, the §5.3 quiet-by-default exception, T-62/T-63, §11 rows, D-15. The indicator is written to the raw stderr stream, not the logger, and is erased at the end of the stage, so stdout, both reports, the exit code, and what remains on stderr at exit are unchanged.
v1.4	All ten findings of the v1.3 `docs/reviews/SPEC_REVIEW_REPORT.md` applied. P1: F-201 the logger is silent while the indicator is displayed (C-11, K-13, §5.3, T-62); F-202 one clock origin t_0 = first draw, K-12 keeps its own (C-11, K-13, T-62); F-203 coalesced cadence — within 100 ms, $\geq$ 1/s in flight, $\leq$ 10/s — and atomic serialized writes (K-13, T-62); F-204 E-41 interrupt rule: exit `3`, message `interrupted`, temporaries and renamed reports removed (E-40, E-41, §3.1, §5.4, T-63, T-64, §11, D-16); F-206 C-07 example uses `0.2000/0.0000` and `max_unknown` is emitted quantized (C-07). P2: F-205 no terminal escapes — draws padded to the widest line so far, erase is `\r` + spaces + `\r` (C-11, T-62, D-15); F-207 C-08 header parenthetical omitted when `judge_available` is null (C-08, T-35); F-208 T-46 cited by R-01..R-15 and T-60 by R-18/I-001 in §11; F-209 I-005 vacuous clause replaced; F-210 T-49 names `fixtures/target/golden/judge_labels.json` and `tools/eval_judge.py`.
v1.5	E-41 tightened from "in-flight requests are not awaited beyond the K-05 timeout" to "abandoned; exit within 1 s" after the first interrupt of a real `--judge llm` run against a local model appeared to be ignored: the main thread was blocked in an untimed future wait (not SIGINT-interruptible on macOS CPython) and then waited for the in-flight requests. T-64 gains the timing case; §11 E-41 row names the mechanism. No other row changed.

Version	Change
v1.6	Swift adapter and two small grammar changes, requested 2026-09-17 after the MonteCarloPi Swift build ran the checker and got `62 unverified; 43 dangling` purely from tool limits: R-31 Swift Testing / XCTest test-case delimiting by lines with doc-comment-inclusive spans and SwiftPM-shaped classnames (C-03, E-42, E-43, D-17, D-18, T-65..T-67), a second `join_name` step that strips a Swift signature (C-04, E-45, D-19, T-68), Swift assertion tokens for the mock judge (C-06, T-69), R-32 decoration after the bold ID in a declaring first cell (C-01, E-44, T-70), a Swift golden fixture (T-71), O-2 and the §0 non-goal updated.
v1.7	A heading-declared ID's statement is its title plus its section body (`docs/proposals/PROPOSAL_v1.7_heading_bodies.md`, 2026-09-17): two LLM judges over the same 130 MonteCarloPi edges disagreed almost entirely on `###`-declared contracts because both were handed a title (`Data structures`, `` `EstimationWorker` (an `actor`) ``) and never the pinned API beneath it — one guessed generously, the other refused, and neither had the contract. R-33; C-01 (b) HEADING LINE / SECTION BODY grammar, fenced blocks included (D-20), table declarations unchanged; C-02 `SpecId.title`; C-06 statement passthrough; C-07 `title` key and `schema_version "1.1"`; C-08 renders `title`, so the Markdown report is byte-identical; C-10 gains the any-clause rule (new `judge_prompt_sha256`); K-14 16,384-byte cap with marker line and Note (the proposal's 8,192 would have truncated this document's C-03); E-46, E-47; T-72..T-74; §11 rows; D-20. The mock judge, statuses, metrics, and exit codes are unchanged; `--judge none` output differs only in `speccheck.json`.
v1.8	All seven findings of the v1.7 `docs/reviews/SPEC_REVIEW_REPORT.md` applied. P1: F-301 line model for `SPEC.md` — split on `\n`, trailing `\r` removed, BLANK := empty or whitespace-only, body lines re-joined with `\n` (C-01, C-02, I-002, T-72); F-302 T-72 and T-73 restated as properties of the current output, the one-time golden diff moved to `SPEC_BUILD_REPORT.md`; F-303 T-49 counts only runs under the current `judge_prompt_sha256`, D-08 re-opened, D-06 and §0 name the clause granularity. P2: F-304 ATX headings only, at most three leading spaces, bare `###` is a heading with an empty title, trailing `#` run stripped from the title, the declaring line is a HEADING LINE (C-01, T-72); F-305 empty title with a body → statement is the body alone (C-01, C-02, E-46, T-72); F-306 C-09 cited by T-33 and T-40; F-307 revision table sorted ascending, Status bullet shortened, `title` in the §1 Extractor row. No behaviour the v1.7 build would produce differently was changed; what changed is what the spec pins.
v1.9	Clause-grounded verdicts and a body in the judge-evaluation fixture (`docs/proposals/PROPOSAL_v1.9_clause_grounding.md`, 2026-09-18; D-21 confirmed). Evidence: with bodies in the statement, `gpt-4o-mini` graded long contracts by their gist — 13 of 18 stable downgrades on the mdv tree were tests asserting a body clause nearly verbatim — while `gemini-3.8-flash` located the clauses and found real gaps (`mdv:E-19`, `mdv:T-34`, this document's R-10); both passed the nine-label T-49 at 1.000. R-34; C-06 `Verdict.clause`, K-15 LOCATED rule, E-48 (`judge: unlocated clause`), E-49 (clause blanked for `UNRELATED`/`UNKNOWN`), I-005 grounded on both sides, mock clause = prefix of the statement (R-22); C-07 `clause` key and `schema_version "1.2"`; C-08 §8 Clause column; C-10 question reworded clause-first, `clause` in the reply (new `judge_prompt_sha256`); T-75; golden fixture gains a heading-declared contract with a multi-clause body 2,048 bytes and six labeled tests, label set 20 entries (T-46, T-76, T-49); §11 rows; D-08 re-opened and made a model row; D-21; D-22 opened (recorded-not-gating T ids under an honest judge). Statuses, metrics, exit codes, and `--judge none`/`mock` Markdown output unchanged; `--judge mock` JSON gains the `clause` key.

Version	Change
v1.10	D-22 settled: RECORDED tests. A T row whose ID cell carries `*(recorded)*` as the first decoration token declares a recorded id (C-01, C-02 `recorded`); its status is computed by C-05 steps 1–4 like any T id (E-51) but its edges are never judged (step 5, I-010) and are not eligible edges for C-11; the marker on a non-T id is ignored with a Note (E-50); JSON gains `recorded` per id (`schema_version "1.3"`), the Markdown ID cell reads `T-48 (recorded)` (C-07, C-08); R-35, T-77; T-48, T-49 and T-51 marked in this document. Motivation: under an honest judge the presence checks that keep those ids cited are `EXECUTES_ONLY` by construction, so `--strict --judge llm` was red on this repository for a reason the spec intended but had left implicit.
v1.11	All seven findings of the v1.10 `docs/reviews/SPEC_REVIEW_REPORT.md` applied. P0: F-401 `schema_version` stated once (C-07, §3.3), T-73/T-75/T-77 refer to "the C-07 value". P1: F-402 the C-06 validation list is numbered, first-match, with a non-string `clause` treated as absent and every coerced `UNKNOWN` recording `clause ""` (C-06, E-16, E-49, T-75); F-403 RECORDED named in the null-`verdict` condition, E-37 owning it (E-37, C-07, T-34); F-404 one marker rule after C-01 (b), the heading title strips it, `~~T-48~~ (recorded)` pinned (C-01, C-08, T-77); F-405 `judge_strength` over `PASSING  RECORDED`, null case stated, T-77 arithmetic (C-07). P2: F-406 K-15 trims the clause (T-75 sub-case); F-407 "the current C-10 text", D-08 measurement moved to its status cell, Status bullet trimmed. No behaviour the build would otherwise get wrong changed; what changed is what the spec pins.
v1.12	`--src` and `--tests` accept a comma-separated **list of files and/or directories**, requested 2026-09-19 after these flags accepted only directories (D-23): each occurrence is split on `,`, every segment whitespace-trimmed and empty segments dropped, paths resolved inside `--root` and deduplicated by resolved path (a file reached by more than one element is scanned and cites exactly once, I-012; emits are in ascending-path order; a file's recorded `--src`/`--tests` root — and thus a directly-named `.swift` file's `MODULE`, D-18 — is the first-seen covering element). Each directory element is descended and each regular file scanned as one, with every per-file filter applied identically; a path inside `--root` that exists as neither directory nor file is a usage error, `--<flag>: no such file or directory: <segment>`, replacing the former "non-directory → usage error" (E-52, exit 2). The change is input-acceptance only: two inputs that name the same physical file set produce byte-identical reports (I-002), so the JSON/Markdown schemas, the summary regex, the statuses, the metrics, and the exit codes are unchanged and `schema_version` stays at the C-07 value. R-03 and R-04 reworded for the `--src`/`--tests` list; a C-03 `PATHS` rule and the directly-named-file `MODULE`; §5.1 synopsis, the `--src`/`--tests` rows, and a PATHS note; E-09 and E-30 generalized to PATHS elements; I-012 and E-52 added; T-78; §11 rows; D-18 extended; D-23 confirmed.

Version	Change
v1.13	Spec-internal edges and the `impact` subcommand (`docs/proposals/PROPOSAL_v1.13_impact.md`, 2026-09-19; D-24 and D-25 confirmed). Evidence: in v1.12 of this document 134 of 203 live ids name another id in their statement — 186 `depends_on` and 240 `verifies` edges — and the §12 *Affects* column holds 110 more; following `depends_on` transitively saturates (from 77 of 125 obligations the closure reaches at least 40), so the direct set is the signal and `--depth` defaults to 1 until the T-82 backtest says otherwise. Changes: C-01 (c) decision rows — a declaration-only family D found by the table whose header has an *Affects* cell; C-02 `Decision`, `Edge`, `SpecIndex.decisions/edges`; C-12 the edge rules (statement tokens → `depends_on/verifies` with normalized direction, *Affects* tokens → `affects`; undeclared targets are Notes; retired targets flagged, E-55; total order); C-07 `decisions` and `edges` after `ids`, `schema_version "1.4"`, nothing else in the JSON or the Markdown report changed; C-13 `impact` — `--changed IDS` or `--against FILE` (diff reasons pinned), breadth-first reverse walk with `via` and `--depth` (default 1, 0 unbounded), REVERIFY, RECITE, TEST_CASES, `impact.json "1.0"`, `IMPACT_REPORT.md`, a summary line with its regex; §3.1 the `impact` pipeline and its temporaries; §3.3 two artifacts; C-03 exclusions extended; §5.1 synopsis and four rows; §5.4 `impact` codes; §0 two non-goals amended; §1 the Impact walker; R-36, R-37; I-001 and I-002 extended, I-013; E-02 extended, E-53..E-55; §9.12 T-79..T-82 (T-82 recorded: the backtest on two ranges of `main`); §10 tools and goldens; §11 rows; §12 read by machine, D-24, D-25.
v1.14	Declared vs. incidental citations (`docs/proposals/PROPOSAL_v1.15_declared_vs_incidental_citations.md`, 2026-09-20; D-26 and D-27 confirmed). Evidence: `JUDGE_CROSSCHECK_REPORT.md` §2b — a `check --judge llm` run of this document's own tree with `gpt-4o-mini`, cross-checked edge-by-edge against an independent second model, found 153/613 genuine conflicts (both models committed and disagreed), 60 concentrated in five ids reused as generic fixture data (`R-01` 36/57, `C-01` 7/34, `R-07` 6/7, `C-06` 6/7, `C-03` 5/12) but 93/496 (18.8%) still conflicting once those are excluded; three non-`R-01` conflicts read in full all showed a test citing an id as placeholder data while proving something else, with neither its own docstring nor any comment naming that id. Changes: C-14 the DECLARED/INCIDENTAL rule — a citation is DECLARED when a citation line of it lies inside the citing test case's own docstring or is a whole-line comment (Python: `#`; Swift: the adapter's existing doc-comment-line recognition, reused rather than duplicated — D-27), INCIDENTAL otherwise, always INCIDENTAL for a `"src"`-kind or file-level citation (E-56); C-03 `Citation.declared`; C-15 `JudgeRequest.declared` (read-only context; no coercion rule reads it — D-26) and `C-10`'s instruction text gains the field and a skepticism rule for `declared: false` (advisory only, D-26); C-16 / C-07 `tests[].declared` and `metrics.declared_ratio` after `unknown_rate`, present under every `--judge` mode including `none`, `schema_version "1.5"`; §9's preamble sentence now names what `declared` mechanically checks; I-014 extends I-002's determinism guarantee; R-39; §9.2 T-85; §9.6 T-88; §9.8 T-86; §9.11 T-87 (recorded: re-running the three real conflicting edges under the new `C-10` text); §11 rows; §12 D-26, D-27. Statuses, the mock judge, `--judge` coercion rules (K-15, E-48, E-49), I-004, I-005, and the Markdown report are unchanged.

Version	Change
v1.15	Jev pre-triage (`docs/proposals/PROPOSAL_v1.16_jev_pre_triage.md`, 2026-09-20; D-29..D-32 confirmed). Evidence: a real `check --judge llm --strict` run of this document's own tree with `openai/gpt-4o-mini`, cross-checked edge-by-edge against `typesafe/jev-1.13` the way `JUDGE_CROSSCHECK_REPORT.md` already does — 656 judge-eligible edges, 582 the real judge committed to — shows Jev's top-choice probability is a monotonic triage signal (agreement with the real judge $279/295 = 94.58\%$ at $p \geq 0.95$, $62/79 = 78.48\%$ at 0.80–0.95, $53/97 = 54.64\%$ at 0.60–0.80, $53/111 = 47.75\%$ below 0.60), and that 51.5% of all eligible edges already sit at ≥ 0.95 (41.2% at ≥ 0.99) — headroom a truncated run was losing to declaration order. Changes: K-16 the triage pass (`--jev-pre-triage`: one task per eligible edge, C-17's request shape, at `--judge-concurrency`, before any real-judge request; ascending-confidence issue order, ties by C-07's id order; a per-edge failure first, with a Note; the D-30 Note under an unlimited budget); C-17 the Jev provider contract (four `SPECCHECK_JEV_*` variables, the `{model, state, questions}` body with the pinned `state` template and criteria, the `answers.verdict` response path, $p(e) := \max$ probabilities, the USABLE rule, and the ordering); K-12 amended — `--judge-budget` grows the `N%` form ($\lceil N/100 \times E \rceil$ of E eligible edges, least-confident first; `0%` a pure triage dry run, `100%` all edges), the `SECONDS` form unchanged; I-015 Jev stays advisory (no `Verdict`, no report field, I-010 unchanged); E-58 (`N%` without a running triage → exit 2), E-59 (per-edge Jev failure → first + Note, `judge_available`/`unknown_rate` untouched); E-35 and E-36 extended to both budget forms and to the triage pass; §3.1's stage table and §3.2 gain the pass, C-11's clock rule covers the `N%` form, §5.1 gains the flag and the `SECONDS|N%` grammar, §10 names C-17, O-1 covers the provider; T-89 (triage + `N%` + C-17 request shape), T-90 (E-58/E-21 messages), T-91 *(recorded)* (the §1 calibration re-measured and its buckets recorded); §11 rows; §12 D-29..D-32. `speccheck.json`, `schema_version "1.5"`, every status, the mock judge, the C-10 text, and both reports are unchanged.

Version	Change
v1.16	Obligation-aware judge (`docs/proposals/PROPOSAL_v1.14_obligation_aware_judge.md`, 2026-09-20; D-28 and its sub-decision D-28b confirmed). Evidence: `JUDGE_CROSSCHECK_REPORT.md` §2b flagged that a test can cite an id as placeholder data while asserting something about a *neighbouring* id it names; a statement's C-12 `depends_on` neighbourhood — the ids it names and that name it — is the context a judge needs to tell its own clause apart from a neighbour's, at a cost of roughly doubling the per-edge judge input, which is cents against the per-edge real judge and the one pass a `--judge llm` run already makes. Changes: R-38 the request carries `related` — the C-12 `depends_on` neighbourhood, both directions, the statement's own references first, at most eight total, each a whitespace-collapsed title of at most 160 characters, a retired neighbour tagged `(retired)` (E-57), the empty list when the id has no neighbour; C-06 `JudgeRequest` gains `related` as a request-side, non-report field (no C-06 coercion rule reads it, so C-16, C-07 and C-08 are unchanged); C-10 gains the field and its rule (an assertion corresponding only to a related obligation is not evidence this statement holds; advisory, no model coercion — D-28's stance extends D-26's); C-17's triage `state` gains a `Related obligations:` section built from the same neighbourhood (D-28b — include, not strip, so Jev ranks the exact request the real judge gets, at roughly double the per-edge triage input); E-57 a retired `related` neighbour keeps its `(retired)` title; §11 rows; §12 D-28 (with D-28b). `related` is never a status, a metric, or a JSON field, so `speccheck.json`, `schema_version "1.5"`, the Markdown report, every status, the mock judge, and both reports are unchanged; the C-10 text changed, so `judge_prompt_sha256` is re-pinned by the build (F-004). §9 gains T-83 and T-84 *(recorded)*; T-89 (triage) and T-76 (the label fixture, now 37 entries, 8 downgraded on an adjacent edge — the build of this version found the row's `UNRELATED` too narrow: C-10's vocabulary grades a test that runs the statement's code and asserts a neighbour's fact `EXECUTES_ONLY`, F-1) are amended for the `related` section and the adjacent subset. The reservation that held D-28/E-57/R-38 in v1.15 is discharged (§12 note).
v1.17	`speccheck explain <ID>` (`docs/proposals/PROPOSAL_v1.17_explain_id.md`, 2026-09-20; D-33..D-36 confirmed, all four on the proposal's recommended branch). Evidence: this is a capability addition, not a defect fix — the tool already computes every fact a reader wants about one id, but a human assembles the trail by hand across `speccheck.json`, the cited source lines and an `impact` run (three structured artifacts plus two greps); §0's own promise is a path to the evidence, and the path was not rendered as one thing. Changes: R-40 the third subcommand runs `check`'s stages and renders one id; C-18 the stdout contract (section order, headings, the C-05 reason per status, the outcome/verdict line, the `clause:`/`rationale:` lines, the `impact (<depth>):` section from C-12's `walk` and C-13's `reverify`, and the exit codes `0`/`2`/`3`); I-016 `explain` is additive and read-only (no report, no new judge call, statuses agree with `check`); E-60 an undeclared id is exit `2` while a declared-but-uncited one renders its reason and exits `0`; E-61 `--judge` carries over `check`'s contract verbatim; §3.1 gains the surface's paragraph, §3.3 records that it writes no durable artifact (D-33), §5.1 gains the synopsis line and the `explain`/`--depth` rows, §5.4 its exit codes; §9.13 gains T-92 (the golden trace and its byte-stability), T-93 (E-60, the retired id, the uncited id) and T-94 *(recorded)* (the live-LLM trace); §11 rows; §12 D-33..D-36. No status, verdict, citation, metric, `schema_version`, `check`/`impact` report or golden moves.

Version	Change
v1.18	The CLI documents its own parameters and environment (`docs/proposals/PROPOSAL_v1.18_cli_help_contract.md`, 2026-09-20; D-37..D-42 confirmed on the recommended branches — D-42's branch adapted, because its preferred landing (the uncommitted v1.16 fold) no longer existed once v1.16 and v1.17 shipped). Evidence: `check --help` rendered 755 bytes of flag names and argparse dest names with no help text at all (14 `check` flags and 9 `impact` flags, 0 with help; `grep -c "choices=" src/speccheck/cli.py` = 0), the only machine-readable statement of a flag's values was its usage error, the environment was stated on five surfaces and one of them was wrong (C-06's `SPECCHECK_LLMMODEL`, read by no code), and the one surface that *was* documented had already drifted — six README statements still said `DIR` two versions after D-23 made `--src`/`--tests` PATHS. Changes: R-41 every flag documents purpose, values, default and preconditions, plus the environment, the exit codes and the summary line; C-19 the entry order, the metavar vocabulary, the `environment:` block and the exit-code epilog; I-017 `--help` is inert; T-95 the help usage-error token equality and the acceptance round-trip; T-96 every definition carries help and a C-19 metavar, and the screens render inertly; T-97 the byte-pinned goldens at `COLUMNS=80`; T-98 the environment block is bound to the code's own `SPECCHECK_*` literals; §5.1 and §5.4 gain their pointers; §9.14 the new test group; §10 the help goldens; §11 rows; §12 D-37..D-42; C-06's variable name corrected. No accepted value, default, exit code, report byte or golden moves.
v1.19	Formal-verification half of the conformance graph (`docs/proposals/PROPOSAL_v1.19_proof_parameter.md`, 2026-09-23; D-43..D-45 confirmed on the recommended branches, 2026-09-24). Evidence: in `hello_world_deepseek`, a sibling Lean 4 proof (66 kernel-checked declarations, generalized during this build into `speccheck/tools/proof_evidence.py`) discharges a module-level half of that project's spec for all inputs and explicitly defers the process-level half to pytest (e.g. `hello:R-09` → `rowInterrupt`, `rowUsageInterrupt`, `runSigint`, `guardKi`, deferred to `T-20`), yet none of it is visible to speccheck: `speccheck.json` carries exactly 16 top-level keys, zero of them `proof`, and the machine graph has only `depends_on`/`verifies` edges — the system is five-way, the machine graph four-way. Changes: R-100 the `--proof PATHS` scan; C-20 the lean adapter (declaration-head split, bold-span doc-comment tags, directory recursion, E-62); C-21 the proof manifest read by `--proof-results FILE` (`build/theorems`/optional `deferred_to_pytest`, three keys read and the rest ignored) and its join to citations by name/file/line (K-17: stale/unknown/failed states); I-018 byte-identity under flag absence (T-100); E-63 a proof state never changes a status or `impact`'s blast radius (C-05, R-06 untouched); E-65 an unreadable or malformed `--proof-results` file (the `--results`/E-05 pattern); E-09 and E-54 extended to the two new flags; `speccheck.json` gains a per-id `proof` array and a top-level `proof` object only when `--proof` is given, `schema_version` unchanged (D-44), the exact key shapes pinned by D-49 (confirmed separately, 2026-09-24, since the confirming proposal left them open); `--strict` fails on a `failed` proof state only when `--proof-results` was given and is readable (D-45); both flags are `check`-only — `impact`/`explain` reject them, and `explain`'s C-18 mechanism is untouched; §5.1 synopsis and two flag rows; §9.15 T-99 (the extended §9.8 fixture plus a live cross-repository run over `hello_world_deepseek`'s own evidence) and T-100; §10 the lean adapter, the fixture's new files, `tools/proof_evidence.py`; §11 rows; §12 D-43..D-45, D-49. No status, verdict, citation, existing metric, exit code, or byte of an absent-flag run moves.

Version	Change
v1.19.1	D-49 amended: the top-level `proof` key is gated on `--proof` **or** `--proof-results`, not `--proof` alone (`docs/reviews/SPEC_MODEL_FINDINGS.md` F-504). Evidence: `proof_from_spec/`'s formal model of this session's own v1.19 fold — before any implementation existed to model instead — found a run given only `--proof-results` reads the manifest (C-21's read is unconditional) but, under the original gate, writes the result nowhere; `f504ManifestReadButKeyAbsent` exhibits the case. Fix applied same-session, before `spec-build` starts on v1.19, rather than shipped as a defect and fixed later: C-07's proof-field rules and the JSON example's comment reworded; D-49 amended with the corrected condition and the rejected no-op alternative. Per-id `proof` (still gated on `--proof` alone), every other v1.19 key, every status, exit code, and existing golden byte is unchanged — the only observable difference is that `--proof-results` given alone now writes a `proof.build` object where it previously wrote nothing.
v1.20	Determinacy of three under-specified behaviours (`docs/proposals/PROPOSAL_v1.20_spec_determinacy.md`, 2026-09-24; D-46..D-48 confirmed on the recommended branches). Evidence: `proof_from_spec/`'s formal model of `SPEC.md` v1.18 (built before v1.19 existed) could not render a total function over three enumerated inputs — `judge_available` in the eligible-but-unissued case, `explain` invoked without `--spec`, and one invocation triggering both an exit-2 and an exit-3 condition — each closed only by returning `none`; three kernel-checked witnesses (`f502JudgeAvailableBudgetSilent`, `f501ExplainAbsentSpecSilent`, `f503ExitPrecedenceDiffer`) exhibit the gaps (F-501, F-502, F-503). Changes: C-07 `judge_available` restated over the eligibility/issuance census — boolean, `false` whenever an edge was eligible and no call succeeded (E-14 or E-35), never `null` under `mock`/`llm`; E-32 extended so an E-35-caused unavailability also records `strict_judge_failure: "unavailable"`; §5.1's `explain` synopsis drops the misleading `[--spec SPEC.md]` brackets to match its own flag table (`Required.`); E-64 the resulting usage error, `explain: --spec is required`; K-18 states usage-fault precedence over an input-contract violation when an invocation could trigger both, closing the one case §5.4's table left silent; T-101 (both budget forms, `judge_available`/`strict_judge_failure` under each), T-102 (`explain`'s missing-`--spec` message, the corrected help golden), T-103 (both fault kinds in one invocation, and split across two, to show the precedence is observed not merely asserted); §11 rows; §12 D-46..D-48. No status, verdict, citation, or existing metric moves; the golden fixtures are byte-identical (`--self-check`/§9.8 run `--judge mock` with no budget, so Part A's changed case never appears in them) — the one behavioural change is the exit reason on an already-red `--strict --judge llm` run whose eligible edges were all budget-skipped, `null` → `"unavailable"`.
v1.21	Four extractor/CLI gaps reported from a regulated-domain trial: C-12 expands ranges (`R-07..R-10`) and reads a table row's non-statement cells for edges (D-50); E-66 Notes a row cut at an unescaped `|` inside `$..$`; E-09's message names the resolved path of a symlink path. No status, verdict, metric, `text` or `title` moves.
v1.22	JavaScript/TypeScript adapter (R-104, D-56; requested in a regulated-domain trial, 2026-10-14, where TypeScript tests could reach `UNVERIFIED` and no further): `.ts .tsx .mts .cts .js .jsx .mjs .cjs` files under `--tests` are delimited into `it`/`test` cases named by their `describe` chain (C-03); C-04 reads a path classname as dotted, joins cases under `join_name(case.name)`, and matches `it.each` titles by placeholder pattern; E-74..E-76, T-113..T-116; T-12 now uses `.rb`. The join shape was measured with `vitest run --reporter=junit` 2.1.8. No status, verdict or metric moves for a project with no such file.

Version	Change
v1.23	Namespaced ids and a citation map (`docs/proposals/PROPOSAL_v1.23_namespaces_and_citation_map.md`, 2026-10-27; D-51..D-55 confirmed on the recommended branches, same day). Evidence: a test docstring naming `platform:C-03` made the local `C-03` PASSING (1/1) and an undeclared foreign id failed `--strict` as dangling (reproduced); a regulated-domain trial retrofitted ~1,100 ids with an out-of-tool injector. R-101..R-103, C-22, C-23, I-019, K-19, E-67..E-73, T-107..T-112; `--namespace` and `--citation-map` on `check`, `impact` and `explain`; the summary line gains an optional `; citations: <M> mapped` suffix. **Folded with three changes to the proposal**, each found reviewing the fold (and, from the review's open items, D-57 and a Note that names the first three foreign tokens): (a) `impact` and `explain` take both flags (the proposal covered `check` only, which would leave a retrofitted project's `impact` re-cite list empty); (b) a label shaped like an ID (`R-01:R-02`) is never a namespace; (c) the summary suffix is pinned in the T-44 regex. No status, verdict or metric moves for a project using neither flag and containing no `ns:`-prefixed token (I-019).
v1.23.1	The first full review since v1.10 (`docs/reviews/SPEC_REVIEW_REPORT.md`, F-701..F-717, 2026-10-27), folded before the v1.23 build. **Behaviour (all still unbuilt):** C-06's pinned request body restored — v1.16 had left a prose paragraph inside it, dropped the user-message opener and added a sentence contradicting `"messages": [`; the code was right (F-701). The `--citation-map` and `--proof-results` files are excluded from every scan (C-03, F-702). A mapped citation is DECLARED by a third C-14 rule, C-10's `declared` definition says so (so `judge_prompt.md` and `judge_prompt_sha256` change at the build and the recorded T-49 runs are void, F-303), and mapped citations are outside `declared_ratio`, now written as a formula over R-39's unchanged population of non-file-level cases (C-07, C-16, D-57; F-703, F-714). Mapped dangling/stale entries carry `mapped: true` (F-704). C-14 covers JS/TS comment-only lines (F-705, D-27). The mock judge recognises `expect(/expect.` (F-706; no golden changes). jest-junit needs `ancestorSeparator: " > "` and is documented, not claimed, until the new recorded T-117 (F-707, D-56). The JS/TS tokenizer has a pinned regex-versus-division rule (F-708). K-18 counts every file flag's readability as a usage fault decided before parsing (F-709). C-18's reasons name C-05's own steps and `<outcome>` gains `error`; T-92/T-93 strings change (F-711). FOREIGN recognition has no length cap; the 32-character limit is the `--namespace` value's (R-101, R-102, C-22; F-712). Foreign-token test inputs live under `tests/data/` (§9 preamble, F-713). A foreign range endpoint names nothing; a map entry cannot address a Python method by class (C-12, C-23; F-716). **Lists and wording:** §5.4's exit-3 lists (F-710); §5.1's `impact`/`explain`/`--root`/`--namespace` rows and E-09, E-68 (F-715); the split §5.1 and §9 tables joined, §2/§8/§11 rows sorted by id, O-2, C-12, E-14 and the Sources bullet brought up to date, proposal-local "Part A/B/C" labels replaced (F-717); C-03's copy of the DECLARED rule replaced by a pointer to C-14, which keeps C-03 (16.2 kB) under K-14's cap, and K-14's stale 8.4 kB reference point corrected. New id: T-117.

Version	Change
v1.24	Three suggestions from a regulated-domain debrief (`docs/proposals/PROPOSAL_v1.24_unproven_cut_rows_isolated_ids.md`, 2026-10-27; D-58..D-60 confirmed on the recommended branches, same day). R-105: a non-strict exit-0 run that `--strict` would reject prints `UNPROVEN`, not `CONFORMING` (§5.1 and its T-44 regex, §5.4); the exit code is unchanged. C-01 (a): a backtick span is the CommonMark code span (D-59), which repairs `mdv*`'s R-24 (a statement cut in half) and 32 rows whose closing `|` leaked into the last cell, this document's T-05 among them. E-66 also Notes a declared row with more cells than its TABLE's header. E-77 and C-12: one Note names the declared ids no edge touches (D-60). T-42 now cites R-17, the id it verifies (it named only §5.3 and would have been the one isolated id here). Not in the proposal, found while folding it: C-03 had crossed K-14's 16,384-byte cap at `b6949a1` (v1.23.1's jest measurement), so the released v1.23.1 truncated C-03's statement with a K-14 Note that no gate reads; the JS/TS adapter moved verbatim into a new C-24 (C-03 is now 11.9 kB), R-104, C-14, T-113..T-117, D-56 and §11 point at it, and K-14's reference point says what happened. New ids: R-105, C-24, E-77, T-118..T-121, D-58..D-60. The fixture goldens gain the E-77 Note at the build.
v1.25	A provenance manifest and hazard-to-control traceability (`docs/proposals/PROPOSAL_v1.25_provenance_manifest_and_risk_controls.md`, 2026-11-05; D-61..D-65 confirmed on the recommended branches, same day; `docs/plans/ISO13485_READINESS_PLAN.md` §3.2, §3.3). Evidence: on a copy of this repository a line appended to `src/speccheck/report.py` after the test run left both v1.24 reports byte-identical, so a report could not identify the code it verified. R-106, C-25, I-020: `check --manifest` writes `speccheck.manifest.json` (tool version, options, SHA-256 and length of every file read and of both reports, `tree_sha256`, `generated_at` only from `SOURCE_DATE_EPOCH`), renamed into place with the two reports. R-107, C-26: `speccheck verify` re-hashes and exits `1` naming each changed or missing file. R-108, C-27: the CONTROLS marker (C-01, `SpecId.controls`) and the `VERIFIED/NOT_VERIFIED/UNCONTROLLED` hazard roll-up in `controls`, `hazards`, C-08's `## 10. Hazards`, `explain`'s `controls:` line and the summary suffix `; hazards: <V>/<H> verified`. R-109, C-28: `--risk-file`; R-15 gains one condition, an `UNCONTROLLED` hazard. I-001 admits the third file; I-021 keeps every run that uses none of it byte-identical to v1.24; K-20; E-78..E-85; T-122..T-129; §0 gains two non-goals (no risk management, no signatures). Settled while folding, beyond the proposal's draft rows: the manifest joins §3.1's temp-and-rename sequence instead of being written after it (so the reports and the manifest are written together or not at all); `--risk-file` is `check`-only, since `explain`'s `controls:` line needs no titles and a marked id's hazards cannot be `UNCONTROLLED` unless it is retired; a symlink skipped by E-11 is not listed in `inputs` (it was never read); `verify` caps a manifest at K-02's 2 MiB (E-83); `impact` parses and ignores the marker. Found while building (F-801): C-27's Notes line said each of E-78..E-81 appears at most once per run, while E-78 and E-80 say one Note per such id; C-27 now agrees with the E rows and states that `impact` emits none of them. Built 2026-11-05 in three waves on `spec/v1.25` (`SPEC_BUILD_REPORT.md` §0p): W1 hazards, W2 manifest and `verify`, W3 proof; the two id-count pins now read 334.